

Phase 4 — Descriptive Analysis: Weather Data

Descriptive and summary-statistics exhibits:

- **Tier 1 — internal QA and exploratory exhibits:** establish that the panel is complete, plausible, and contains meaningful temporal and spatial variation. Outputs are fast & minimally-styled.
- **Tier 2 — polished explanatory exhibits:** communicate the project's core weather story: long-run winter warming and county-level anomalously warm winter shocks.

PRISM is the primary weather dataset. ERA5 is a secondary robustness source and does not require a fully duplicated polished exhibit suite unless a direct PRISM–ERA5 comparison is useful.

0. Setup, definitions, and analytical conventions

Establish paths, imports, dataset configuration, labels, output directories, and common plotting settings. Document all analytical conventions before generating exhibits.

Required conventions:

- Winter is December–February (months DJF), with December assigned to the following winter year.
- With PRISM beginning in January 1981, complete DJF winters span 1982–2025.
- Construct one observation per county-winter before producing winter exhibits.
- Sum day counts and accumulated totals across winter months.
- Calculate winter temperature means using an appropriate mean, preferably weighted by observed days.
- Treat monthly variances and state variables such as snow depth according to explicitly documented aggregation rules.
- Unless otherwise specified, national and state summaries are unweighted means across counties.
- Keep freezing days ($TMIN < 32^{\circ}F$) separate from extreme-cold days ($TMIN < 0^{\circ}F$).

```
In [36]: from dataclasses import dataclass
        from pathlib import Path
        import calendar
        import io
```

```

import os
import warnings

import matplotlib.pyplot as plt
from matplotlib.backends.backend_pdf import PdfPages
from IPython.display import SVG, display
import numpy as np
import pandas as pd

# Make Matplotlib output render beneath each notebook cell while figures
# are also saved separately as vector PDFs.
try:
    get_ipython().run_line_magic("matplotlib", "inline")
except NameError:
    pass # Allows the same helpers to run from a regular Python process.

def resolve_repo_root():
    """Resolve the project checkout on either Kodama or a local Mac.

    Unlike a .py file, a notebook has no reliable __file__. Jupyter's current
    working directory depends on how the server/editor was launched, so search
    plausible roots and accept the first directory with this repo's structure.
    Set COLLISIONS_WEATHER_ROOT to override discovery explicitly.
    """
    cwd = Path.cwd().resolve()
    candidates = []
    env_root = os.environ.get("COLLISIONS_WEATHER_ROOT")
    if env_root:
        candidates.append(Path(env_root).expanduser())
    # Prefer the documented production roots before generic cwd discovery.
    candidates.extend([
        Path("/mnt/data_d/Dropbox/Research/AnimalCollisionsWeather"),
        Path("/Users/wendyhtw/Documents/CAPP ('25-'27)/Q4 - Summer'26/EPIC/Repos/collisions-and-climate"),
    ])
    for base in (cwd, *cwd.parents):
        candidates.extend([
            base,
            base / "collisions-and-climate",
            base / "Repos" / "collisions-and-climate",
        ])
    checked = []
    for candidate in candidates:

```

```

candidate = candidate.resolve()
if candidate in checked:
    continue
checked.append(candidate)
required = [
    candidate / "codePYTHON",
    candidate / "dataCSV" / "PRISM" / "prism_county_month.csv",
    candidate / "dataCSV" / "PRISM" / "prism_derived_weather_vars.csv",
]
if required[0].is_dir() and all(path.is_file() for path in required[1:]):
    return candidate
attempted = "\n - ".join(str(path) for path in checked)
raise FileNotFoundError(
    "Could not locate the collisions-and-climate project root. Checked:\n - "
    + attempted
    + "\nSet COLLISIONS_WEATHER_ROOT to the project directory and rerun."
)

REPO_ROOT = resolve_repo_root()
DATA_DIR = REPO_ROOT / "dataCSV"
TABLES_DIR = REPO_ROOT / "tables" / "weather" / "tier1"
FIGURES_DIR = REPO_ROOT / "figures" / "weather" / "tier1"
TABLES_DIR.mkdir(parents=True, exist_ok=True)
FIGURES_DIR.mkdir(parents=True, exist_ok=True)

ID_COLS = ["geoid", "state_fips", "county_fips", "county_name"]
WINTER_MONTHS = (12, 1, 2)
EXPECTED_YEARS = set(range(1981, 2026))
PRIMARY_TREND_VARS = [
    "mean_temp_c",
    "days_extremely_cold",
    "days_below_freezing_32f",
    "freeze_thaw_days",
]
SPAGHETTI_VARS = ["mean_temp_c"]
OUTLIER_Z_THRESHOLD = 5.0

STATE_FIPS_TO_NAME = {
    "01": "Alabama", "02": "Alaska", "04": "Arizona", "05": "Arkansas", "06": "California",
    "08": "Colorado", "09": "Connecticut", "10": "Delaware", "11": "District of Columbia",
    "12": "Florida", "13": "Georgia", "15": "Hawaii", "16": "Idaho", "17": "Illinois",
    "18": "Indiana", "19": "Iowa", "20": "Kansas", "21": "Kentucky", "22": "Louisiana",

```

```

    "23": "Maine", "24": "Maryland", "25": "Massachusetts", "26": "Michigan",
    "27": "Minnesota", "28": "Mississippi", "29": "Missouri", "30": "Montana",
    "31": "Nebraska", "32": "Nevada", "33": "New Hampshire", "34": "New Jersey",
    "35": "New Mexico", "36": "New York", "37": "North Carolina", "38": "North Dakota",
    "39": "Ohio", "40": "Oklahoma", "41": "Oregon", "42": "Pennsylvania",
    "44": "Rhode Island", "45": "South Carolina", "46": "South Dakota",
    "47": "Tennessee", "48": "Texas", "49": "Utah", "50": "Vermont", "51": "Virginia",
    "53": "Washington", "54": "West Virginia", "55": "Wisconsin", "56": "Wyoming",
}
VARIABLE_LABELS = {
    "mean_temp_c": "Mean winter temperature (°C)",
    "days_extremely_cold": "Extreme-cold days per winter (TMIN < 0°F)",
    "days_below_freezing_32f": "Freezing days per winter (TMIN < 32°F)",
    "freeze_thaw_days": "Freeze-thaw days per winter",
    "ppt_total": "Total precipitation (mm)",
    "heating_degree_days": "Heating degree days",
    "cooling_degree_days": "Cooling degree days",
}
VARIABLE_UNITS = {
    "mean_temp_c": "°C", "days_extremely_cold": "days",
    "days_below_freezing_32f": "days", "freeze_thaw_days": "days",
    "ppt_total": "mm", "heating_degree_days": "degree-days",
    "cooling_degree_days": "degree-days",
}

@dataclass(frozen=True)
class DatasetConfig:
    name: str
    monthly_path: Path
    derived_path: Path
    is_primary: bool = False

PRISM_CONFIG = DatasetConfig(
    "PRISM", DATA_DIR / "PRISM" / "prism_county_month.csv",
    DATA_DIR / "PRISM" / "prism_derived_weather_vars.csv", True,
)
ERA5_CONFIG = DatasetConfig(
    "ERA5", DATA_DIR / "ERA5" / "era5_county_month.csv",
    DATA_DIR / "ERA5" / "era5_derived_weather_vars.csv", False,
)
config = PRISM_CONFIG
print(f"Dataset: {config.name}")

```



```

print(f"Repository root: {REPO_ROOT}")
print(f"Monthly input: {config.monthly_path}")
print(f"Derived input: {config.derived_path}")
print(f"Table outputs: {TABLES_DIR}")
print(f"Figure outputs: {FIGURES_DIR}")

def variable_label(variable):
    return VARIABLE_LABELS.get(variable, variable.replace("_", " ").title())

def state_label(fips):
    code = str(fips).zfill(2)
    return f"{code} - {STATE_FIPS_TO_NAME.get(code, '(unmapped)')}"

def display_figure_inline(fig):
    """Embed a vector SVG in notebook output, independent of backend."""
    buffer = io.StringIO()
    fig.savefig(buffer, format="svg", bbox_inches="tight")
    display(SVG(data=buffer.getvalue()))
    buffer.close()

def save_figure_pdf(fig, filename):
    path = FIGURES_DIR / filename
    fig.savefig(path, format="pdf", bbox_inches="tight")
    display_figure_inline(fig)
    plt.close(fig)
    print(f"Saved {path}")
    return path

def _format_table_values(table):
    formatted_table = table.copy()
    for col in formatted_table.select_dtypes(include="number"):
        formatted_table[col] = formatted_table[col].map(
            lambda x: "" if pd.isna(x) else f"{x:,.3f}" if not float(x).is_integer() else f"{x:,.0f}"
        )
    return formatted_table

def save_table_pdf(table, path, title, rows_per_page=32, fontsize=7):
    """Save a DataFrame as one or more vector-PDF table pages."""
    formatted_table = _format_table_values(table)
    with PdfPages(path) as pdf:
        for start in range(0, max(len(formatted_table), 1), rows_per_page):
            page = formatted_table.iloc[start:start + rows_per_page]

```

```

fig_height = max(3.0, 0.30 * (len(page) + 4))
fig, ax = plt.subplots(figsize=(7.5, fig_height))
ax.axis("off")
ax.set_title(title, loc="left", fontsize=12, pad=12)
if page.empty:
    ax.text(
        0.01, 0.88,
        "No rows available. Check the preceding sample-construction diagnostics.",
        transform=ax.transAxes, ha="left", va="top", fontsize=10,
    )
    pdf.savefig(fig, bbox_inches="tight")
    display_figure_inline(fig)
    plt.close(fig)
    continue
artist = ax.table(
    cellText=page.values, colLabels=page.columns,
    loc="upper left", cellLoc="right", colLoc="center",
    bbox=[0, 0, 1, 0.94],
)
artist.auto_set_font_size(False)
artist.set_fontsize(fontsize)
artist.auto_set_column_width(col=list(range(len(page.columns))))
pdf.savefig(fig, bbox_inches="tight")
display_figure_inline(fig)
plt.close(fig)
print(f"Saved {path}")
return path

```

Dataset: PRISM

Repository root: /mnt/data_d/Dropbox/Research/AnimalCollisionsWeather

Monthly input: /mnt/data_d/Dropbox/Research/AnimalCollisionsWeather/dataCSV/PRISM/prism_county_month.csv

Derived input: /mnt/data_d/Dropbox/Research/AnimalCollisionsWeather/dataCSV/PRISM/prism_derived_weather_variables.csv

Table outputs: /mnt/data_d/Dropbox/Research/AnimalCollisionsWeather/tables/weather/tier1

Figure outputs: /mnt/data_d/Dropbox/Research/AnimalCollisionsWeather/figures/weather/tier1

1. Load and validate the county-year-month panel

Load the raw monthly and derived-variable files, verify key uniqueness, compare overlapping QA fields, and merge on county, year, and month. Preserve clear diagnostics for duplicates, unmatched rows, missing values, and incomplete months.

```

In [37]: def load_weather_panel(dataset_config):
    dtype = {"geoid": str, "state_fips": str, "county_fips": str}
    monthly = pd.read_csv(dataset_config.monthly_path, dtype=dtype)
    derived = pd.read_csv(dataset_config.derived_path, dtype=dtype)

    keys = ["geoid", "year", "month"]
    duplicate_monthly = int(monthly.duplicated(keys).sum())
    duplicate_derived = int(derived.duplicated(keys).sum())
    if duplicate_monthly or duplicate_derived:
        raise ValueError(
            f"Duplicate keys: monthly={duplicate_monthly:}, derived={duplicate_derived:}"
        )

    redundant_ids = [c for c in ID_COLS if c != "geoid"]
    qa_cols = [
        c for c in ("n_days", "expected_days", "is_incomplete", "dataset_types")
        if c in monthly.columns and c in derived.columns
    ]
    qa_mismatches = {}
    if qa_cols:
        check = monthly[keys + qa_cols].merge(
            derived[keys + qa_cols], on=keys, how="inner",
            suffixes=("_monthly", "_derived"), validate="one_to_one",
        )
        for col in qa_cols:
            left, right = check[f"{col}_monthly"], check[f"{col}_derived"]
            qa_mismatches[col] = int((left.fillna("<NA>") != right.fillna("<NA>")).sum())

    slim = derived.drop(columns=redundant_ids + qa_cols, errors="ignore")
    merged = monthly.merge(
        slim, on=keys, how="outer", indicator=True, validate="one_to_one"
    )
    merge_counts = merged["_merge"].value_counts().to_dict()
    merged = merged.drop(columns="_merge")

    diagnostics = {
        "dataset": dataset_config.name,
        "monthly_rows": len(monthly),
        "derived_rows": len(derived),
        "merged_rows": len(merged),
        "counties": merged["geoid"].nunique(),
    }

```

```

    "first_year": int(merged["year"].min()),
    "last_year": int(merged["year"].max()),
    "duplicate_monthly_keys": duplicate_monthly,
    "duplicate_derived_keys": duplicate_derived,
    "monthly_only_rows": int(merge_counts.get("left_only", 0)),
    "derived_only_rows": int(merge_counts.get("right_only", 0)),
    "incomplete_months": int(merged.get("is_incomplete", pd.Series(False, index=merged.index)).fillna(
        **{f"qa_mismatch_{key}": value for key, value in qa_mismatches.items()}),
    }
    if diagnostics["monthly_only_rows"] or diagnostics["derived_only_rows"]:
        warnings.warn("Monthly and derived files do not match completely; inspect coverage table.")
    return merged, diagnostics

df, load_diagnostics = load_weather_panel(config)
display(pd.DataFrame([load_diagnostics]))
display(df.head())

```

	dataset	monthly_rows	derived_rows	merged_rows	counties	first_year	last_year	duplicate_monthly_keys	duplicate_derived
0	PRISM	1678320	1678320	1678320	3108	1981	2025	0	

	geoid	state_fips	county_fips	county_name	year	month	n_days	ppt_total	tmean_mean	tmin_mean	...	days_extremely
0	01001	01	001	Autauga	1981	1	31	41.513720	4.920923	-1.935180	...	
1	01001	01	001	Autauga	1981	2	28	183.999778	9.292531	2.905967	...	
2	01001	01	001	Autauga	1981	3	31	243.287245	11.805146	4.266983	...	
3	01001	01	001	Autauga	1981	4	30	39.155931	19.964888	13.403788	...	
4	01001	01	001	Autauga	1981	5	31	81.438487	20.525748	14.197903	...	

5 rows × 27 columns

2. Construct the county-winter panel

Convert the county-month panel into one observation per county-winter using variable-specific aggregation rules. Exclude or explicitly flag partial winters so that incomplete boundary years are not presented as full winters.

```

In [38]: NON_WEATHER_COLS = {
    *ID_COLS, "year", "month", "winter_year", "n_days", "expected_days",
    "is_incomplete", "complete_winter", "dataset_types",
}
COUNT_OR_TOTAL_PATTERNS = (
    "days_", "heating_degree_days", "cooling_degree_days",
    "ppt_total", "total_snowfall",
)
VARIANCE_TO_MEAN = {
    "tmean_variance_c2": "tmean_mean",
    "tmin_variance_c2": "tmin_mean",
    "tmax_variance_c2": "tmax_mean",
}

def _numeric_weather_columns(frame):
    return [
        c for c in frame.columns
        if c not in NON_WEATHER_COLS and pd.api.types.is_numeric_dtype(frame[c])
    ]

def _is_total_variable(variable):
    return variable.startswith(COUNT_OR_TOTAL_PATTERNS)

def construct_county_winter_panel(monthly):
    """Collapse county-months to complete county-winters with vectorized rules."""
    work = monthly.copy()
    work["winter_year"] = work["year"] + (work["month"] == 12).astype(int)
    work = work[work["month"].isin(WINTER_MONTHS)].copy()

    group_keys = ID_COLS + ["winter_year"]
    completeness = (
        work.groupby(["geoid", "winter_year"], as_index=False)
        .agg(n_months=("month", "nunique"), observed_days=("n_days", "sum"))
    )
    completeness["expected_winter_days"] = completeness["winter_year"].map(
        lambda y: 31 + calendar.monthrange(int(y), 2)[1] + 31
    )
    completeness["complete_winter"] = (
        completeness["n_months"].eq(3)
        & completeness["observed_days"].eq(completeness["expected_winter_days"])
    )

```

```

if "is_incomplete" in work:
    monthly_flags = (
        work.groupby(["geoid", "winter_year"], as_index=False)["is_incomplete"]
        .any().rename(columns={"is_incomplete": "has_flagged_month"})
    )
    completeness = completeness.merge(
        monthly_flags, on=["geoid", "winter_year"], validate="one_to_one"
    )
    completeness["complete_winter"] &= ~completeness["has_flagged_month"]

complete = work.merge(
    completeness[["geoid", "winter_year", "complete_winter"]],
    on=["geoid", "winter_year"], validate="many_to_one",
)
complete = complete[complete["complete_winter"]].copy()
grouped = complete.groupby(group_keys, sort=True, dropna=False)
county_winter = grouped["n_days"].sum().rename("n_days").reset_index()

variables = _numeric_weather_columns(complete)
variance_vars = [v for v in VARIANCE_TO_MEAN if v in variables and VARIANCE_TO_MEAN[v] in complete]
total_vars = [v for v in variables if _is_total_variable(v)]
mean_vars = [v for v in variables if v not in total_vars and v not in variance_vars]

# Counts and accumulated quantities are additive across Dec/Jan/Feb.
if total_vars:
    totals = grouped[total_vars].sum(min_count=1).reset_index()
    county_winter = county_winter.merge(totals, on=group_keys, validate="one_to_one")

# Monthly means are combined using observed county-month day counts.
for variable in mean_vars:
    valid = complete[variable].notna() & complete["n_days"].gt(0)
    temp = complete.loc[valid, group_keys].copy()
    temp["_weighted_value"] = complete.loc[valid, variable] * complete.loc[valid, "n_days"]
    temp["_weight"] = complete.loc[valid, "n_days"]
    weighted = temp.groupby(group_keys, as_index=False)[["_weighted_value", "_weight"]].sum()
    weighted[variable] = weighted["_weighted_value"] / weighted["_weight"]
    county_winter = county_winter.merge(
        weighted[group_keys + [variable]], on=group_keys, how="left", validate="one_to_one"
    )

# Combine monthly daily-sample variances exactly:
# SS = sum[(n_m-1)s_m^2 + n_m*mean_m^2] - (sum[n_m*mean_m])^2/N.

```

```

for variance_col in variance_vars:
    mean_col = VARIANCE_TO_MEAN[variance_col]
    valid = complete[ [*group_keys, "n_days", mean_col, variance_col]].dropna()
    valid = valid[valid["n_days"].gt(0)].copy()
    valid["_sum_x"] = valid["n_days"] * valid[mean_col]
    valid["_sum_x2"] = (
        (valid["n_days"] - 1) * valid[variance_col]
        + valid["n_days"] * valid[mean_col].pow(2)
    )
    combined = (
        valid.groupby(group_keys, as_index=False)
        .agg(_n=("n_days", "sum"), _sum_x=("_sum_x", "sum"), _sum_x2=("_sum_x2", "sum"))
    )
    numerator = combined["_sum_x2"] - combined["_sum_x"].pow(2) / combined["_n"]
    combined[variance_col] = numerator.clip(lower=0) / (combined["_n"] - 1)
    combined.loc[combined["_n"].le(1), variance_col] = np.nan
    county_winter = county_winter.merge(
        combined[group_keys + [variance_col]],
        on=group_keys, how="left", validate="one_to_one"
    )

winter_diagnostics = {
    "candidate_county_winters": len(completeness),
    "complete_county_winters": int(completeness["complete_winter"].sum()),
    "excluded_incomplete_county_winters": int((~completeness["complete_winter"]).sum()),
    "first_complete_winter": (
        int(county_winter["winter_year"].min()) if not county_winter.empty else None
    ),
    "last_complete_winter": (
        int(county_winter["winter_year"].max()) if not county_winter.empty else None
    ),
}
return county_winter, completeness, winter_diagnostics

county_winter, winter_completeness, winter_diagnostics = construct_county_winter_panel(df)
display(pd.DataFrame([winter_diagnostics]))
display(county_winter.head())

```

	candidate_county_winters	complete_county_winters	excluded_incomplete_county_winters	first_complete_winter	last_complete
0	142968	136752	6216	1982	

	geoid	state_fips	county_fips	county_name	winter_year	n_days	ppt_total	days_extremely_cold	days_below_freezing_32f
0	01001	01	001	Autauga	1982	90	533.936134	0	33
1	01001	01	001	Autauga	1983	90	495.568865	0	24
2	01001	01	001	Autauga	1984	91	400.920770	0	39
3	01001	01	001	Autauga	1985	90	314.749628	2	42
4	01001	01	001	Autauga	1986	90	200.896798	0	41

5 rows × 23 columns

Tier 1 — Internal QA and exploratory exhibits

Tier 1 checks whether the weather panel is complete, internally coherent, and within plausible ranges. It also establishes whether the data contain meaningful variation across counties, states, and winters. These exhibits are intended primarily for internal review.

3. Data coverage and integrity table

Report the dataset, observation period, number of counties, number of county-months and county-winters, missing or incomplete records, merge mismatches, duplicate keys, and complete DJF winters retained. This table establishes the analysis sample before any substantive summaries are interpreted.

```
In [39]: coverage = {
    **load_diagnostics,
    **winter_diagnostics,
    "expected_calendar_years": f"{min(EXPECTED_YEARS)}--{max(EXPECTED_YEARS)}",
    "observed_calendar_years": f"{df['year'].min()}--{df['year'].max()}",
}
```



```
        "observed_complete_winters": (  
            f"{county_winter['winter_year'].min()}-{county_winter['winter_year'].max()}"  
        ),  
    }  
coverage_table = pd.DataFrame(  
    [{"metric": key, "value": value} for key, value in coverage.items()  
)  
coverage_table.to_csv(TABLES_DIR / f"{config.name.lower()}_tier1_coverage_integrity.csv", index=False)  
save_table_pdf(  
    coverage_table,  
    TABLES_DIR / f"{config.name.lower()}_tier1_coverage_integrity.pdf",  
    f"{config.name}: Tier 1 data coverage and integrity",  
)  
display(coverage_table)
```

PRISM: Tier 1 data coverage and integrity

metric	value
dataset	PRISM
monthly_rows	1678320
derived_rows	1678320
merged_rows	1678320
counties	3108
first_year	1981
last_year	2025
duplicate_monthly_keys	0
duplicate_derived_keys	0
monthly_only_rows	0
derived_only_rows	0
incomplete_months	0
qa_mismatch_n_days	0
qa_mismatch_expected_days	0
qa_mismatch_is_incomplete	0
qa_mismatch_dataset_types	0
candidate_county_winters	142968
complete_county_winters	136752
excluded_incomplete_county_winters	6216
first_complete_winter	1982
last_complete_winter	2025
expected_calendar_years	1981-2025
observed_calendar_years	1981-2025
observed_complete_winters	1982-2025

Saved /mnt/data_d/Dropbox/Research/AnimalCollisionsWeather/tables/weather/tier1/prism_tier1_coverage_integrity.pdf

	metric	value
0	dataset	PRISM
1	monthly_rows	1678320
2	derived_rows	1678320
3	merged_rows	1678320
4	counties	3108
5	first_year	1981
6	last_year	2025
7	duplicate_monthly_keys	0
8	duplicate_derived_keys	0
9	monthly_only_rows	0
10	derived_only_rows	0
11	incomplete_months	0
12	qa_mismatch_n_days	0
13	qa_mismatch_expected_days	0
14	qa_mismatch_is_incomplete	0
15	qa_mismatch_dataset_types	0
16	candidate_county_winters	142968
17	complete_county_winters	136752
18	excluded_incomplete_county_winters	6216
19	first_complete_winter	1982
20	last_complete_winter	2025
21	expected_calendar_years	1981–2025
22	observed_calendar_years	1981–2025
23	observed_complete_winters	1982–2025

4. Pooled summary-statistics table

For every raw and derived weather variable, report the unit, observation count, mean, standard deviation, minimum, p1, p5, median, p95, p99, and maximum across all county-month observations. Use the tails to identify values that require investigation.

```
In [40]: SUMMARY_QUANTILES = [0.01, 0.05, 0.50, 0.95, 0.99]

def summarize_long(frame, group_cols=None):
    group_cols = list(group_cols or [])
    variables = _numeric_weather_columns(frame)
    records = []
    groups = [(), frame] if not group_cols else frame.groupby(group_cols, dropna=False)
    for keys, group in groups:
        keys = keys if isinstance(keys, tuple) else (keys,)
        group_values = dict(zip(group_cols, keys))
        for variable in variables:
            values = group[variable].dropna()
            record = {
                **group_values,
                "variable": variable,
                "unit": VARIABLE_UNITS.get(variable, ""),
                "n": values.count(),
                "mean": values.mean(),
                "sd": values.std(),
                "min": values.min(),
                "p01": values.quantile(0.01),
                "p05": values.quantile(0.05),
                "median": values.quantile(0.50),
                "p95": values.quantile(0.95),
                "p99": values.quantile(0.99),
                "max": values.max(),
            }
            records.append(record)
    return pd.DataFrame(records)

summary_pooled = summarize_long(df)
summary_pooled.to_csv(TABLES_DIR / f"{config.name.lower()}_summary_pooled.csv", index=False)
save_table_pdf(
    summary_pooled,
```

```

TABLES_DIR / f"{config.name.lower()}_summary_pooled.pdf",
f"{config.name}: pooled county-month summary statistics",
rows_per_page=24,
)
display(summary_pooled)

```

PRISM: pooled county-month summary statistics

variable	unit	n	mean	sd	min	p01	p05	median	p95	p99	max
ppt_total	mm	1,678,320	84.114	63.442	0	0.676	7.320	72.6192	100.2472	88.2461	282.293
tmean_mean		1,678,320	12.574	9.883	-25.086	-10.581	-4.619	13.407	26.740	28.770	36.389
tmin_mean		1,678,320	6.427	9.669	-31.219	-16.077	-9.892	6.705	20.951	23.080	28.953
tmax_mean		1,678,320	18.722	10.244	-19.476	-5.270	0.345	20.077	32.727	35.357	44.151
tdmean_mean		1,678,320	6.306	9.521	-28.675	-14.023	-9.125	6.232	20.735	22.826	25.848
vpdmin_mean		1,678,320	1.078	1.174	0	0.141	0.239	0.758	2.970	6.158	25.170
vpdmax_mean		1,678,320	14.444	8.847	0.405	1.369	2.573	13.586	30.618	41.253	77.406
days_extremely_cold	days	1,678,320	0.557	2.322	0	0	0	0	4	13	31
days_below_freezing_32f	days	1,678,320	8.685	10.937	0	0	0	2	30	31	31
freeze_thaw_days	days	1,678,320	6.682	8.269	0	0	0	2	23	27	31
days_precip_above_10mm		1,678,320	2.650	2.256	0	0	0	2	7	9	26
heating_degree_days	degree-days	1,678,320	32.0862	38.201	0	0	0	159.0226	94.3958	73.5141	346.004
cooling_degree_days	degree-days	1,678,320	58.315	87.131	0	0	0	7.8102	57.9503	22.056	559.731
mean_temp_c	°C	1,678,320	12.574	9.883	-25.086	-10.581	-4.619	13.407	26.740	28.770	36.389
tmean_variance_c2		1,678,320	18.998	14.560	0.081	1.051	2.628	15.819	46.510	69.677	166.956
tmin_variance_c2		1,678,320	19.191	14.947	0.105	0.851	2.441	15.799	47.411	72.928	183.712
tmax_variance_c2		1,678,320	25.191	18.720	0.078	1.987	4.012	21.174	60.516	87.622	170.805

Saved /mnt/data_d/Dropbox/Research/AnimalCollisionsWeather/tables/weather/tier1/prism_summary_pooled.pdf

	variable	unit	n	mean	sd	min	p01	p05	median	p
0	ppt_total	mm	1678320	84.113505	63.442325	0.000000	0.675782	7.320352	72.619359	200.247
1	tmean_mean		1678320	12.574239	9.883187	-25.086144	-10.580801	-4.619237	13.407224	26.7395
2	tmin_mean		1678320	6.427151	9.668881	-31.218806	-16.077472	-9.891864	6.705057	20.9514
3	tmax_mean		1678320	18.721636	10.243949	-19.476206	-5.269822	0.345155	20.077369	32.7266
4	tdmean_mean		1678320	6.306368	9.520643	-28.674932	-14.022960	-9.125288	6.231510	20.7352
5	vpdmin_mean		1678320	1.078322	1.174098	0.000000	0.141476	0.238731	0.757871	2.9703
6	vpdmax_mean		1678320	14.443675	8.847299	0.405437	1.369489	2.573106	13.586212	30.6178
7	days_extremely_cold	days	1678320	0.557025	2.322020	0.000000	0.000000	0.000000	0.000000	4.0000
8	days_below_freezing_32f	days	1678320	8.684700	10.936999	0.000000	0.000000	0.000000	2.000000	30.0000
9	freeze_thaw_days	days	1678320	6.682357	8.269132	0.000000	0.000000	0.000000	2.000000	23.0000
10	days_precip_above_10mm		1678320	2.650242	2.256015	0.000000	0.000000	0.000000	2.000000	7.0000
11	heating_degree_days	degree-days	1678320	232.085527	238.200710	0.000000	0.000000	0.000000	159.021916	694.395
12	cooling_degree_days	degree-days	1678320	58.315041	87.131075	0.000000	0.000000	0.000000	7.810028	257.9502
13	mean_temp_c	°C	1678320	12.574239	9.883187	-25.086144	-10.580801	-4.619237	13.407224	26.7395
14	tmean_variance_c2		1678320	18.997847	14.560158	0.080759	1.051163	2.628350	15.818945	46.510
15	tmin_variance_c2		1678320	19.190507	14.946581	0.104567	0.851065	2.440800	15.798950	47.4113
16	tmax_variance_c2		1678320	25.190853	18.719902	0.077850	1.986968	4.011561	21.173797	60.5157

5. Monthly summary-statistics table

Report the same statistics separately by calendar month. This reveals expected seasonality and helps identify month-specific values that are implausible or inconsistent with neighboring months.

```
In [41]: summary_by_month = summarize_long(df, ["month"])
summary_by_month.to_csv(
```

```
TABLES_DIR / f"{config.name.lower()}_summary_by_calendar_month.csv", index=False
)
save_table_pdf(
    summary_by_month,
    TABLES_DIR / f"{config.name.lower()}_summary_by_calendar_month.pdf",
    f"{config.name}: summary statistics by calendar month",
    rows_per_page=30,
)
display(summary_by_month.head(24))
```

PRISM: summary statistics by calendar month

month	variable	unit	n	mean	sd	min	p01	p05	median	p95	p99	max
1	ppt_total	mm	139,860	59.302	53.659	0	0.519	4.242	54.174	81.574	292.704	908.595
1	tmean_mean		139,860	0.248	6.688	-25.086	-15.219	-10.774	0.239	10.896	14.957	22.476
1	tmin_mean		139,860	-5.282	6.471	-13.219	-21.040	-16.199	-5.103	4.872	9.062	19.001
1	tmax_mean		139,860	5.777	7.118	19.476	-9.591	-5.517	5.627	17.273	21.349	27.192
1	tdmean_mean		139,860	-4.555	6.106	28.675	-18.024	-14.027	-4.980	6.022	10.297	18.505
1	vpdmin_mean		139,860	0.519	0.330	0.003	0.131	0.184	0.453	1.055	1.773	5.781
1	vpdmax_mean		139,860	5.343	3.333	0.405	0.876	1.336	4.668	11.616	15.282	24.068
1	days_extremely_cold	days	139,860	2.622	4.778	0	0	0	0	13	21	31
1	days_below_freezing_32f	days	139,860	23.302	8.604	0	0	4	26	31	31	31
1	freeze_thaw_days	days	139,860	15.196	7.052	0	0	2	16	26	29	31
1	days_precip_above_10mm	days	139,860	2.137	2.297	0	0	0	2	6	9	26
1	heating_degree_daysdeg	degree-days	139,860	51.440	105.719	0.736	21.310	83.726	50.953	202.328	1,040.126	3,346.004
1	cooling_degree_daysdeg	degree-days	139,860	0.785	4.588	0	0	0	0	3.504	18.597	130.052
1	mean_temp_c	°C	139,860	0.248	6.688	-25.086	-15.219	-10.774	0.239	10.896	14.957	22.476
1	tmean_variance_c2		139,860	29.682	16.743	0.907	3.804	8.155	26.796	51.785	78.464	166.956
1	tmin_variance_c2		139,860	32.111	19.127	0.823	4.171	8.179	28.293	59.966	89.569	183.712
1	tmax_variance_c2		139,860	35.505	18.783	1.010	5.021	9.800	32.548	59.264	90.585	169.934
2	ppt_total	mm	139,860	59.017	52.086	0	0.778	4.886	53.464	87.165	286.554	878.315
2	tmean_mean		139,860	2.209	6.934	-21.934	-13.973	-9.589	2.373	13.096	17.142	24.089
2	tmin_mean		139,860	-3.679	6.711	-27.162	-19.987	-15.346	-3.425	6.855	11.269	20.600
2	tmax_mean		139,860	8.096	7.338	16.756	-8.213	-3.986	8.165	19.669	23.477	29.758
2	tdmean_mean		139,860	-3.158	6.397	24.107	-17.186	-13.371	-3.503	7.841	12.342	19.644
2	vpdmin_mean		139,860	0.594	0.387	0.006	0.148	0.207	0.514	1.193	2.103	6.549
2	vpdmax_mean		139,860	6.814	3.943	0.514	1.124	1.820	6.187	14.041	18.458	30.651
2	days_extremely_cold	days	139,860	1.890	4.021	0	0	0	0	11	18	28
2	days_below_freezing_32f	days	139,860	19.085	8.837	0	0	2	22	28	29	29
2	freeze_thaw_days	days	139,860	13.439	6.722	0	0	1	14	24	27	29

days	precip_above_10mm	139,860	2.118	2.178	0	0	0	2	6	9	23
2	heating_degree_daysdegree-days	139,860	56.851	92.090	0	54.776	56.285	50.947	35.918	907.716	127.490
2	cooling_degree_daysdegree-days	139,860	1.648	7.470	0	0	0	0	8.868	37.287	161.145

PRISM: summary statistics by calendar month

month	variable	unit	n	mean	sd	min	p01	p05	median	p95	p99	max
2	mean_temp_c	°C	139,860	2.209	6.934	-21.934	13.973	9.589	2.373	13.096	17.142	24.089
2	tmean_variance_c2		139,860	30.419	19.754	0.688	3.557	8.126	25.595	71.448	99.614	53.859
2	tmin_variance_c2		139,860	30.297	19.954	0.887	3.983	7.725	25.698	70.919	100.023	77.572
2	tmax_variance_c2		139,860	39.260	24.518	1.008	4.864	10.248	33.940	90.058	123.801	67.287
3	ppt_total	mm	139,860	82.591	59.981	0	1.985	9.945	71.936	193.633	274.971	99.133
3	tmean_mean		139,860	7.035	5.931	-12.976	-6.039	2.658	6.975	16.611	19.825	24.967
3	tmin_mean		139,860	0.760	5.665	-18.600	-11.925	-8.364	0.622	10.202	14.053	21.070
3	tmax_mean		139,860	13.310	6.395	-7.444	-0.482	2.753	13.380	23.305	26.134	31.491
3	tdmean_mean		139,860	0.436	5.679	-17.304	-10.965	-8.103	-0.009	10.611	14.350	20.684
3	vpdmin_mean		139,860	0.849	0.520	0.044	0.200	0.296	0.748	1.672	2.808	7.885
3	vpdmax_mean		139,860	10.150	4.656	0.734	2.300	3.667	9.694	18.274	23.174	36.970
3	days_extremely_cold	days	139,860	0.417	1.502	0	0	0	0	3	8	21
3	days_below_freezing_32f	days	139,860	14.656	10.090	0	0	0	14	30	31	31
3	freeze_thaw_days	days	139,860	12.717	8.372	0	0	0	13	26	30	31
3	days_precip_above_10mm		139,860	2.610	2.189	0	0	0	2	7	9	24
3	heating_degree_days	degree-days	139,860	55.825	17.345	0	26.910	1.968	52.705	50.730	55.529	70.582
3	cooling_degree_days	degree-days	139,860	5.578	14.887	0	0	0	0	31.103	76.894	209.507
3	mean_temp_c	°C	139,860	7.035	5.931	-12.976	-6.039	2.658	6.975	16.611	19.825	24.967
3	tmean_variance_c2		139,860	26.494	14.499	0.834	3.986	8.041	23.797	55.147	72.661	42.477
3	tmin_variance_c2		139,860	25.177	14.494	0.813	3.516	6.951	22.574	53.078	73.360	65.643
3	tmax_variance_c2		139,860	36.463	19.065	0.842	6.283	11.103	33.759	72.481	92.803	58.713
4	ppt_total	mm	139,860	87.394	58.241	0	1.812	1.889	78.221	192.685	278.938	60.502
4	tmean_mean		139,860	12.191	4.987	-5.339	1.227	4.093	12.224	20.164	22.661	27.484
4	tmin_mean		139,860	5.593	4.955	-12.523	-5.117	2.285	5.523	13.799	17.183	22.937
4	tmax_mean		139,860	18.789	5.235	0.710	7.188	10.113	18.972	26.817	29.136	35.708
4	tdmean_mean		139,860	4.551	5.596	-17.235	-8.258	4.466	4.466	14.003	17.015	21.656
4	vpdmin_mean		139,860	1.177	0.745	0	0.306	0.459	1.033	2.275	4.305	12.327

4	vpdmax_mean		139,860	14.461	4.979	2.201	5.841	7.530	13.982	23.280	30.223	47.305
4	days_extremely_cold	days	139,860	0.006	0.117	0	0	0	0	0	0	8
4	days_below_freezing_32f	days	139,860	6.124	7.288	0	0	0	3	22	28	30

PRISM: summary statistics by calendar month

month	variable	unit	n	mean	sd	min	p01	p05	median	p95	p99	max
4	freeze_thaw_days	days	139,860	5.911	6.930	0	0	0	3	21	27	30
4	days_precip_above_10mm		139,860	2.760	2.079	0	0	0	3	6	8	17
4	heating_degree_days	degree-days	139,860	202.579	28.461	0	3.129	1.793	89.859	27.237	13.214	10.159
4	cooling_degree_days	degree-days	139,860	18.323	29.018	0	0	0	6.135	78.906	35.776	74.532
4	mean_temp_c	°C	139,860	12.191	4.987	-5.339	1.227	4.093	12.224	20.164	22.661	27.484
4	tmean_variance_c2		139,860	21.345	10.438	0.430	4.557	7.953	19.483	41.508	53.917	84.290
4	tmin_variance_c2		139,860	19.133	8.688	0.540	3.741	6.746	18.076	35.030	44.424	69.793
4	tmax_variance_c2		139,860	31.326	16.564	0.363	4.789	9.222	28.655	63.908	80.670	115.717
5	ppt_total	mm	139,860	102.052	62.080	0	2.293	18.737	92.977	17.247	289.507	69.608
5	tmean_mean		139,860	17.445	4.386	0.443	6.908	10.108	17.534	24.265	26.065	30.811
5	tmin_mean		139,860	11.112	4.649	-6.338	0.103	3.293	11.231	18.405	20.864	25.572
5	tmax_mean		139,860	23.778	4.351	6.771	13.398	16.546	23.853	30.462	32.570	38.206
5	tdmean_mean		139,860	10.353	5.580	12.206	4.909	0.005	10.816	18.447	20.529	25.372
5	vpdmin_mean		139,860	1.306	1.075	0	0.200	0.397	1.078	2.843	6.074	15.526
5	vpdmax_mean		139,860	17.785	5.569	3.374	8.747	10.919	16.739	27.722	38.124	61.819
5	days_extremely_cold	days	139,860	0	0	0	0	0	0	0	0	0
5	days_below_freezing_32f	days	139,860	1.047	3.007	0	0	0	0	6	16	31
5	freeze_thaw_days	days	139,860	1.045	2.997	0	0	0	0	6	16	31
5	days_precip_above_10mm		139,860	3.308	2.296	0	0	0	3	7	9	17
5	heating_degree_days	degree-days	139,860	88.295	85.072	0	0	0	64.369	56.015	54.181	54.610
5	cooling_degree_days	degree-days	139,860	60.766	61.095	0	0	0	40.011	85.283	39.997	86.803
5	mean_temp_c	°C	139,860	17.445	4.386	0.443	6.908	10.108	17.534	24.265	26.065	30.811
5	tmean_variance_c2		139,860	15.103	7.894	0.272	2.502	4.565	13.642	30.113	37.948	57.145
5	tmin_variance_c2		139,860	14.588	7.112	0.322	2.717	5.106	13.369	28.200	35.918	50.790
5	tmax_variance_c2		139,860	21.040	11.959	0.088	2.735	5.064	19.211	43.140	56.552	91.305
6	ppt_total	mm	139,860	101.310	62.964	0	0.611	13.654	92.649	16.298	293.154	69.034
6	tmean_mean		139,860	22.060	3.897	5.752	11.545	14.864	22.481	27.528	29.065	32.939

6	tmin_mean	139,860	15.844	4.320	-1.341	4.041	7.692	16.340	21.905	23.558	27.096
6	tmax_mean	139,860	28.278	3.775	12.605	18.239	1.602	28.597	33.725	36.062	41.703
6	tdmean_mean	139,860	15.005	5.437	-9.843	2.194	3.275	16.029	21.719	22.993	25.848

PRISM: summary statistics by calendar month

month	variable	unit	n	mean	sd	min	p01	p05	median	p95	p99	max
6	vpdmin_mean		139,860	1.671	1.667	0	0.144	0.333	1.255	4.355	8.989	22.353
6	vpdmax_mean		139,860	22.209	7.092	4.742	1.110	3.828	20.766	35.753	46.887	74.964
6	days_extremely_cold	days	139,860	0	0	0	0	0	0	0	0	0
6	days_below_freezing_32f	days	139,860	0.080	0.656	0	0	0	0	0	3	22
6	freeze_thaw_days	days	139,860	0.080	0.656	0	0	0	0	0	3	22
6	days_precip_above_10mm		139,860	3.316	2.384	0	0	0	3	8	10	19
6	heating_degree_days	degree-days	139,860	23.0434	1.830	0	0	0	4.5471	13.0712	204.539	377.454
6	cooling_degree_days	degree-days	139,860	34.8578	4.877	0	0	6.7091	30.1322	75.8843	21.972	438.175
6	mean_temp_c	°C	139,860	22.060	3.897	5.752	1.545	4.864	22.481	27.528	29.065	32.939
6	tmean_variance_c2		139,860	8.979	5.470	0.147	0.956	1.808	8.130	19.156	25.110	52.151
6	tmin_variance_c2		139,860	8.655	4.824	0.200	0.966	1.929	8.133	17.637	22.225	38.458
6	tmax_variance_c2		139,860	12.687	8.368	0.240	1.556	2.892	10.793	28.982	38.935	72.852
7	ppt_total	mm	139,860	97.0876	2.135	0	0.138	8.646	89.6172	10.3302	77.256	589.672
7	tmean_mean		139,860	24.272	3.340	8.801	5.233	8.277	24.638	28.808	30.457	36.389
7	tmin_mean		139,860	18.054	3.832	1.025	7.230	0.851	18.550	23.156	24.494	28.953
7	tmax_mean		139,860	30.492	3.239	5.332	2.035	4.972	30.753	35.290	37.639	44.151
7	tdmean_mean		139,860	17.310	4.912	6.089	1.854	6.015	18.399	22.925	23.818	25.492
7	vpdmin_mean		139,860	1.808	1.914	0	0.115	0.292	1.178	5.611	9.380	25.170
7	vpdmax_mean		139,860	24.683	8.020	4.260	2.506	5.174	22.830	40.933	49.406	77.406
7	days_extremely_cold	days	139,860	0	0	0	0	0	0	0	0	0
7	days_below_freezing_32f	days	139,860	0.003	0.086	0	0	0	0	0	0	10
7	freeze_thaw_days	days	139,860	0.003	0.086	0	0	0	0	0	0	10
7	days_precip_above_10mm		139,860	3.150	2.402	0	0	0	3	8	10	20
7	heating_degree_days	degree-days	139,860	6.3251	1.160	0	0	0	0	35.2291	100.744	295.494
7	cooling_degree_days	degree-days	139,860	90.4389	1.888	0	1.392	1.751	95.8763	24.7193	75.836	559.731
7	mean_temp_c	°C	139,860	24.272	3.340	8.801	5.233	8.277	24.638	28.808	30.457	36.389
7	tmean_variance_c2		139,860	5.375	3.377	0.111	0.574	1.032	4.827	11.778	15.244	27.825

7	tmin_variance_c2		139,860	5.240	3.512	0.105	0.461	0.791	4.726	11.715	15.241	27.563
7	tmax_variance_c2		139,860	8.011	5.102	0.078	1.071	2.044	6.918	17.988	25.492	44.965
8	ppt_total	mm	139,860	89.2016	1.423	0	0.209	9.131	79.8062	202.1872	282.5161	282.293

PRISM: summary statistics by calendar month

month	variable	unit	n	mean	sd	min	p01	p05	median	p95	p99	max
8	tmean_mean		139,860	23.461	3.528	8.838	5.001	7.572	23.541	28.787	30.357	35.753
8	tmin_mean		139,860	17.136	3.977	1.652	6.719	10.201	17.367	22.886	24.397	28.888
8	tmax_mean		139,860	29.786	3.453	5.168	1.873	4.210	29.805	35.399	37.640	43.597
8	tdmean_mean		139,860	16.723	4.898	6.665	1.077	5.609	17.653	22.670	23.794	25.576
8	vpdmin_mean		139,860	1.581	1.744	0	0.091	0.230	0.980	5.049	8.441	22.071
8	vpdmax_mean		139,860	23.752	8.097	4.790	2.180	4.051	21.785	39.623	48.199	76.646
8	days_extremely_cold	days	139,860	0	0	0	0	0	0	0	0	0
8	days_below_freezing_32f	days	139,860	0.008	0.154	0	0	0	0	0	0	10
8	freeze_thaw_days	days	139,860	0.008	0.154	0	0	0	0	0	0	10
8	days_precip_above_10mm		139,860	2.799	2.184	0	0	0	3	7	9	17
8	heating_degree_days	degree-days	139,860	9.597	2.329	0	0	0	0.086	50.243	108.239	194.358
8	cooling_degree_days	degree-days	139,860	68.541	5.037	0	0.615	4.126	62.454	124.061	172.733	240.011
8	mean_temp_c	°C	139,860	23.461	3.528	8.838	5.001	7.572	23.541	28.787	30.357	35.753
8	tmean_variance_c2		139,860	6.232	3.973	0.081	0.598	1.138	5.577	13.405	18.856	36.189
8	tmin_variance_c2		139,860	6.527	4.430	0.155	0.481	0.893	5.863	14.753	19.541	33.029
8	tmax_variance_c2		139,860	9.044	5.844	0.202	1.223	2.306	7.690	20.487	28.881	61.125
9	ppt_total	mm	139,860	81.874	63.797	0	0.901	9.395	67.679	203.524	300.047	799.479
9	tmean_mean		139,860	19.591	4.019	3.910	0.167	2.972	19.606	26.040	27.704	31.617
9	tmin_mean		139,860	13.028	4.457	2.600	2.458	5.612	12.980	20.310	22.672	26.524
9	tmax_mean		139,860	26.154	3.911	9.309	7.047	9.588	26.256	32.202	34.104	40.057
9	tdmean_mean		139,860	12.990	5.215	8.950	2.147	2.164	13.558	20.364	22.521	25.592
9	vpdmin_mean		139,860	1.261	1.250	0	0.113	0.243	0.882	3.593	6.261	18.675
9	vpdmax_mean		139,860	19.808	6.396	4.756	9.127	1.074	18.863	31.689	38.218	65.443
9	days_extremely_cold	days	139,860	0	0	0	0	0	0	0	0	0
9	days_below_freezing_32f	days	139,860	0.414	1.500	0	0	0	0	3	8	27
9	freeze_thaw_days	days	139,860	0.413	1.493	0	0	0	0	3	8	27
9	days_precip_above_10mm		139,860	2.466	1.980	0	0	0	2	6	8	19

9	heating_degree_days	degree-days	139,860	50.5245	7.437	0	0	0	30.1241	67.2812	45.6084	32.692
9	cooling_degree_days	degree-days	139,860	88.2497	2.835	0	0	1.918	69.5982	31.5312	81.1633	98.521
9	mean_temp_c	°C	139,860	19.591	4.019	3.9101	0.1671	2.972	19.606	26.040	27.704	31.617

PRISM: summary statistics by calendar month

month	variable	unit	n	mean	sd	min	p01	p05	median	p95	p99	max
9	tmean_variance_c2		139,860	13.950	8.489	0.138	1.084	3.192	12.460	30.167	41.652	72.306
9	tmin_variance_c2		139,860	15.169	8.511	0.189	0.979	3.308	13.957	30.825	41.393	59.260
9	tmax_variance_c2		139,860	18.165	12.220	0.125	1.905	4.203	15.435	41.786	61.576	101.614
10	ppt_total	mm	139,860	78.394	61.132	0	0.595	7.341	65.639	190.556	288.952	842.133
10	tmean_mean		139,860	13.347	4.641	-3.272	3.208	5.889	13.270	21.046	24.001	28.212
10	tmin_mean		139,860	6.804	4.731	-9.974	-3.272	0.616	6.612	14.855	18.765	25.300
10	tmax_mean		139,860	19.891	4.851	3.293	8.885	11.897	19.959	27.596	29.830	35.244
10	tdmean_mean		139,860	7.041	5.337	-14.786	-6.245	2.288	7.113	15.617	19.008	24.276
10	vpdmin_mean		139,860	0.940	0.793	0	0.156	0.265	0.737	2.242	4.113	13.797
10	vpdmax_mean		139,860	13.991	4.993	2.486	5.431	7.123	13.320	23.242	28.433	52.769
10	days_extremely_cold	days	139,860	0.008	0.127	0	0	0	0	0	0	4
10	days_below_freezing_32f	days	139,860	4.569	6.017	0	0	0	2	18	25	31
10	freeze_thaw_days	days	139,860	4.446	5.774	0	0	0	2	17	24	31
10	days_precip_above_10mm	days	139,860	2.411	2.009	0	0	0	2	6	8	22
10	heating_degree_days	degree-days	139,860	79.024	15.697	0	1.079	7.065	66.225	136.019	168.875	669.753
10	cooling_degree_days	degree-days	139,860	24.448	38.128	0	0	0	7.660	105.353	180.230	306.254
10	mean_temp_c	°C	139,860	13.347	4.641	-3.272	3.208	5.889	13.270	21.046	24.001	28.212
10	tmean_variance_c2		139,860	20.036	10.497	0.240	3.655	7.194	18.102	38.799	55.942	112.900
10	tmin_variance_c2		139,860	20.778	9.801	0.408	3.487	7.167	19.339	39.391	49.172	94.655
10	tmax_variance_c2		139,860	27.262	16.167	0.325	4.155	8.219	24.046	56.327	84.730	157.079
11	ppt_total	mm	139,860	74.065	63.339	0	0.419	4.933	60.402	187.236	285.426	1039.368
11	tmean_mean		139,860	7.017	5.453	-13.099	-5.462	1.995	7.004	15.852	19.717	26.114
11	tmin_mean		139,860	1.076	5.128	-18.316	-10.656	7.293	1.048	9.649	14.258	23.185
11	tmax_mean		139,860	12.958	6.054	-8.205	-0.833	2.792	13.051	22.511	25.697	30.182
11	tdmean_mean		139,860	1.505	5.468	-16.915	-10.555	7.258	1.299	10.904	15.169	22.440
11	vpdmin_mean		139,860	0.717	0.472	0.000	0.163	0.249	0.621	1.472	2.469	7.590
11	vpdmax_mean		139,860	8.751	4.020	0.646	2.026	3.027	8.349	15.840	20.048	32.952

11	days_extremely_cold	days	139,860	0.191	0.957	0	0	0	0	1	5	18
11	days_below_freezing_32f	days	139,860	13.799	9.101	0	0	0	13	29	30	30
11	freeze_thaw_days	days	139,860	12.016	7.433	0	0	0	12	24	27	30

PRISM: summary statistics by calendar month

month	variable	unit	n	mean	sd	min	p01	p05	median	p95	p99	max
11	days_precip_above_10mm		139,860	2.331	2.226	0	0	0	2	6	9	24
11	heating_degree_days	degree-days	139,860	44.019	55.923	0	29.147	97.713	40.115	609.857	13.860	942.972
11	cooling_degree_days	degree-days	139,860	4.514	14.368	0	0	0	0	25.202	74.694	233.421
11	mean_temp_c	°C	139,860	7.017	5.453	-13.099	-5.462	-1.995	7.004	15.852	19.717	26.114
11	tmean_variance_c2		139,860	23.424	10.841	0.341	4.782	8.683	22.063	42.652	56.171	141.518
11	tmin_variance_c2		139,860	23.740	11.069	0.507	4.735	8.368	22.564	43.254	57.248	137.750
11	tmax_variance_c2		139,860	30.857	15.216	0.324	5.604	11.035	28.414	58.288	78.194	164.824
12	ppt_total	mm	139,860	77.075	68.347	0	0.525	5.043	61.906	198.353	19.801	248.506
12	tmean_mean		139,860	2.015	6.506	-20.732	-13.760	-8.752	2.088	12.407	16.749	24.863
12	tmin_mean		139,860	-3.321	6.253	-25.627	-18.964	-13.953	-2.977	6.489	11.238	22.419
12	tmax_mean		139,860	7.351	6.974	-16.141	-8.702	-3.956	7.266	18.504	22.629	28.207
12	tdmean_mean		139,860	-2.525	6.211	-23.919	-16.521	-12.379	-2.693	8.039	12.766	22.041
12	vpdmin_mean		139,860	0.518	0.330	0.000	0.129	0.188	0.449	1.056	1.785	6.078
12	vpdmax_mean		139,860	5.577	3.243	0.408	0.912	1.450	5.024	11.536	15.158	24.458
12	days_extremely_cold	days	139,860	1.549	3.629	0	0	0	0	10	17	31
12	days_below_freezing_32f	days	139,860	21.129	9.172	0	0	3	23	31	31	31
12	freeze_thaw_days	days	139,860	14.915	6.858	0	0	2	16	25	29	31
12	days_precip_above_10mm		139,860	2.399	2.358	0	0	0	2	7	10	26
12	heating_degree_days	degree-days	139,860	507.503	198.435	0	34.125	191.575	503.617	1339.649	94.889	211.039
12	cooling_degree_days	degree-days	139,860	1.634	7.870	0	0	0	0	8.558	37.872	202.416
12	mean_temp_c	°C	139,860	2.015	6.506	-20.732	-13.760	-8.752	2.088	12.407	16.749	24.863
12	tmean_variance_c2		139,860	26.934	15.984	0.802	4.215	8.492	23.225	57.795	80.921	141.734
12	tmin_variance_c2		139,860	28.869	16.973	0.916	4.765	8.517	25.063	61.127	84.874	162.588
12	tmax_variance_c2		139,860	32.669	18.917	1.304	5.045	10.012	28.601	67.578	98.668	170.805

Saved /mnt/data_d/Dropbox/Research/AnimalCollisionsWeather/tables/weather/tier1/prism_summary_by_calendar_month.pdf

	month	variable	unit	n	mean	sd	min	p01	p05	median
0	1	ppt_total	mm	139860	69.302086	63.658517	0.000000	0.519214	4.241862	54.173623
1	1	tmean_mean		139860	0.247689	6.688056	-25.086144	-15.219131	-10.774010	0.238532
2	1	tmin_mean		139860	-5.281511	6.471120	-31.218806	-21.039570	-16.198806	-5.103047
3	1	tmax_mean		139860	5.776963	7.118363	-19.476206	-9.590765	-5.516573	5.626801
4	1	tdmean_mean		139860	-4.555374	6.106142	-28.674932	-18.023729	-14.027483	-4.979637
5	1	vpdmin_mean		139860	0.518525	0.330165	0.002796	0.130769	0.183901	0.453087
6	1	vpdmax_mean		139860	5.342757	3.333215	0.405437	0.875723	1.335854	4.668457
7	1	days_extremely_cold	days	139860	2.621922	4.778270	0.000000	0.000000	0.000000	0.000000
8	1	days_below_freezing_32f	days	139860	23.302174	8.604183	0.000000	0.000000	4.000000	26.000000
9	1	freeze_thaw_days	days	139860	15.196160	7.052454	0.000000	0.000000	2.000000	16.000000
10	1	days_precip_above_10mm		139860	2.136572	2.297472	0.000000	0.000000	0.000000	2.000000
11	1	heating_degree_days	degree-days	139860	561.439595	205.719248	0.735994	121.309989	233.726224	560.953177
12	1	cooling_degree_days	degree-days	139860	0.784628	4.588120	0.000000	0.000000	0.000000	0.000000
13	1	mean_temp_c	°C	139860	0.247689	6.688056	-25.086144	-15.219131	-10.774010	0.238532
14	1	tmean_variance_c2		139860	29.681628	16.742633	0.907455	3.803849	8.155078	26.795653
15	1	tmin_variance_c2		139860	32.111103	19.126775	0.822870	4.171016	8.179250	28.292632
16	1	tmax_variance_c2		139860	35.505357	18.782846	1.010423	5.020870	9.799911	32.548080
17	2	ppt_total	mm	139860	69.017213	62.085681	0.000000	0.778496	4.886015	53.463694
18	2	tmean_mean		139860	2.208621	6.933573	-21.934184	-13.973100	-9.588891	2.372754
19	2	tmin_mean		139860	-3.678880	6.710893	-27.161887	-19.987494	-15.346145	-3.425394
20	2	tmax_mean		139860	8.096177	7.338429	-16.756080	-8.212540	-3.985658	8.164842
21	2	tdmean_mean		139860	-3.158050	6.396675	-24.106918	-17.186038	-13.370867	-3.502747
22	2	vpdmin_mean		139860	0.593525	0.386728	0.005654	0.147675	0.206714	0.514006
23	2	vpdmax_mean		139860	6.813523	3.943340	0.513750	1.123663	1.820251	6.186962

6. County summary-statistics table

Calculate summary statistics separately by county. Save the complete table as a machine-readable file; show only selected examples and counties flagged by transparent outlier rules in the rendered report.

```
In [42]: summary_by_county = summarize_long(df, ["geoid"])
county_names = df[["geoid", "state_fips", "county_name"]].drop_duplicates("geoid")
summary_by_county = summary_by_county.merge(county_names, on="geoid", how="left", validate="many_to_one")
summary_by_county.to_csv(
    TABLES_DIR / f"{config.name.lower()}_summary_by_county.csv", index=False
)

# Flag county-variable means that are unusually far from the national
# distribution of county means. These are review flags, not automatic errors.
county_mean_distribution = (
    summary_by_county.groupby("variable")["mean"]
    .agg(mean_across_counties="mean", std_across_counties="std")
)
flagged_counties = summary_by_county.merge(
    county_mean_distribution, left_on="variable", right_index=True,
)
flagged_counties["abs_z"] = (
    (flagged_counties["mean"] - flagged_counties["mean_across_counties"])
    / flagged_counties["std_across_counties"]
).abs()
flagged_counties = flagged_counties[
    flagged_counties["abs_z"].ge(OUTLIER_Z_THRESHOLD)
].sort_values(["variable", "abs_z"], ascending=[True, False])
flagged_counties.to_csv(
    TABLES_DIR / f"{config.name.lower()}_summary_by_county_review_flags.csv", index=False
)
flag_columns = ["geoid", "state_fips", "county_name", "variable", "unit", "mean", "sd", "min", "max", "abs_
save_table_pdf(
    flagged_counties[flag_columns],
    TABLES_DIR / f"{config.name.lower()}_summary_by_county_review_flags.pdf",
    f"{config.name}: county-variable review flags (|z| ≥ {OUTLIER_Z_THRESHOLD:g})",
    rows_per_page=28,
```

```
)
display(flagged_counties[flag_columns].head(30))
```

PRISM: county-variable review flags ($|z| \geq 5$)

geoid	state_fips	county_name	variable	unit	mean	sd	min	max	abs_z
41057	41	Tillamook	ppt_total	mm	241.960	201.172	1.596	1,039.368	5.262
04027	04	Yuma	vpdmax_mean		39.906	17.896	11.310	76.646	6.039
04012	04	La Paz	vpdmax_mean		37.944	17.834	10.621	77.406	5.574
04013	04	Maricopa	vpdmax_mean		37.362	17.385	10.577	74.440	5.436
06025	06	Imperial	vpdmax_mean		37.271	16.838	10.722	74.018	5.414
32003	32	Clark	vpdmin_mean		9.028	6.030	1.099	25.170	10.519
04013	04	Maricopa	vpdmin_mean		9.017	5.623	0.966	24.439	10.503
04027	04	Yuma	vpdmin_mean		8.559	5.223	1.115	23.736	9.897
04012	04	La Paz	vpdmin_mean		8.550	5.337	0.970	22.728	9.885
06071	06	San Bernardino	vpdmin_mean		8.073	5.098	0.937	23.631	9.255
04021	04	Pinal	vpdmin_mean		7.760	4.842	0.771	20.644	8.840
06025	06	Imperial	vpdmin_mean		7.667	4.491	0.957	20.577	8.717
04015	04	Mohave	vpdmin_mean		7.414	4.793	0.959	19.823	8.383
06027	06	Inyo	vpdmin_mean		7.370	5.003	0.664	21.597	8.325
04019	04	Pima	vpdmin_mean		6.838	4.085	0.806	18.758	7.620
06065	06	Riverside	vpdmin_mean		6.733	3.801	0.812	18.937	7.481
48141	48	El Paso	vpdmin_mean		5.551	3.601	0.554	18.703	5.918
49053	49	Washington	vpdmin_mean		5.545	3.902	0.679	16.096	5.910
32017	32	Lincoln	vpdmin_mean		5.117	3.699	0.486	15.028	5.344
04025	04	Yavapai	vpdmin_mean		5.066	3.139	0.644	14.290	5.276

Saved /mnt/data_d/Dropbox/Research/AnimalCollisionsWeather/tables/weather/tier1/prism_summary_by_county_review_flags.pdf

	geoid	state_fips	county_name	variable	unit	mean	sd	min	max	abs_z
37434	41057	41	Tillamook	ppt_total	mm	241.959760	201.172258	1.596386	1039.367545	5.261667
1383	04027	04	Yuma	vpdmax_mean		39.906258	17.895697	11.310006	76.646234	6.039083
1247	04012	04	La Paz	vpdmax_mean		37.943877	17.833950	10.621446	77.406020	5.573655
1264	04013	04	Maricopa	vpdmax_mean		37.361513	17.385000	10.576653	74.439812	5.435533
2879	06025	06	Imperial	vpdmax_mean		37.271014	16.838419	10.722200	74.017737	5.414069
29143	32003	32	Clark	vpdmin_mean		9.028451	6.029890	1.099227	25.170275	10.518618
1263	04013	04	Maricopa	vpdmin_mean		9.016967	5.622722	0.966291	24.439483	10.503423
1382	04027	04	Yuma	vpdmin_mean		8.558738	5.223427	1.115318	23.736027	9.897152
1246	04012	04	La Paz	vpdmin_mean		8.549660	5.336735	0.969517	22.728241	9.885142
3269	06071	06	San Bernardino	vpdmin_mean		8.073054	5.098230	0.936982	23.631054	9.254556
1331	04021	04	Pinal	vpdmin_mean		7.759765	4.842107	0.770928	20.643865	8.840051
2878	06025	06	Imperial	vpdmin_mean		7.667097	4.491151	0.956721	20.577108	8.717445
1280	04015	04	Mohave	vpdmin_mean		7.414481	4.792605	0.958888	19.822701	8.383215
2895	06027	06	Inyo	vpdmin_mean		7.370424	5.002898	0.664234	21.596976	8.324924
1314	04019	04	Pima	vpdmin_mean		6.837826	4.084996	0.805841	18.758336	7.620257
3218	06065	06	Riverside	vpdmin_mean		6.732822	3.800765	0.812388	18.936807	7.481328
43508	48141	48	El Paso	vpdmin_mean		5.551228	3.601426	0.553876	18.703452	5.917990
47078	49053	49	Washington	vpdmin_mean		5.544981	3.902254	0.678788	16.096304	5.909726
29262	32017	32	Lincoln	vpdmin_mean		5.117291	3.699395	0.486473	15.028367	5.343859
1365	04025	04	Yavapai	vpdmin_mean		5.066043	3.139329	0.644160	14.290226	5.276054

7. National winter-trend figures

Plot the annual unweighted county mean and the full-period linear trend for each principal winter measure:

1. Mean winter temperature

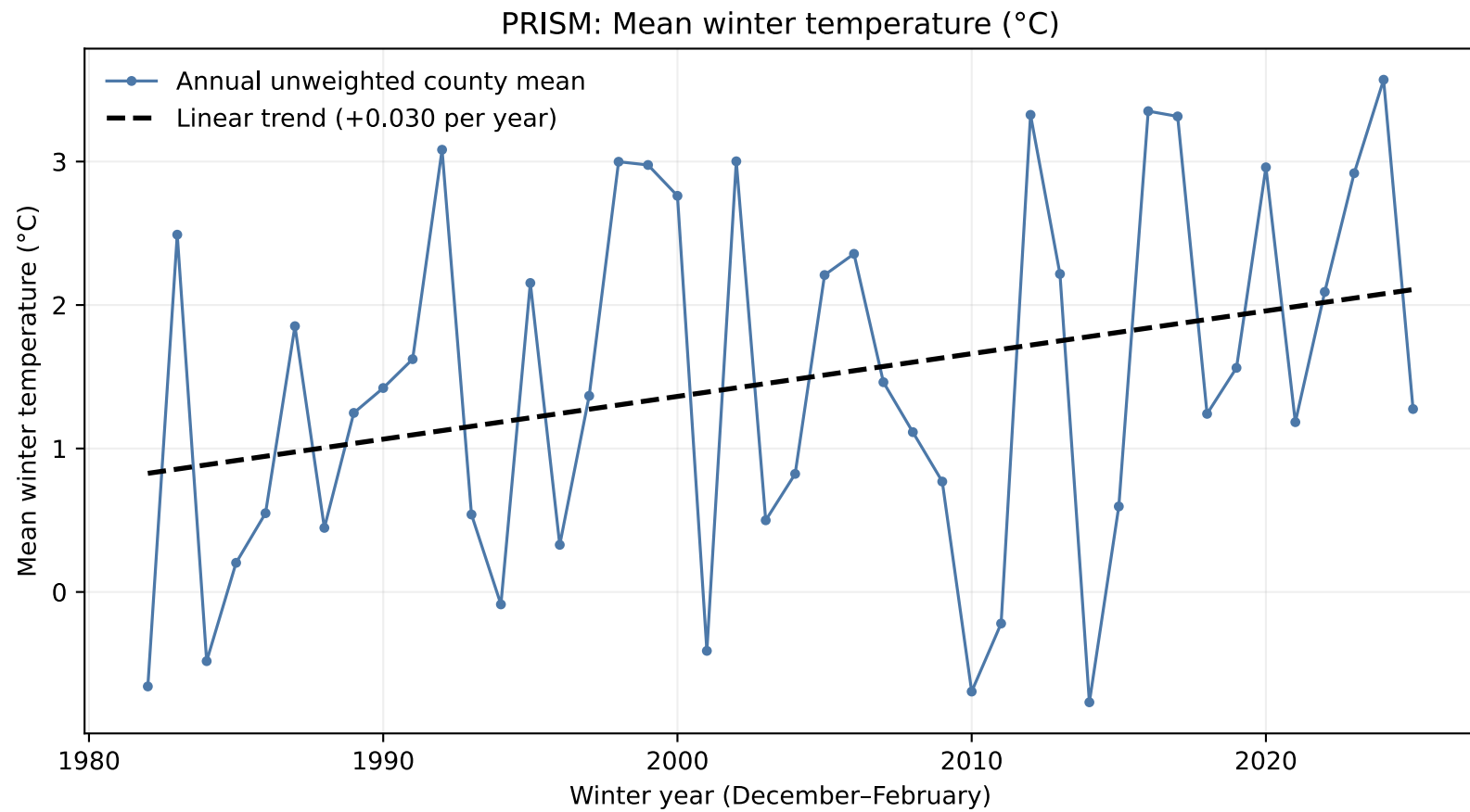
2. Extreme-cold-day count ($TMIN < 0^{\circ}F$)
3. Freezing-day count ($TMIN < 32^{\circ}F$)
4. Freeze-thaw-day count

Show annual values so that common shocks remain visible; the fitted trend should supplement rather than replace them.

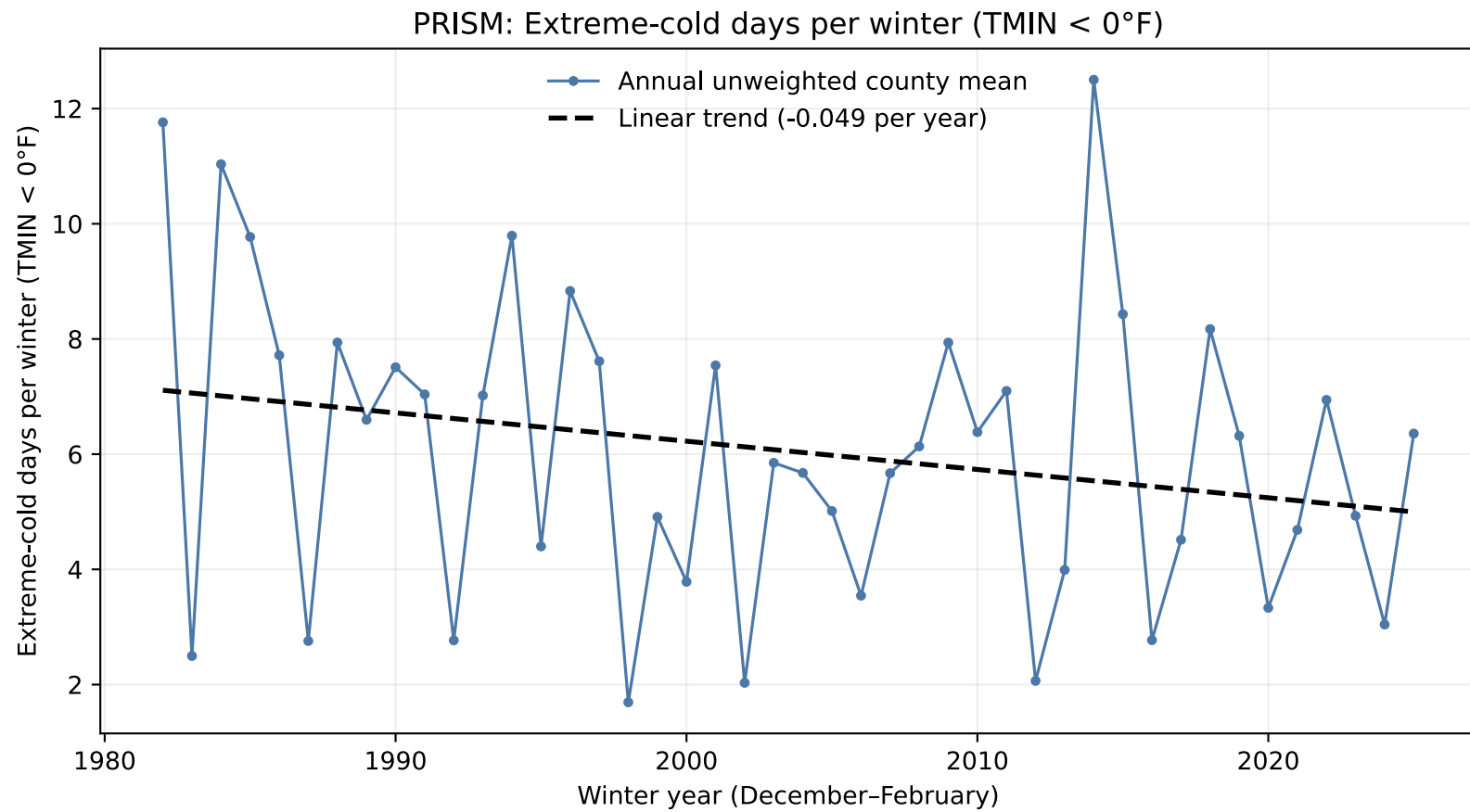
```
In [43]: def _linear_trend(x, y):
    valid = pd.DataFrame({"x": x, "y": y}).dropna()
    if valid["x"].nunique() < 2:
        return None
    slope, intercept = np.polyfit(valid["x"], valid["y"], 1)
    x_line = np.array([valid["x"].min(), valid["x"].max()])
    return x_line, intercept + slope * x_line, slope

def plot_national_winter_trend(frame, variable):
    annual = frame.groupby("winter_year", as_index=False)[variable].mean()
    trend = _linear_trend(annual["winter_year"], annual[variable])
    fig, ax = plt.subplots(figsize=(8.5, 4.8))
    ax.plot(annual["winter_year"], annual[variable], color="#4C78A8", lw=1.2, marker="o", ms=3,
            label="Annual unweighted county mean")
    if trend is not None:
        ax.plot(trend[0], trend[1], color="black", lw=2, ls="--",
                label=f"Linear trend ({trend[2]:+.3f} per year)")
    ax.set(title=f"{config.name}: {variable_label(variable)}",
           xlabel="Winter year (December–February)", ylabel=variable_label(variable))
    ax.grid(alpha=0.2)
    ax.legend(frameon=False)
    fig.tight_layout()
    return save_figure_pdf(fig, f"{config.name.lower()}_national_trend_{variable}.pdf")

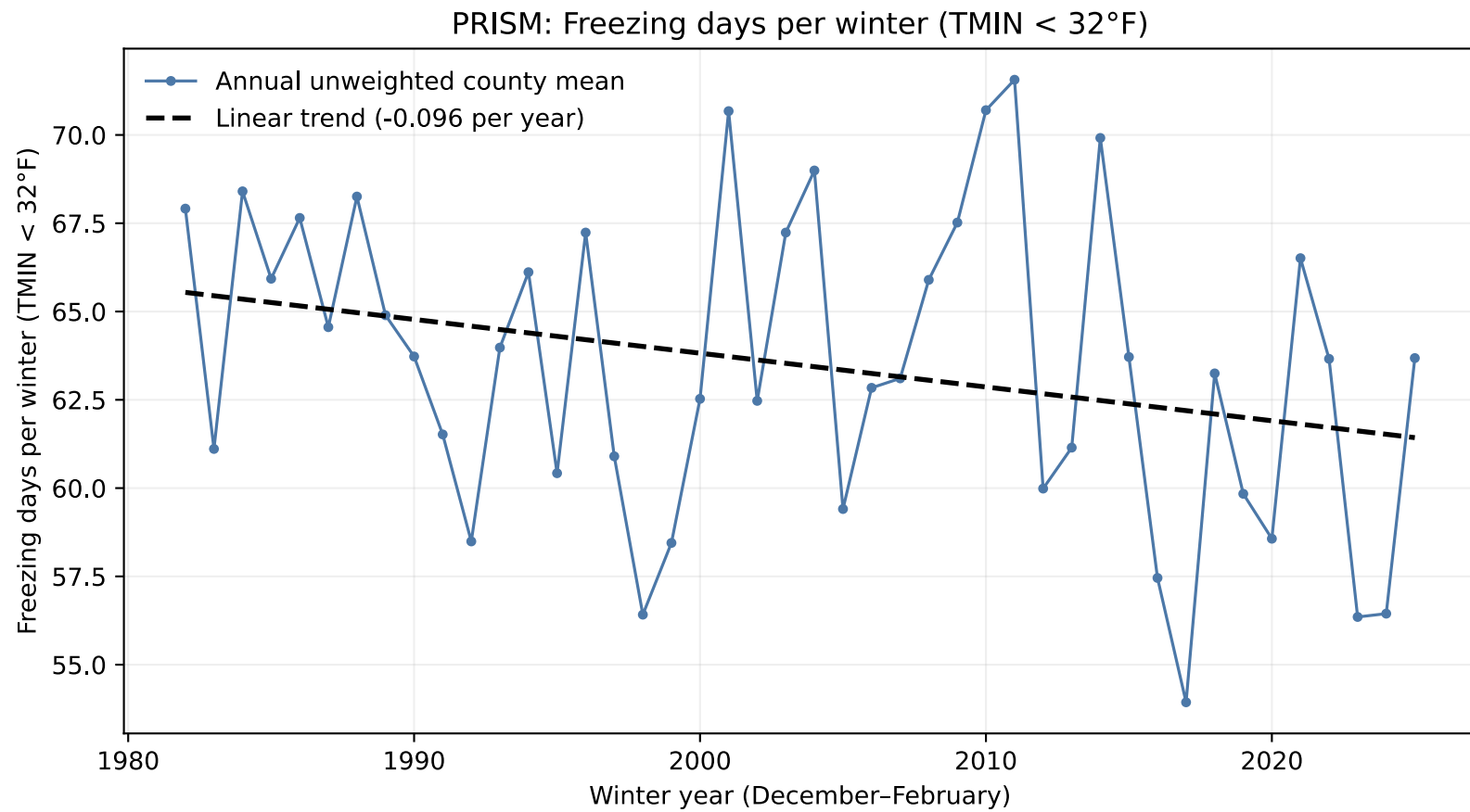
national_trend_paths = [
    plot_national_winter_trend(county_winter, variable)
    for variable in PRIMARY_TREND_VARS if variable in county_winter
]
```



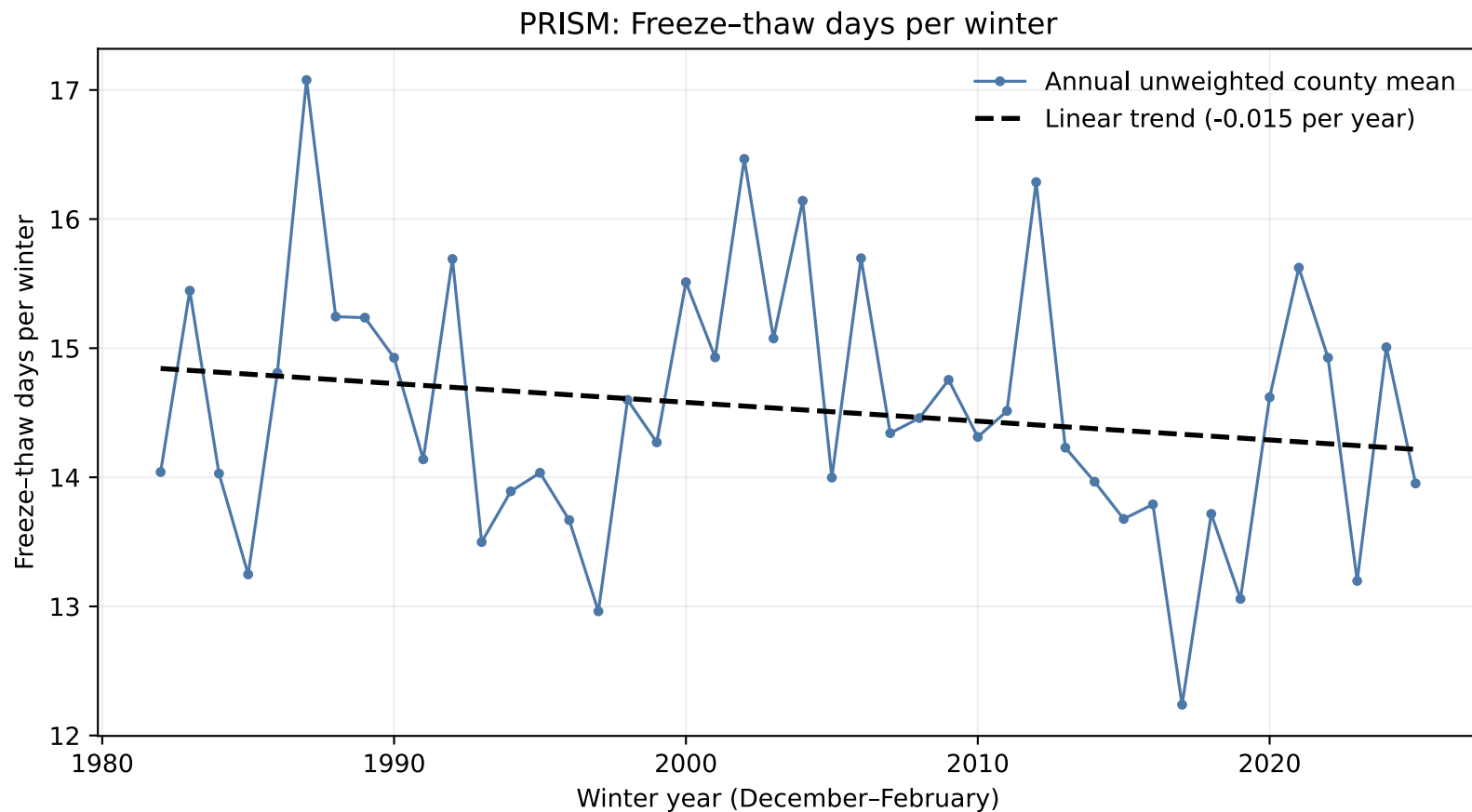
Saved /mnt/data_d/Dropbox/Research/AnimalCollisionsWeather/figures/weather/tier1/prism_national_trend_mean_temp_c.pdf



Saved /mnt/data_d/Dropbox/Research/AnimalCollisionsWeather/figures/weather/tier1/prism_national_trend_days_extremely_cold.pdf



Saved /mnt/data_d/Dropbox/Research/AnimalCollisionsWeather/figures/weather/tier1/prism_national_trend_days_below_freezing_32f.pdf



Saved /mnt/data_d/Dropbox/Research/AnimalCollisionsWeather/figures/weather/tier1/prism_national_trend_freeze_thaw_days.pdf

8. State winter-trend small multiples

Plot annual state means and full-period linear trends for the four principal winter measures, faceted by state. These figures assess geographic heterogeneity in levels, trends, and year-to-year movement. Use multi-page layouts when necessary for legibility.

Findings:

- Some linear trends for extreme-cold days approach or cross below zero by the end of the sample, implying impossible predicted values of "negative # of cold days." Which is acceptable for basic descriptive stats & internal QA (but not polished exhibits).

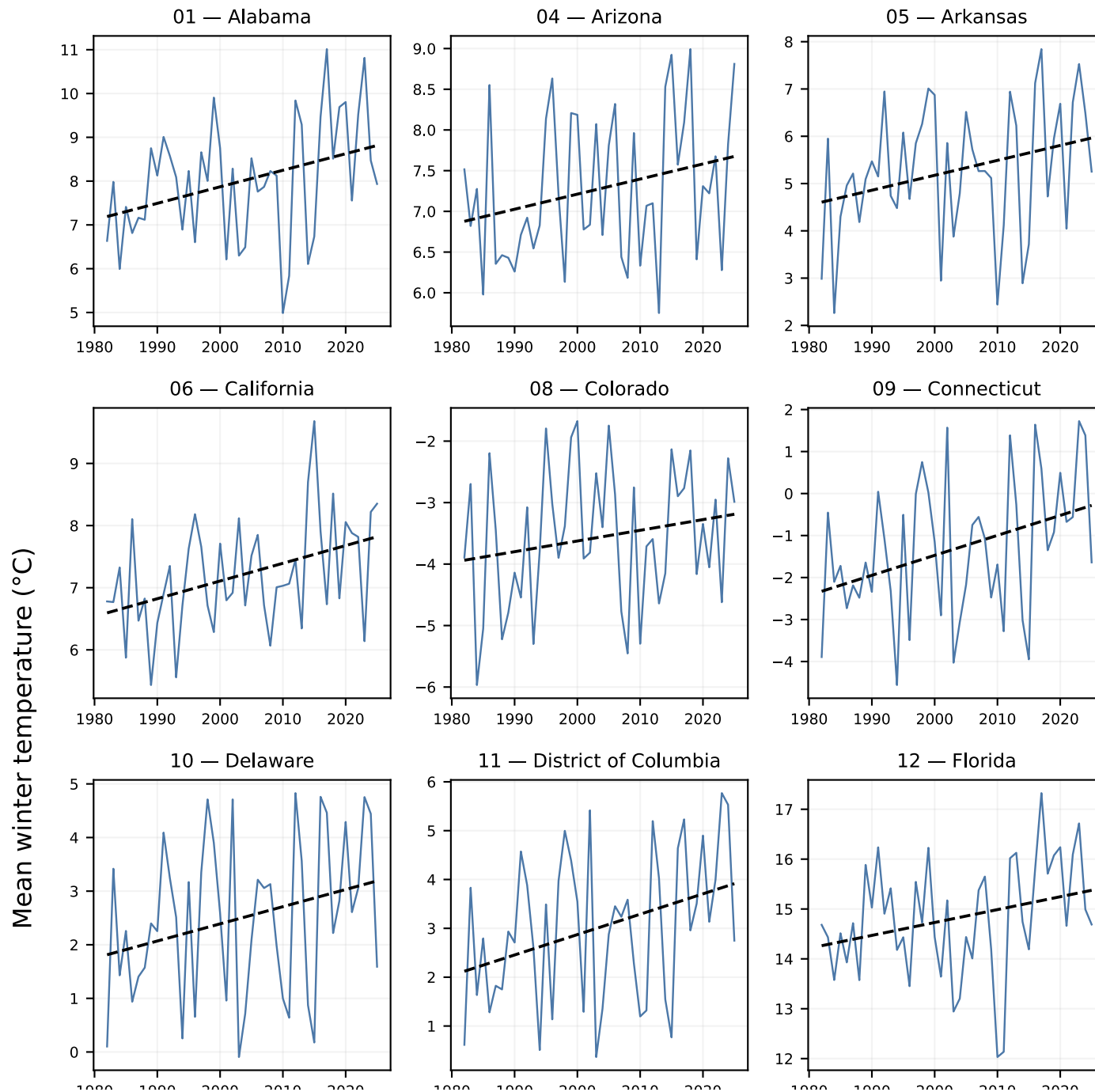
```

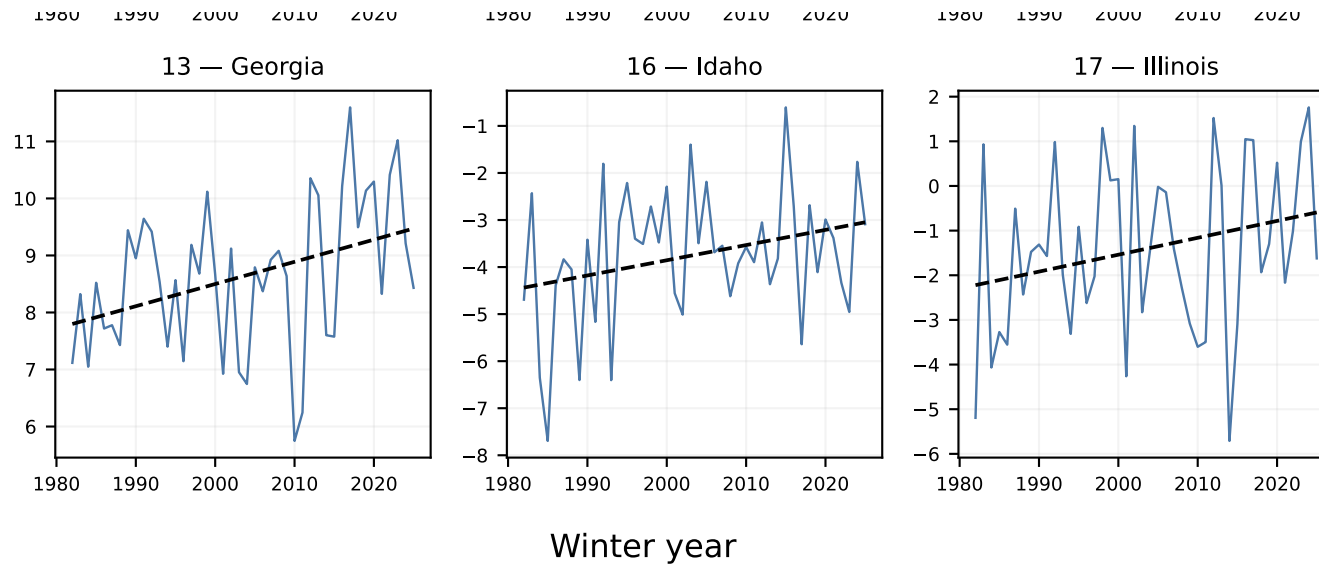
In [44]: def save_state_trend_multipage(frame, variable, states_per_page=12, n_cols=3):
    states = sorted(frame["state_fips"].dropna().unique())
    path = FIGURES_DIR / f"{config.name.lower()}_state_trends_{variable}.pdf"
    with PdfPages(path) as pdf:
        for start in range(0, len(states), states_per_page):
            page_states = states[start:start + states_per_page]
            n_rows = int(np.ceil(len(page_states) / n_cols))
            fig, axes = plt.subplots(n_rows, n_cols, figsize=(7.5, 2.65 * n_rows), squeeze=False)
            for ax, state in zip(axes.flat, page_states):
                annual = (
                    frame[frame["state_fips"].eq(state)]
                    .groupby("winter_year", as_index=False)[variable].mean()
                )
                trend = _linear_trend(annual["winter_year"], annual[variable])
                ax.plot(annual["winter_year"], annual[variable], color="#4C78A8", lw=0.9)
                if trend is not None:
                    ax.plot(trend[0], trend[1], color="black", lw=1.4, ls="--")
                ax.set_title(state_label(state), fontsize=9)
                ax.tick_params(labelsize=7)
                ax.grid(alpha=0.15)
            for ax in list(axes.flat)[len(page_states):]:
                ax.axis("off")
            fig.suptitle(f"{config.name}: {variable_label(variable)} by state", fontsize=13)
            fig.supxlabel("Winter year")
            fig.supylabel(variable_label(variable))
            fig.tight_layout()
            pdf.savefig(fig, bbox_inches="tight")
            display_figure_inline(fig)
            plt.close(fig)
    print(f"Saved {path}")
    return path

state_trend_paths = [
    save_state_trend_multipage(county_winter, variable)
    for variable in PRIMARY_TREND_VARS if variable in county_winter
]

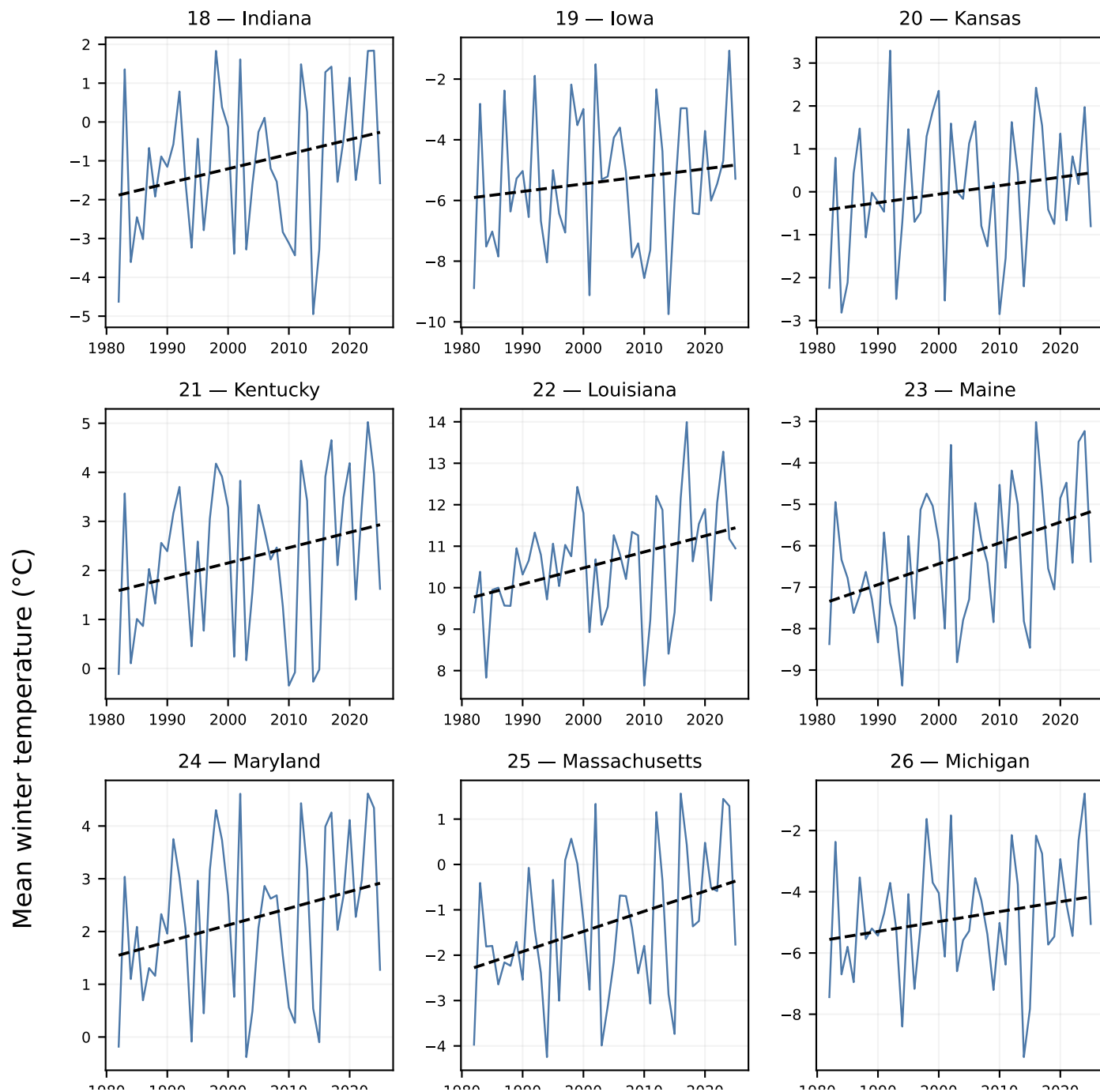
```

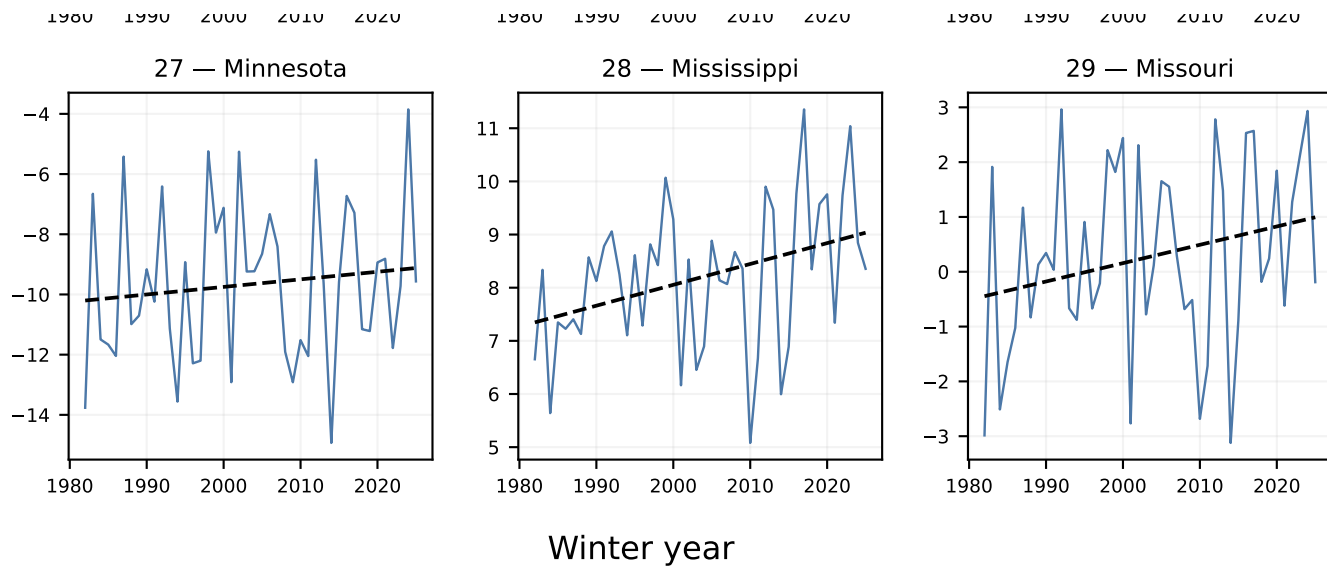
PRISM: Mean winter temperature (°C) by state



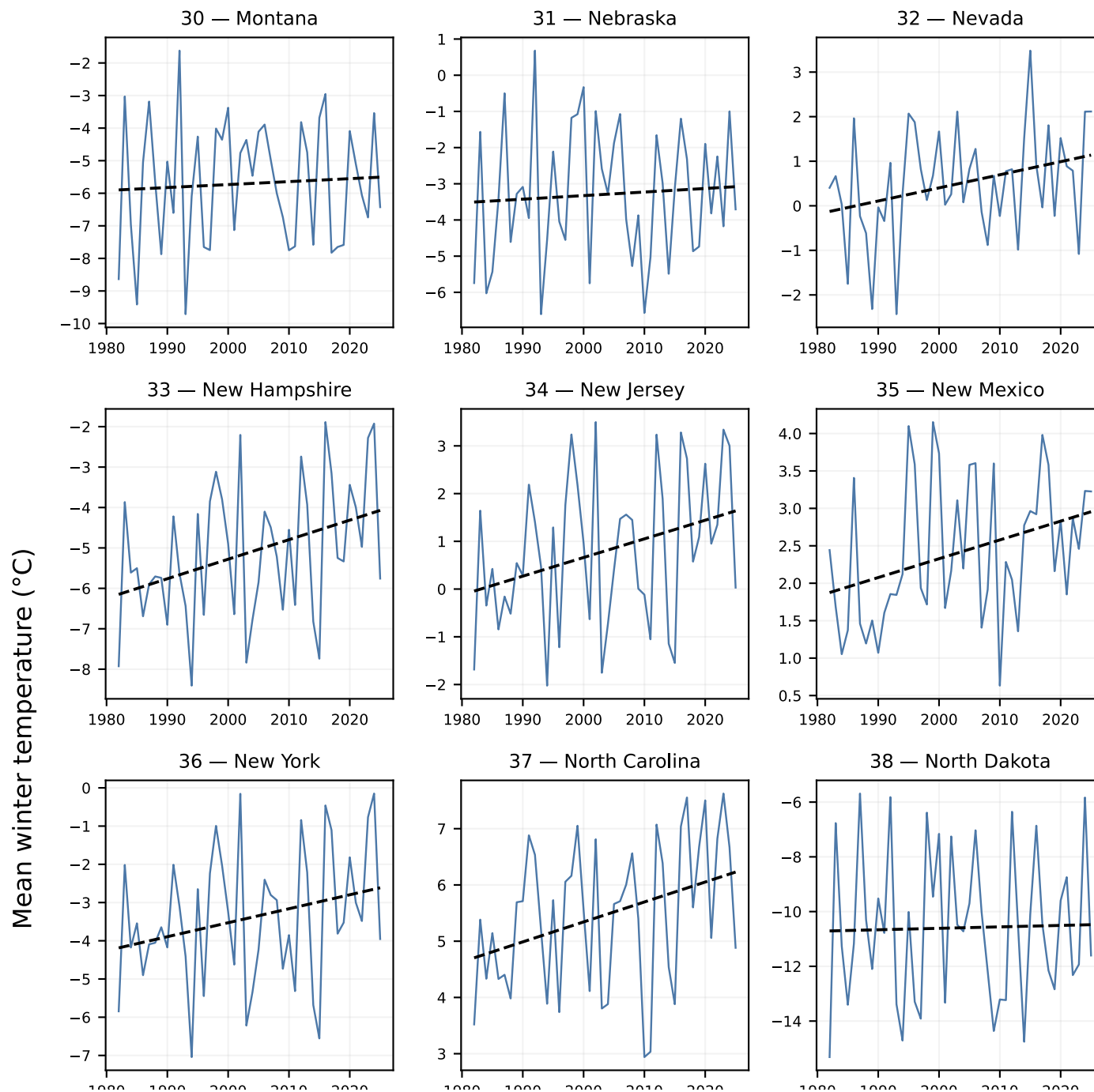


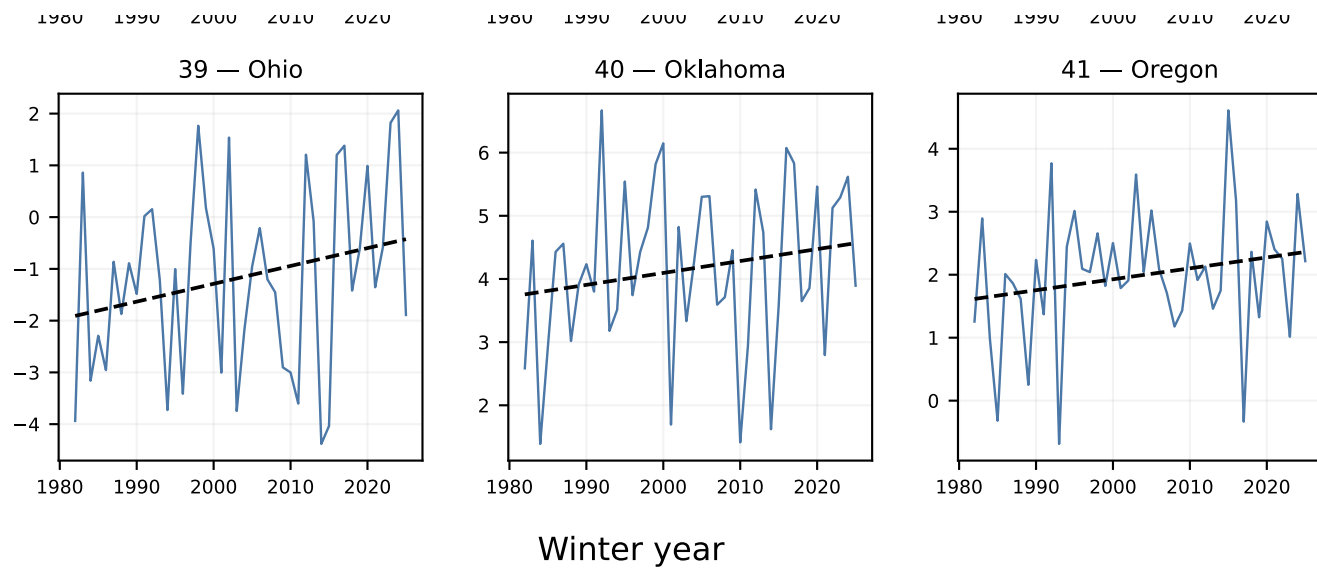
PRISM: Mean winter temperature (°C) by state



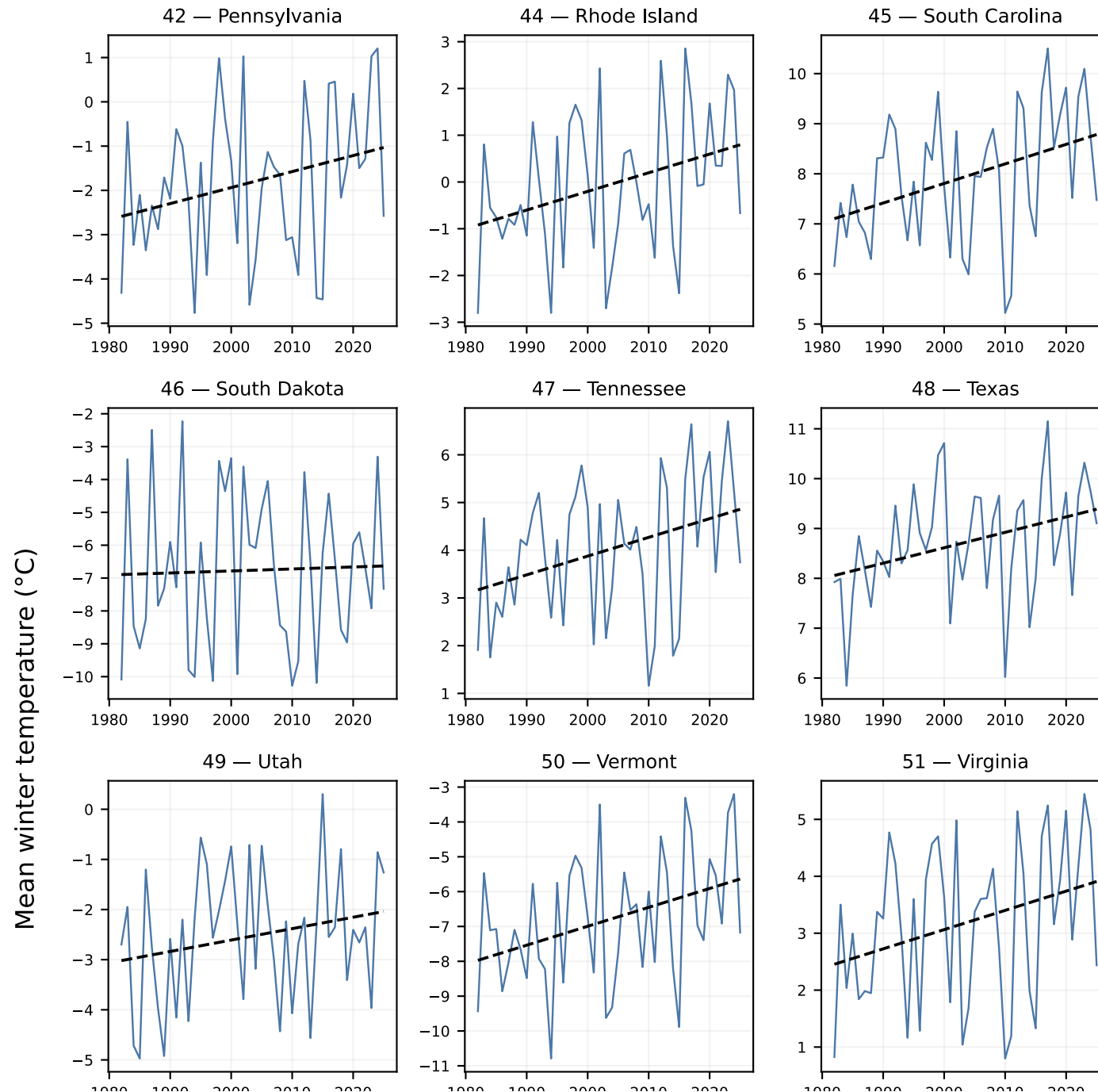


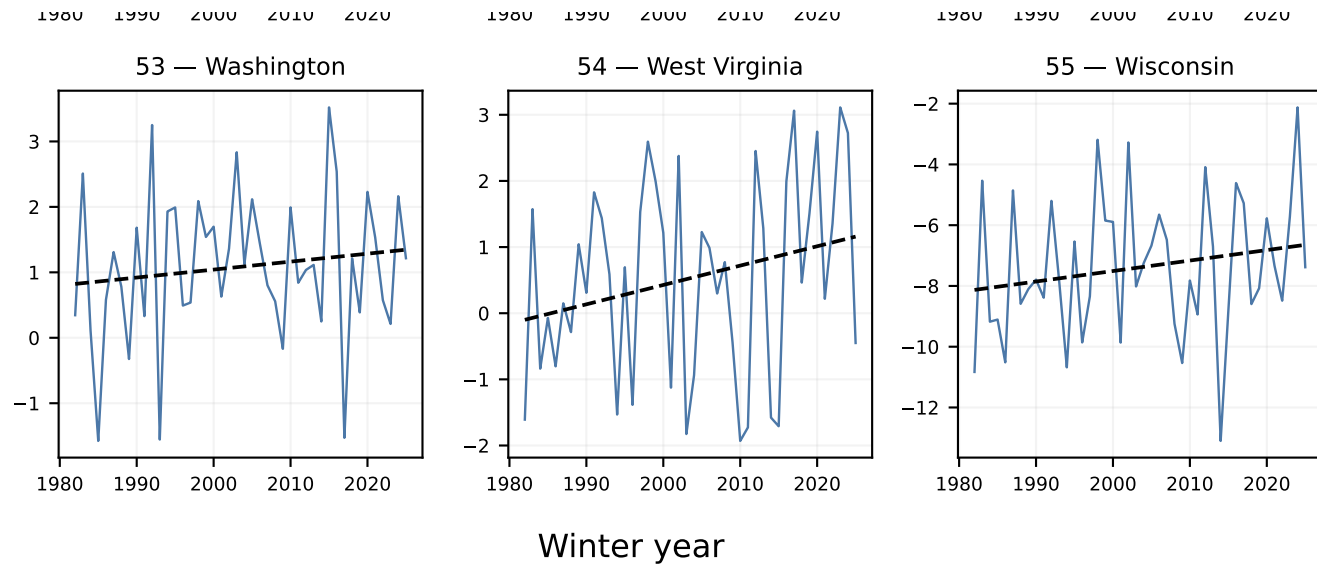
PRISM: Mean winter temperature (°C) by state



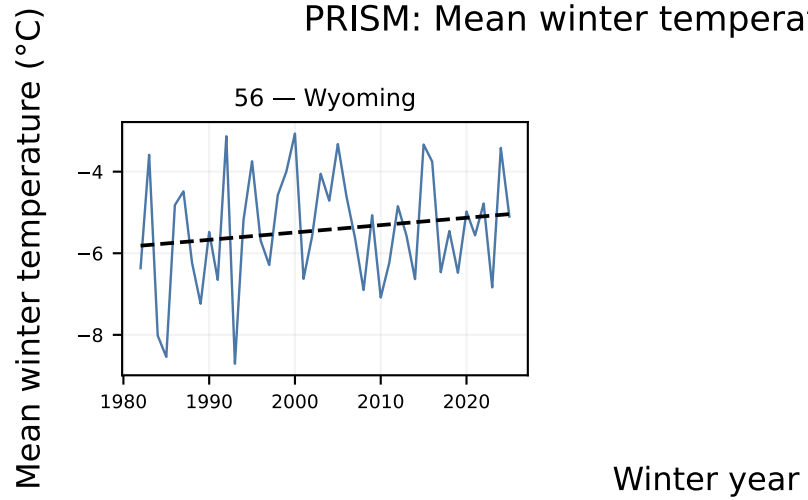


PRISM: Mean winter temperature (°C) by state



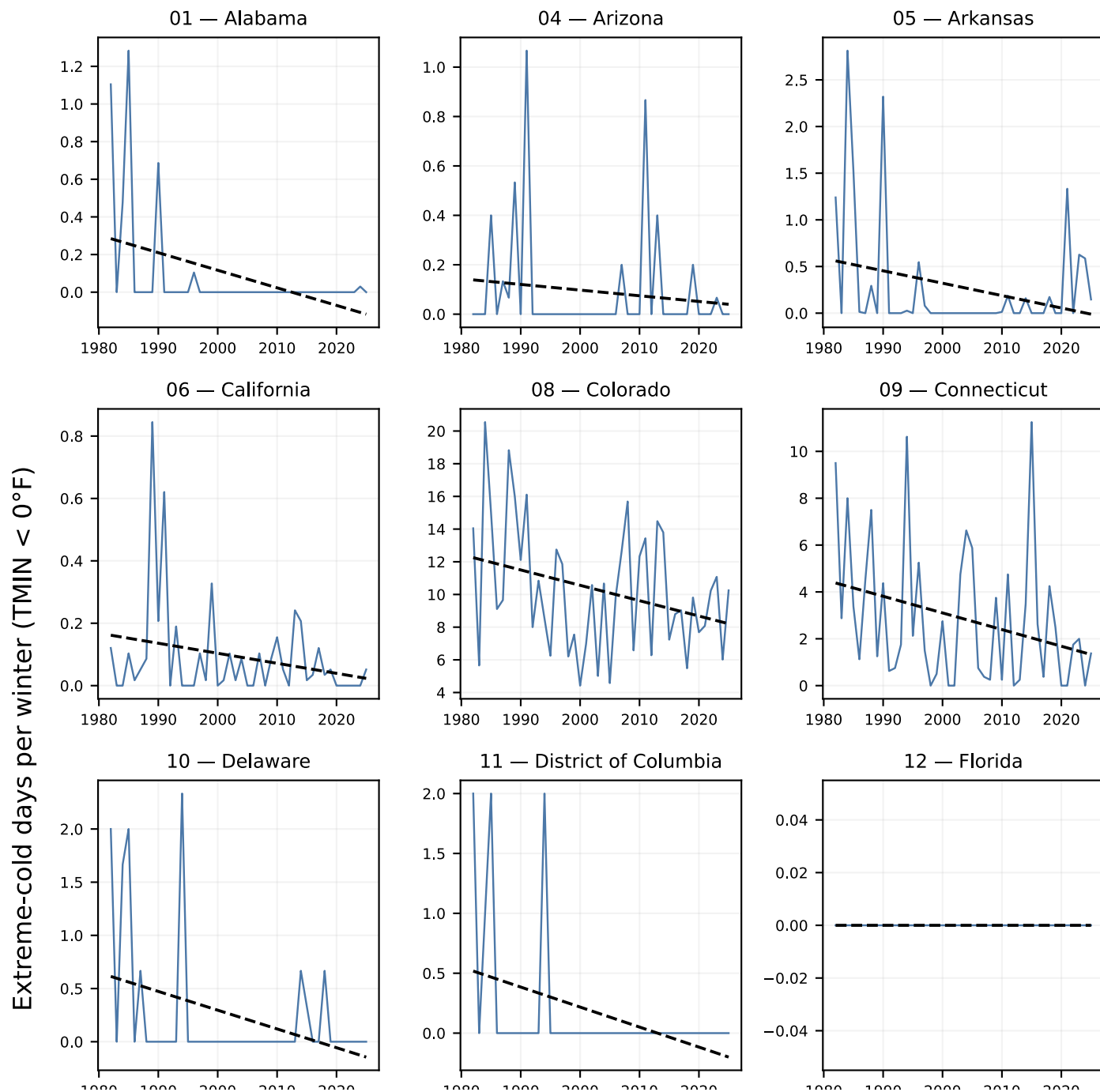


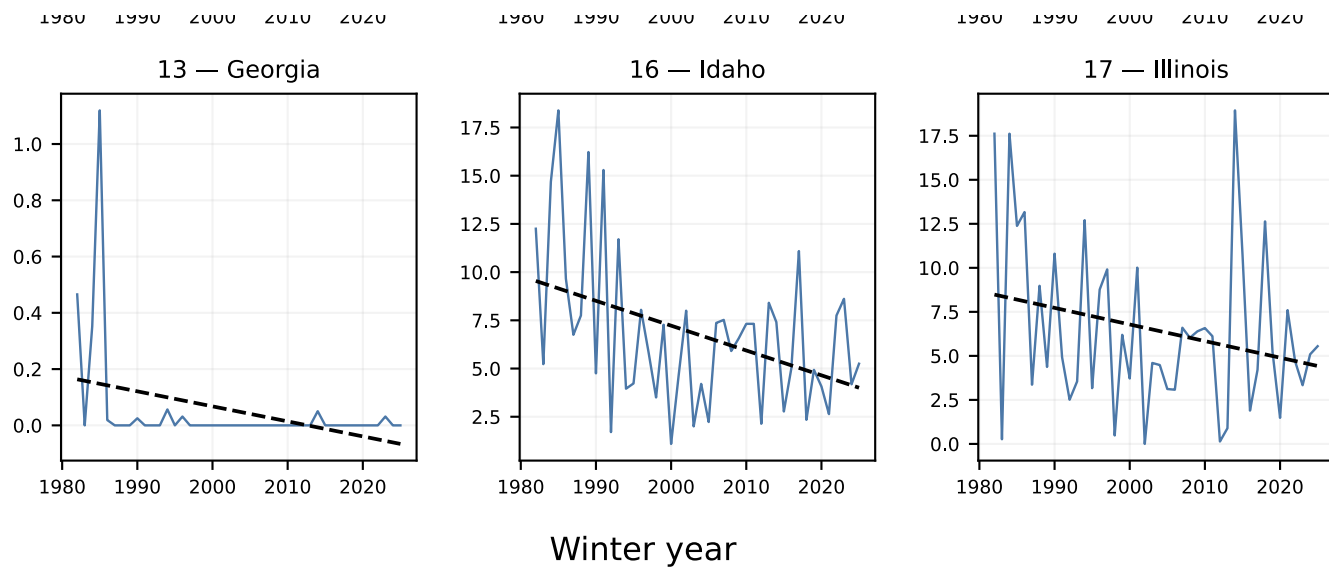
PRISM: Mean winter temperature (°C) by state



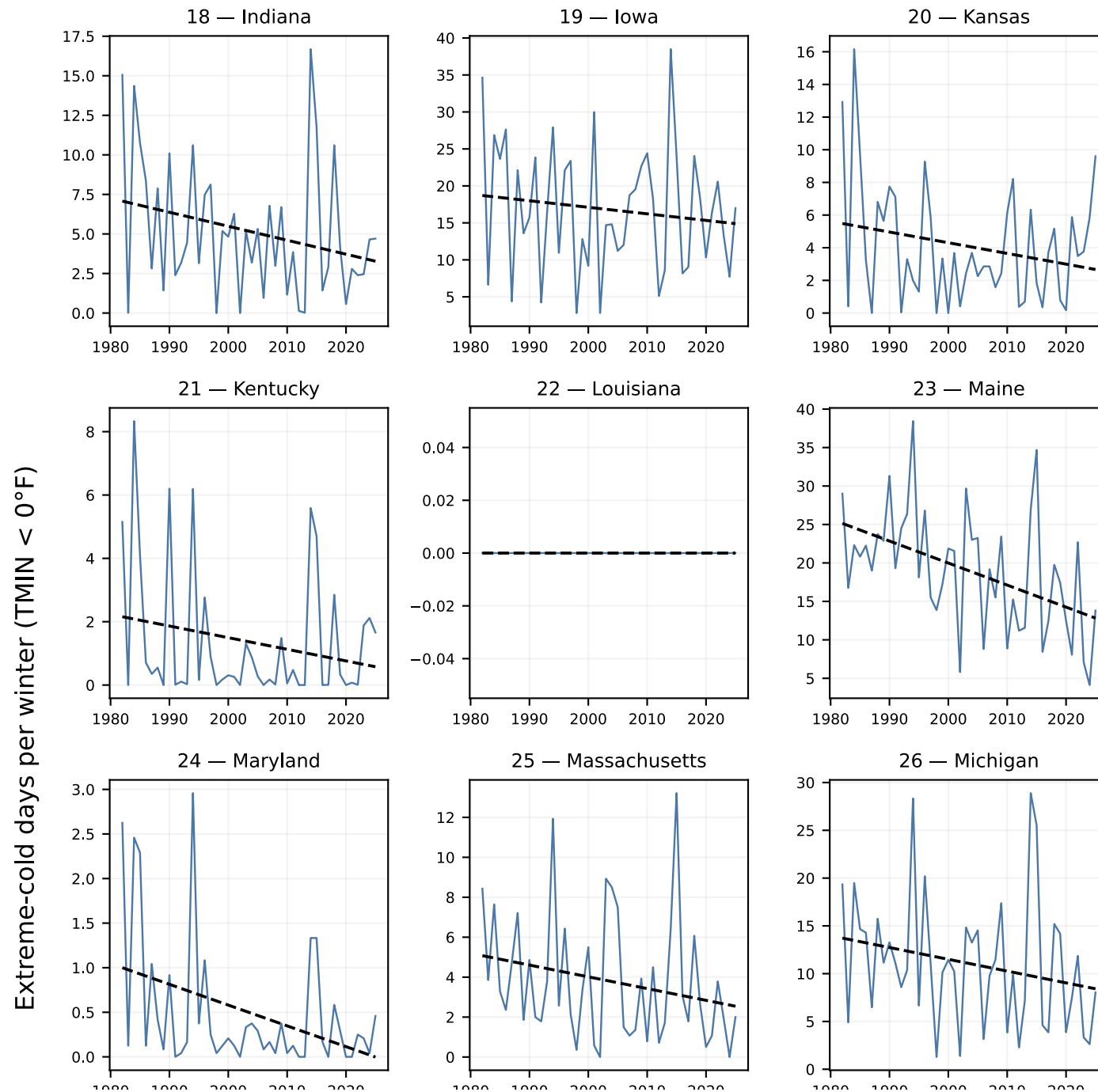
Saved /mnt/data_d/Dropbox/Research/AnimalCollisionsWeather/figures/weather/tier1/prism_state_trends_mean_temper_c.pdf

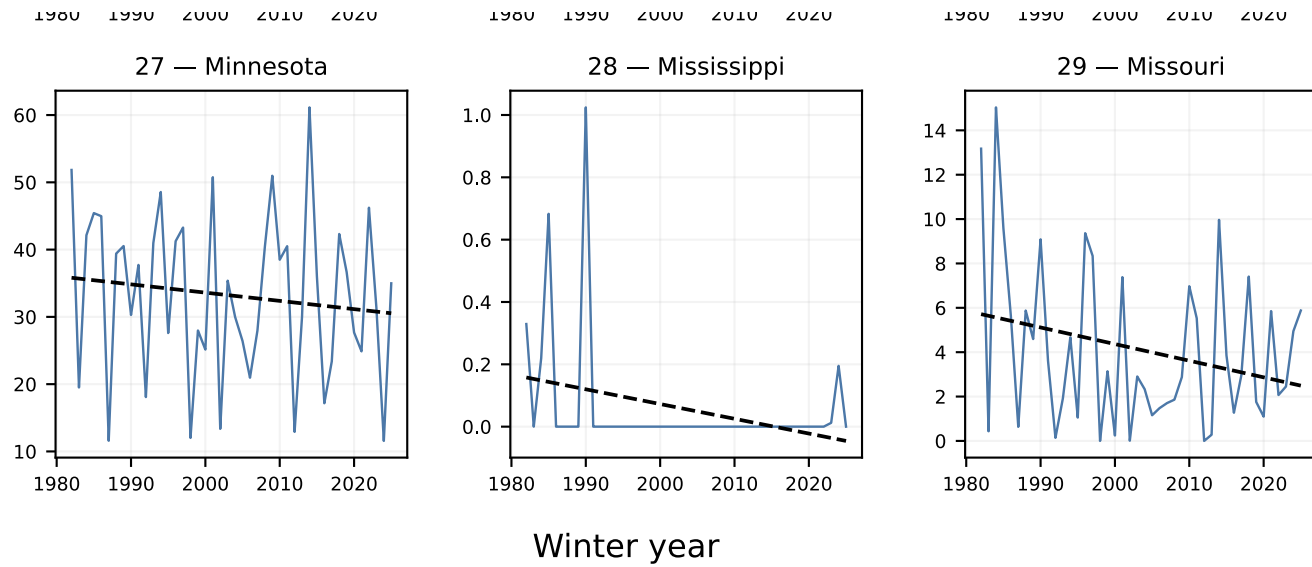
PRISM: Extreme-cold days per winter (TMIN < 0°F) by state



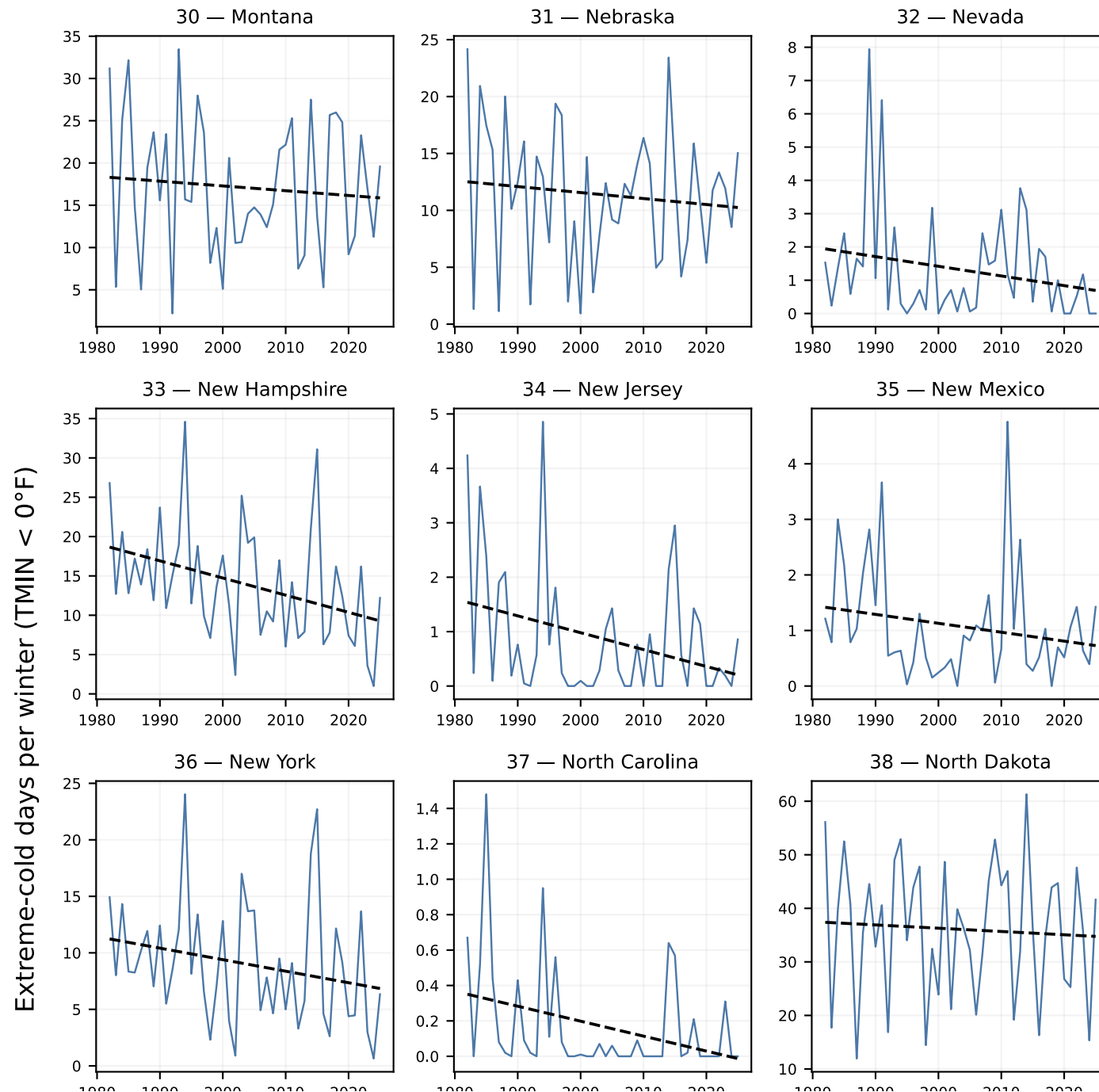


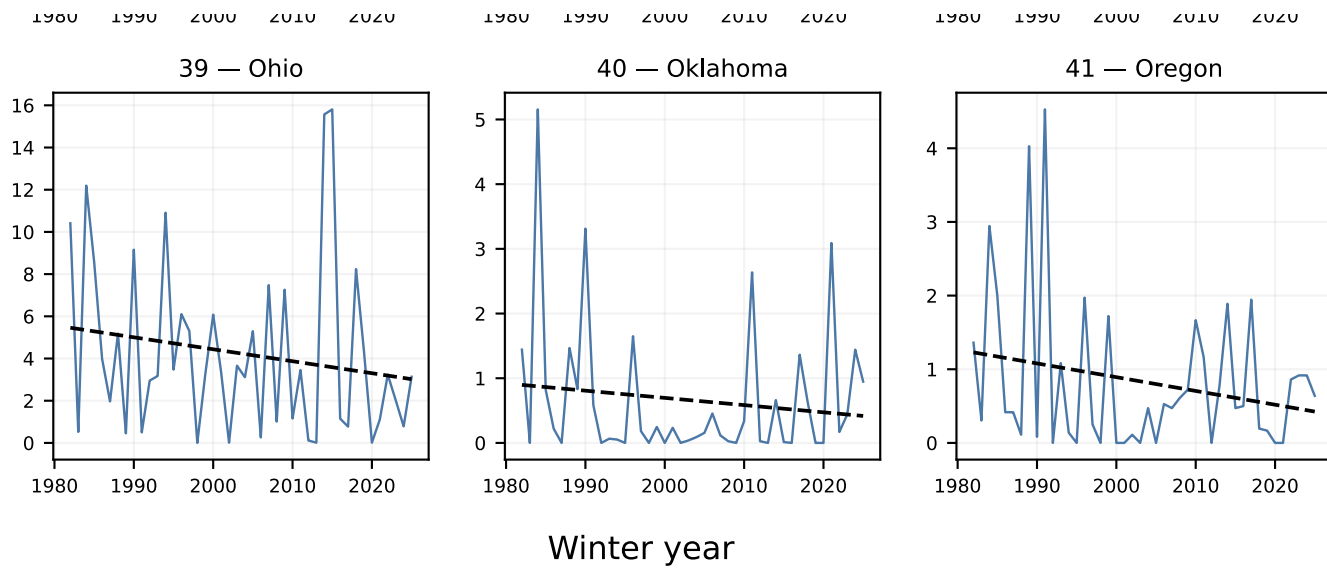
PRISM: Extreme-cold days per winter (TMIN < 0°F) by state



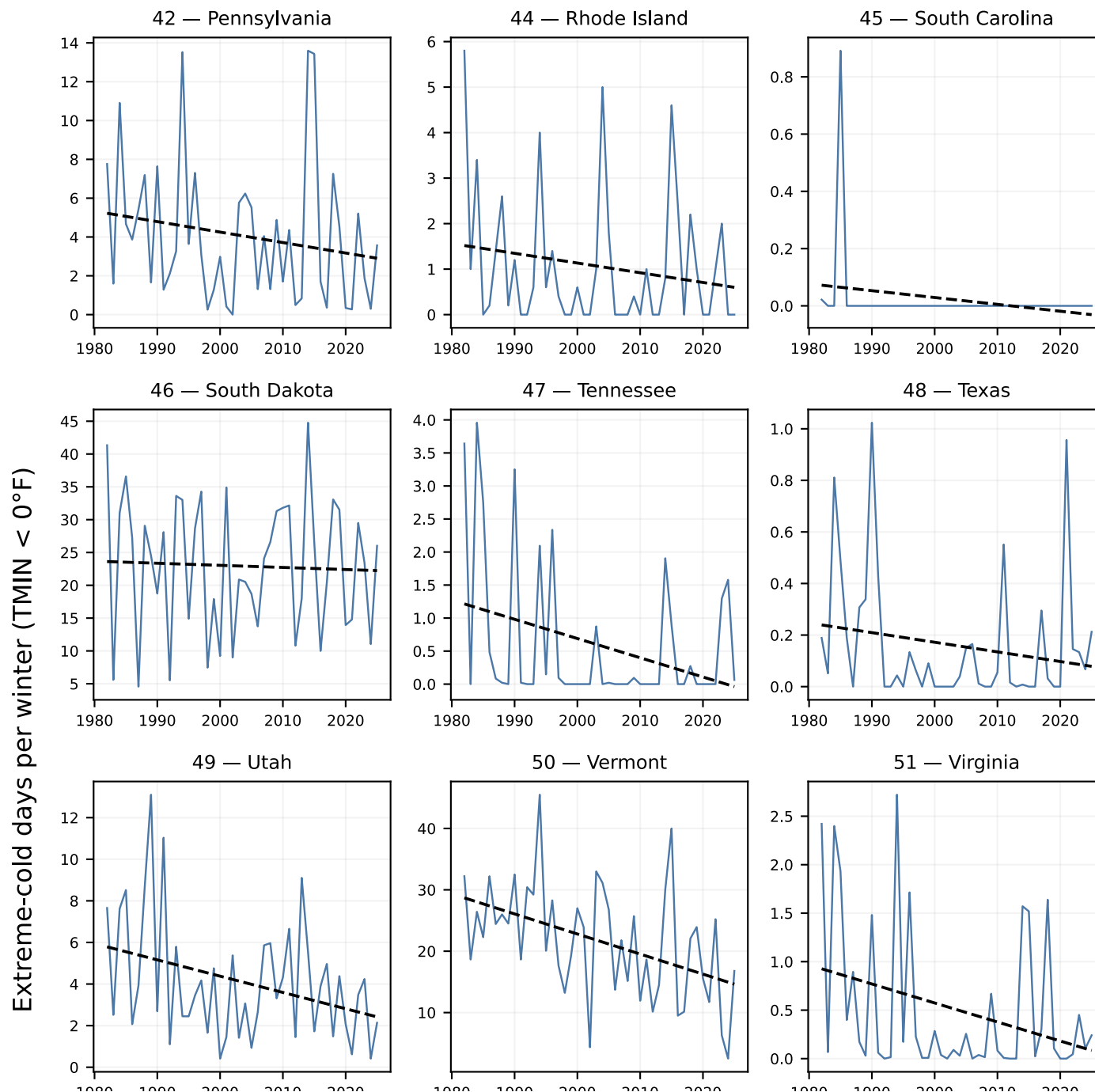


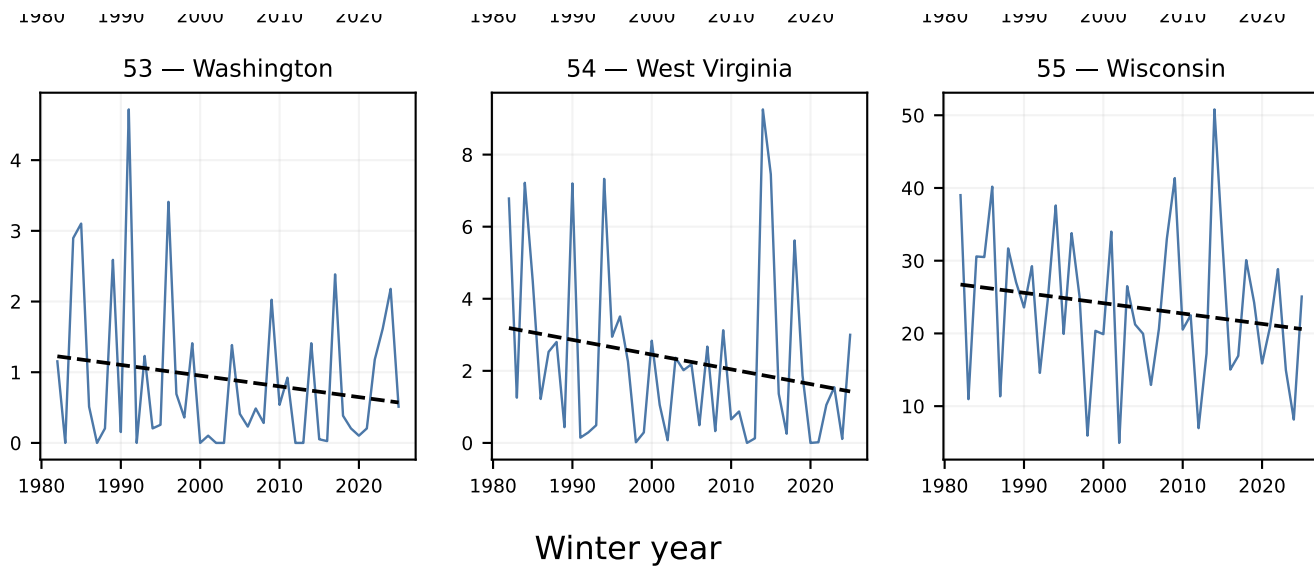
PRISM: Extreme-cold days per winter (TMIN < 0°F) by state





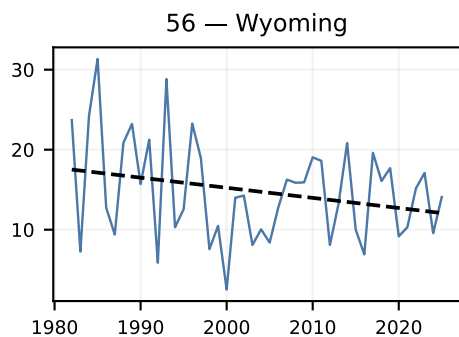
PRISM: Extreme-cold days per winter (TMIN < 0°F) by state





Extreme-cold days per winter (TMIN < 0°F)

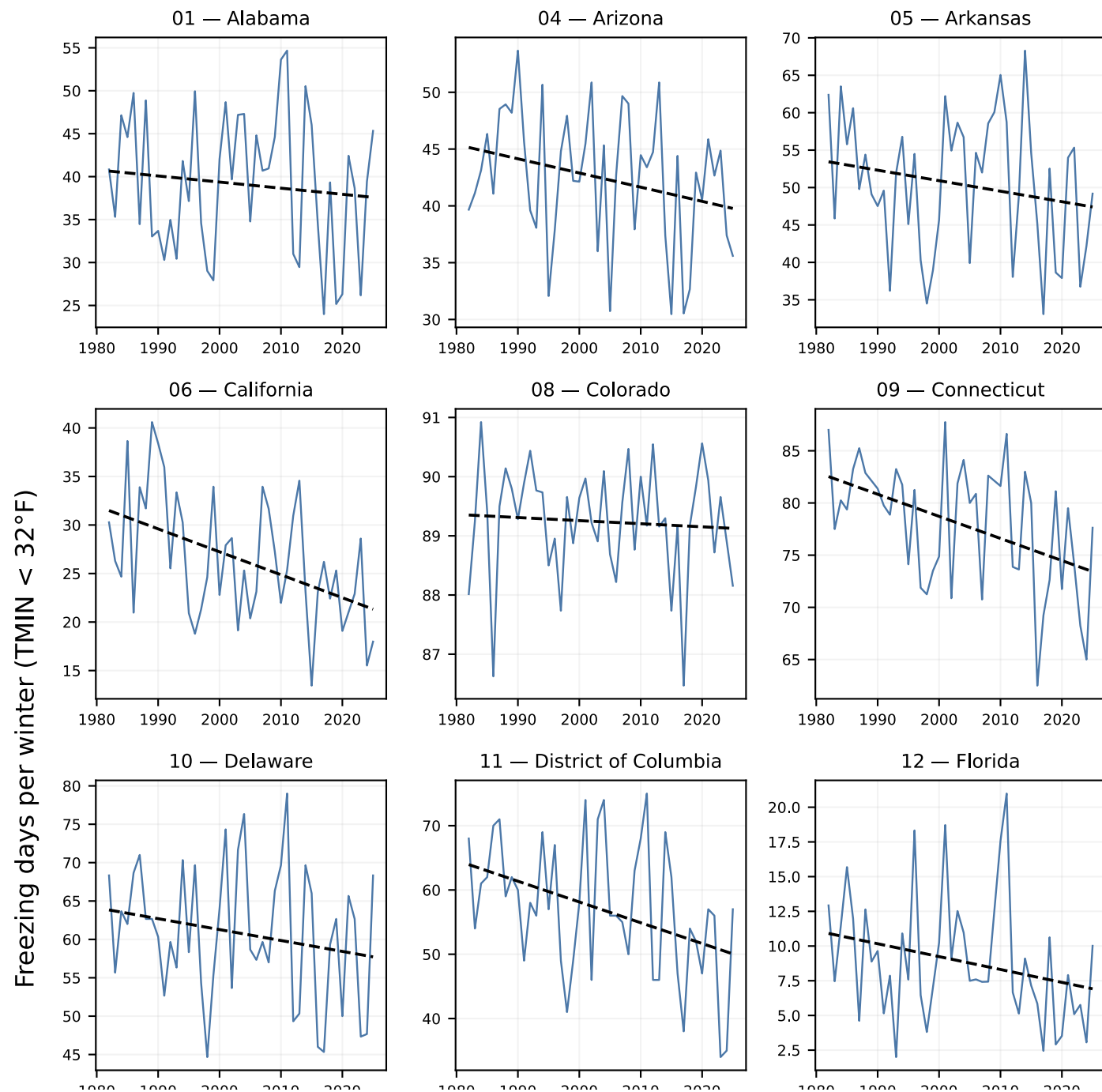
PRISM: Extreme-cold days per winter (TMIN < 0°F) by state

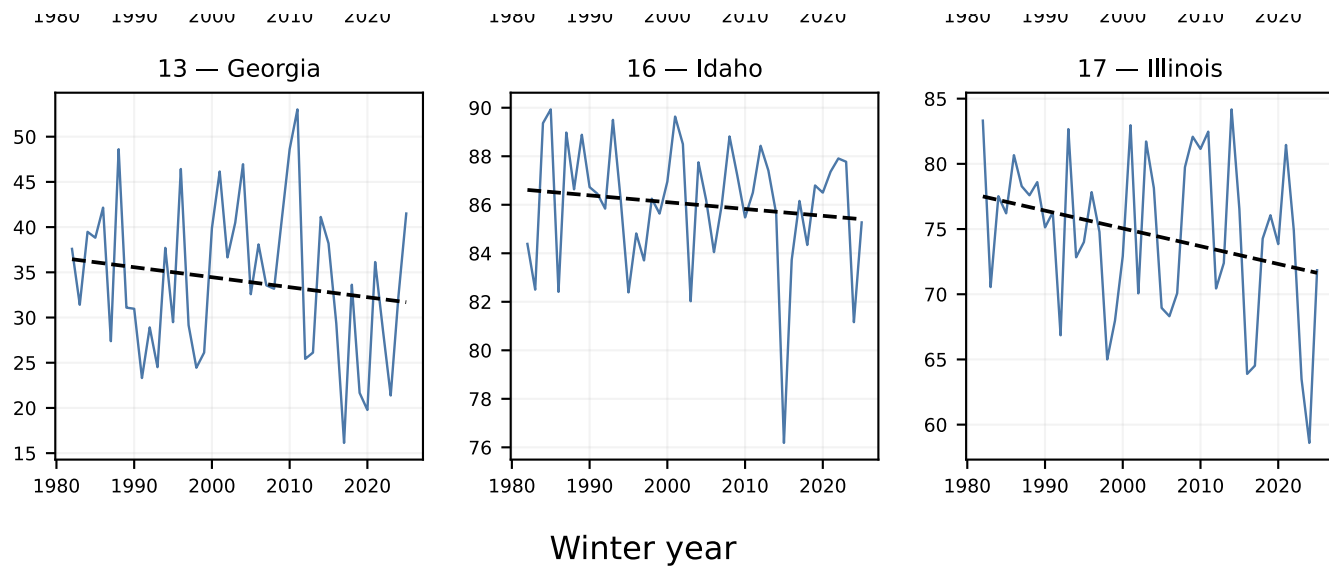


Winter year

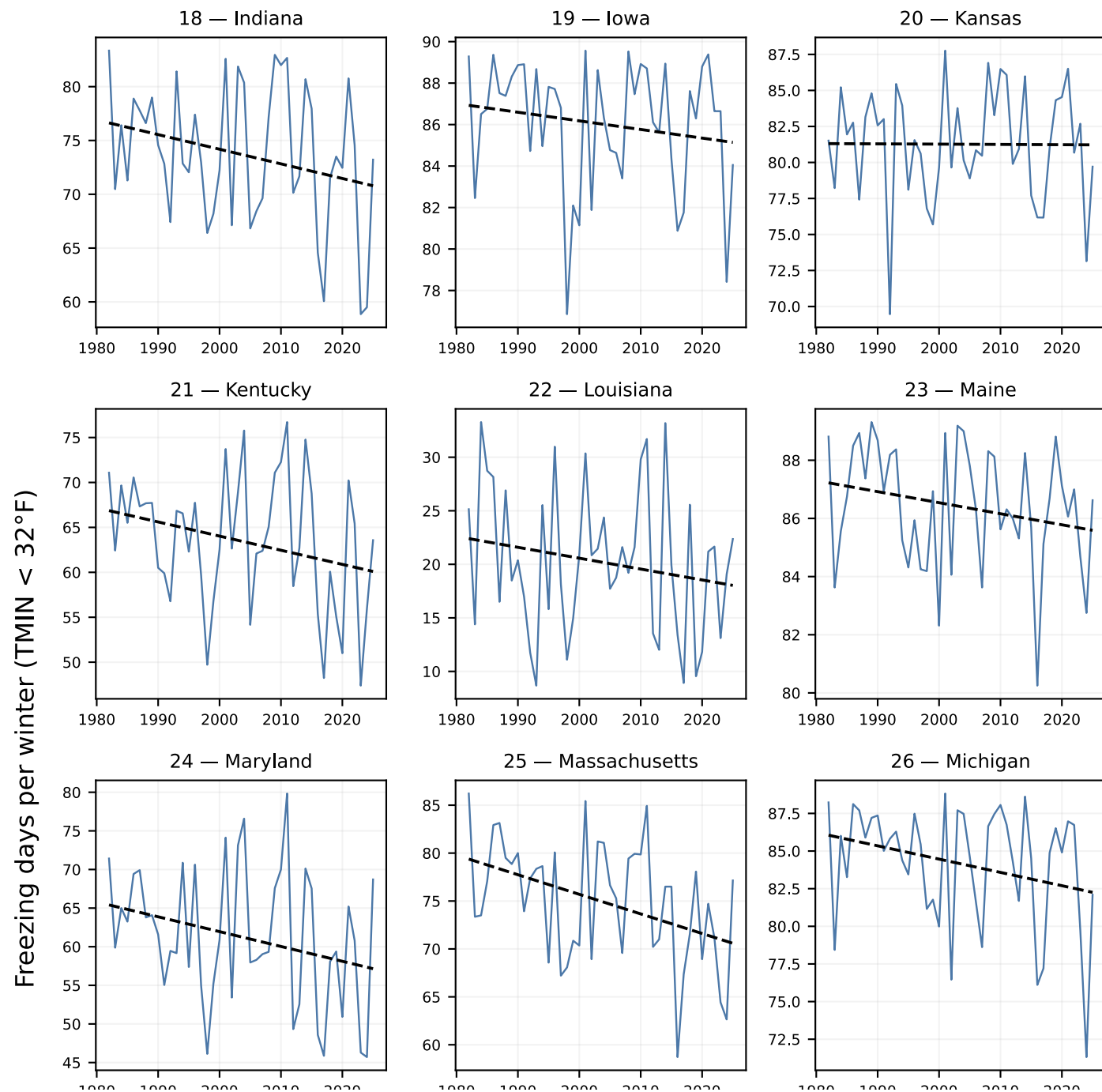
Saved /mnt/data_d/Dropbox/Research/AnimalCollisionsWeather/figures/weather/tier1/prism_state_trends_days_extremely_cold.pdf

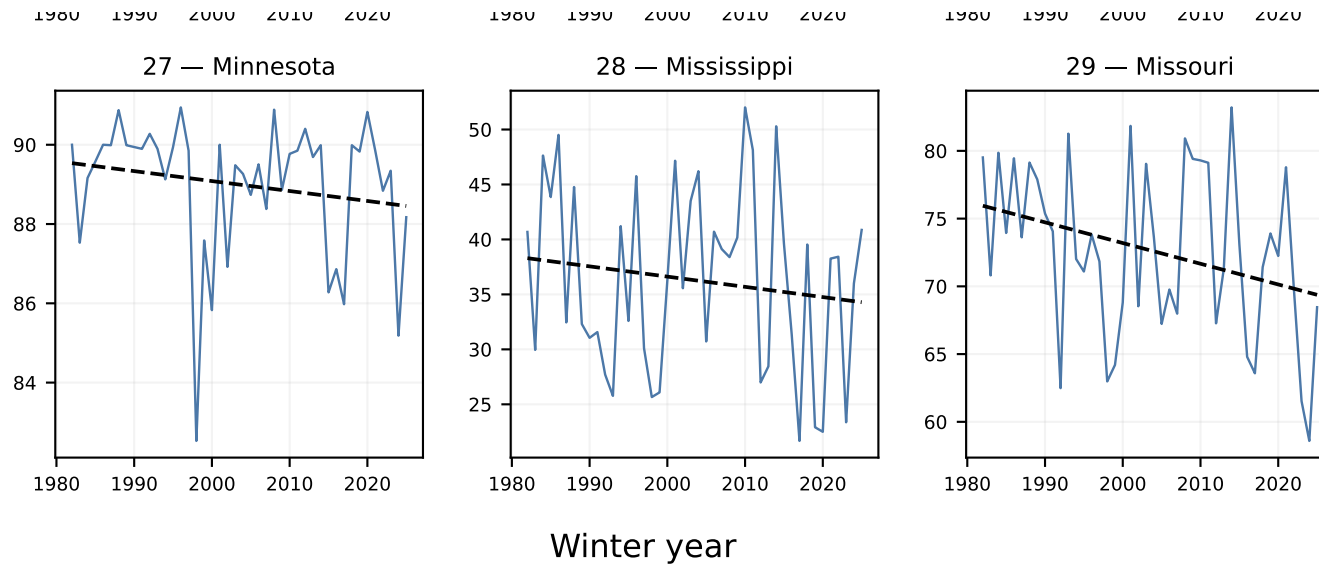
PRISM: Freezing days per winter (TMIN < 32°F) by state



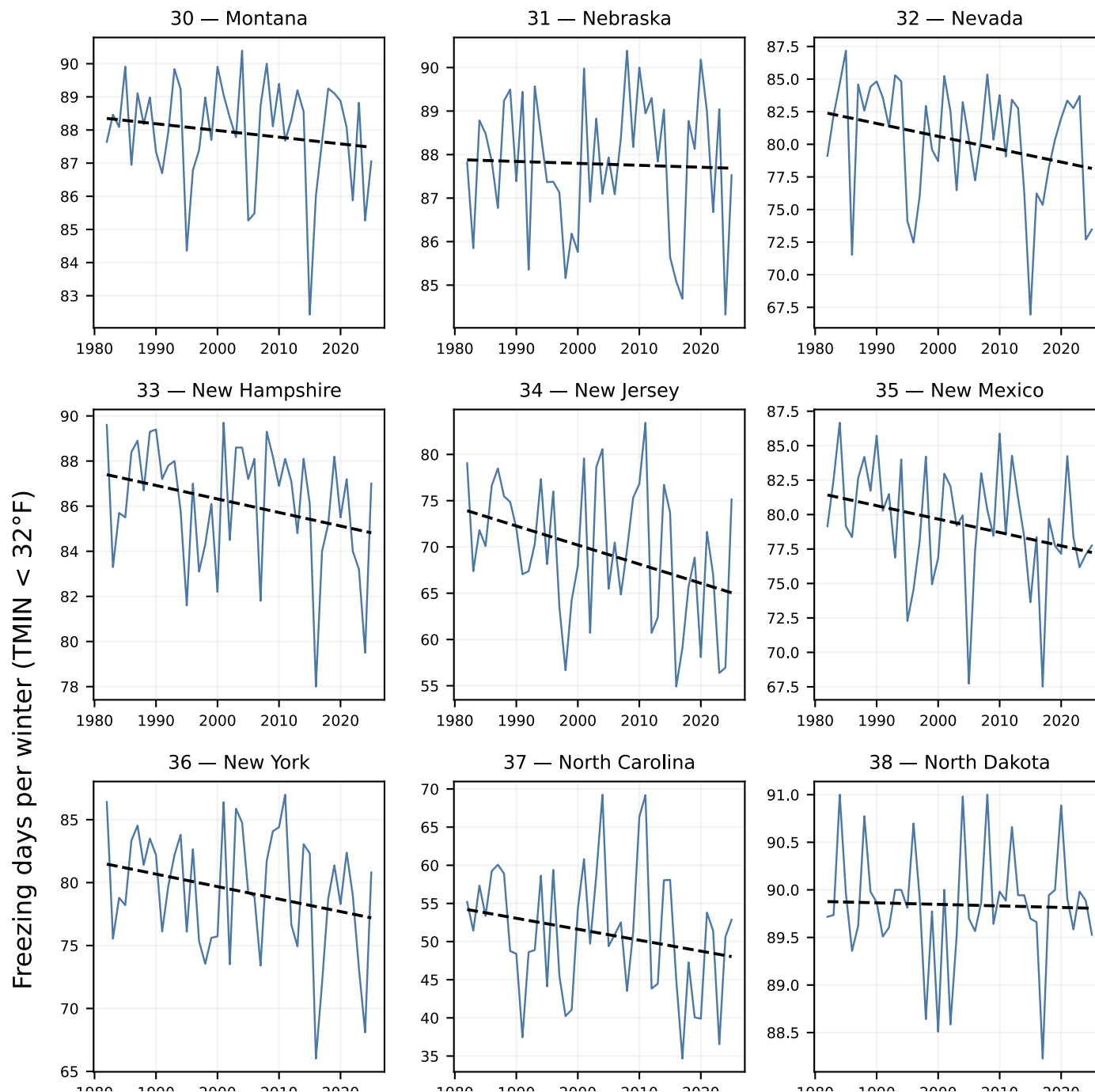


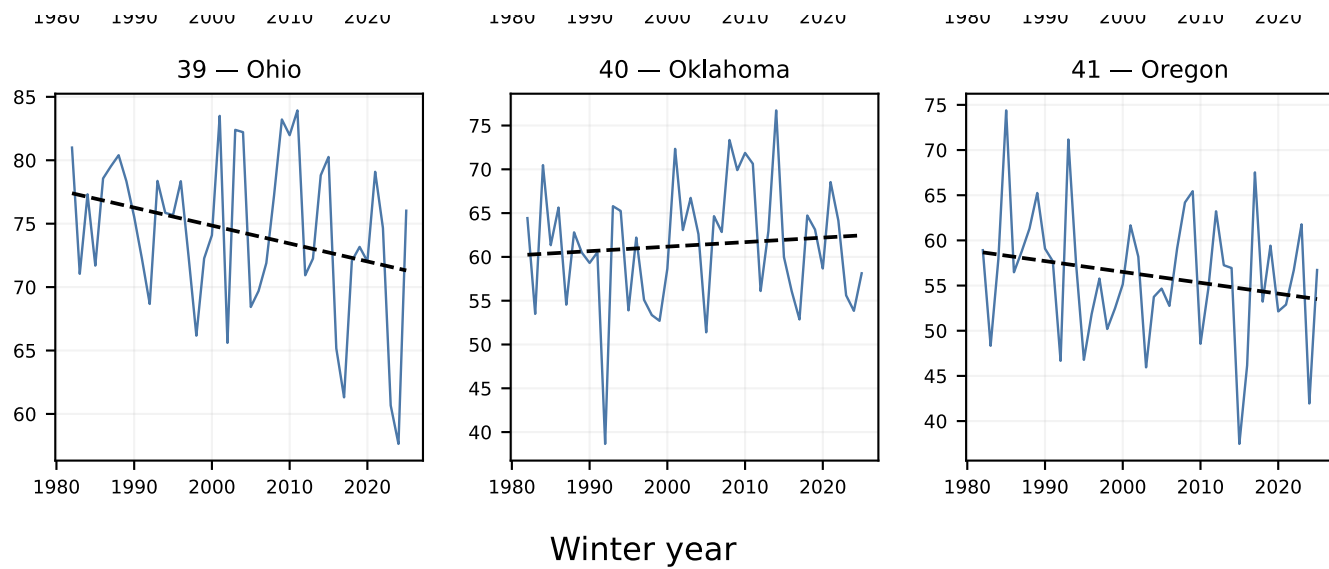
PRISM: Freezing days per winter (TMIN < 32°F) by state



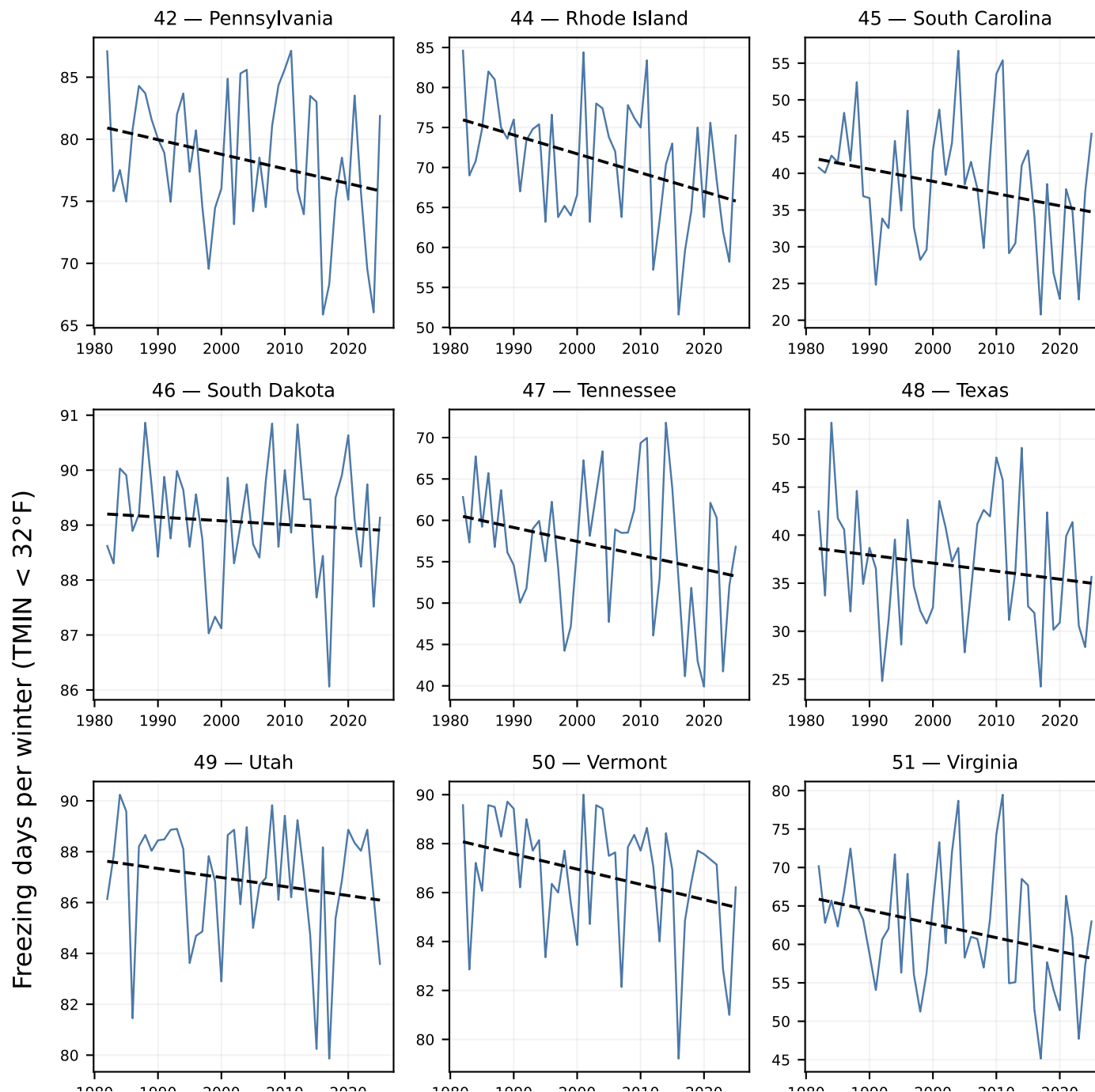


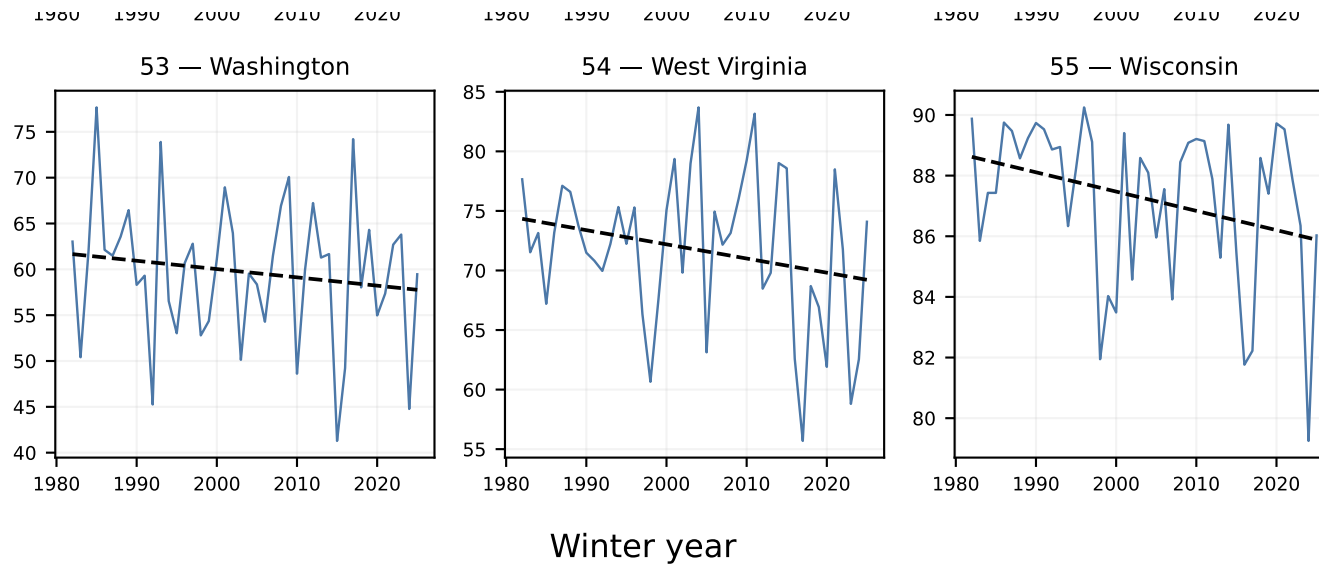
PRISM: Freezing days per winter (TMIN < 32°F) by state





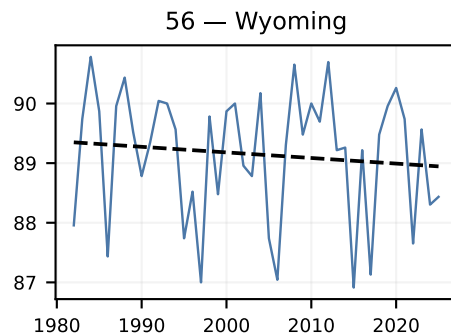
PRISM: Freezing days per winter (TMIN < 32°F) by state





PRISM: Freezing days per winter (TMIN < 32°F) by state

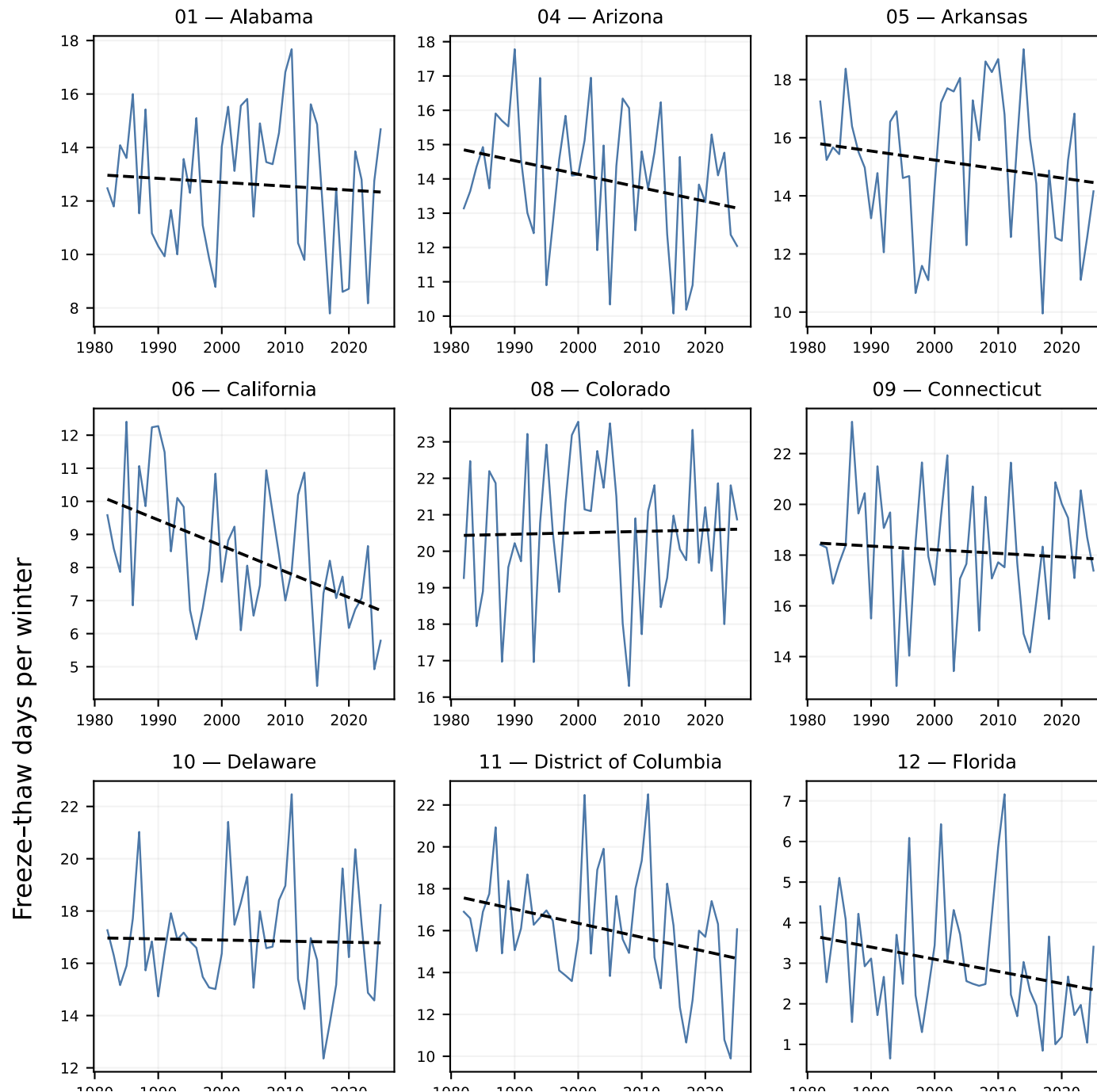
Freezing days per winter (TMIN < 32°F)

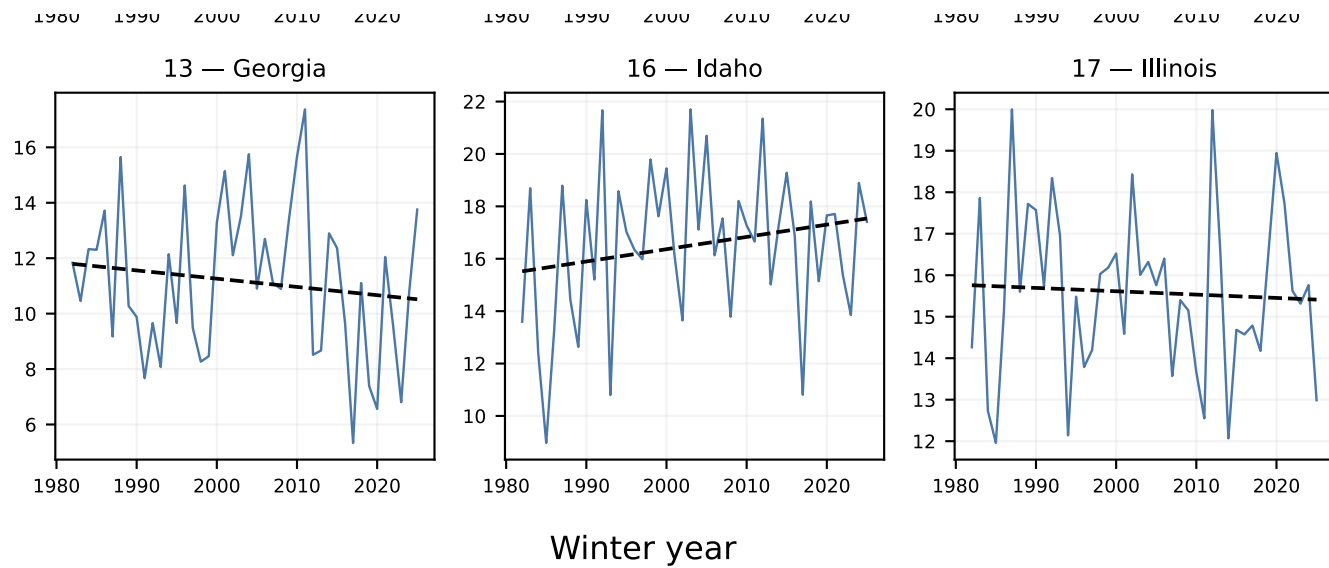


Winter year

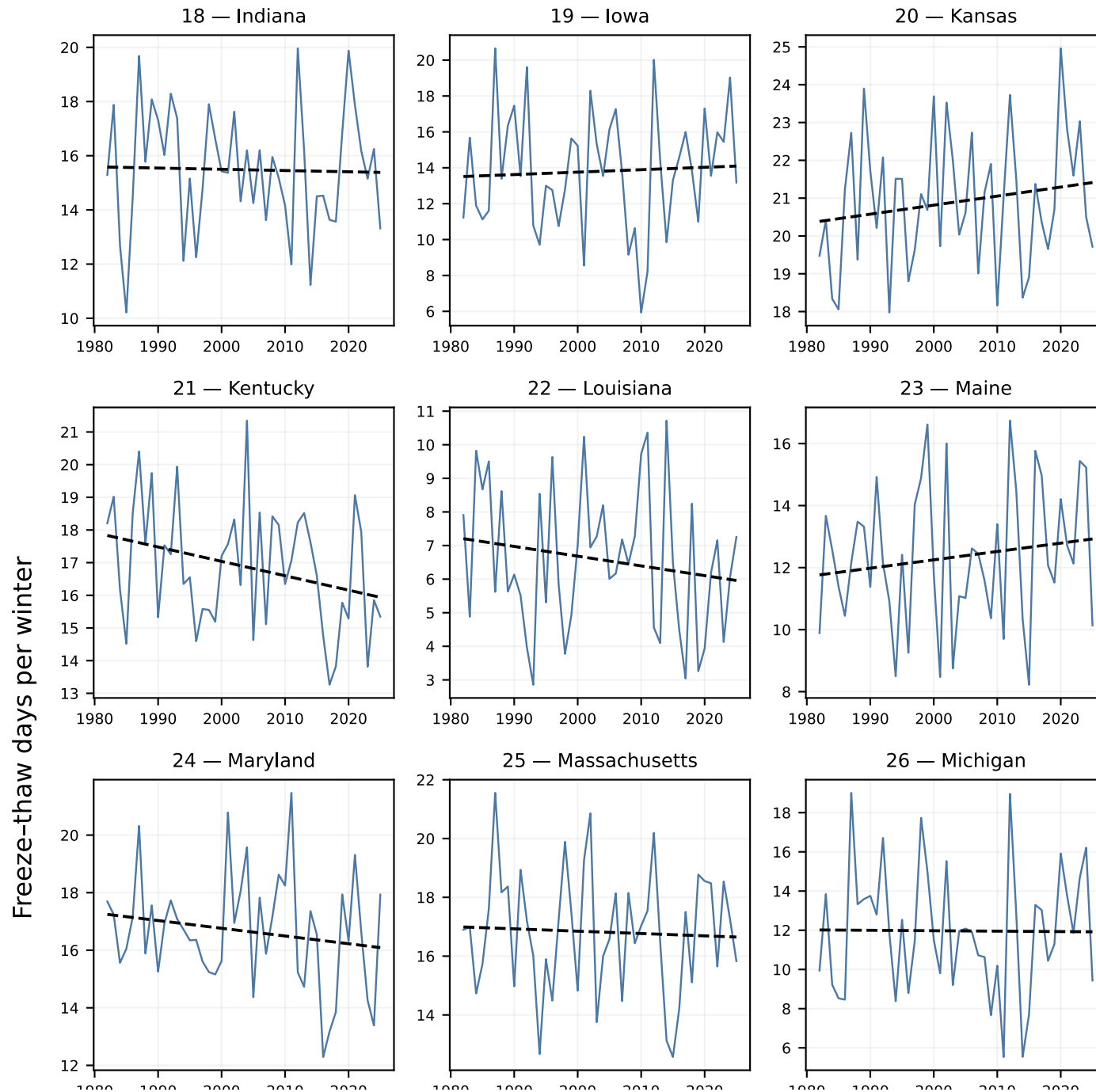
Saved /mnt/data_d/Dropbox/Research/AnimalCollisionsWeather/figures/weather/tier1/prism_state_trends_days_below_freezing_32f.pdf

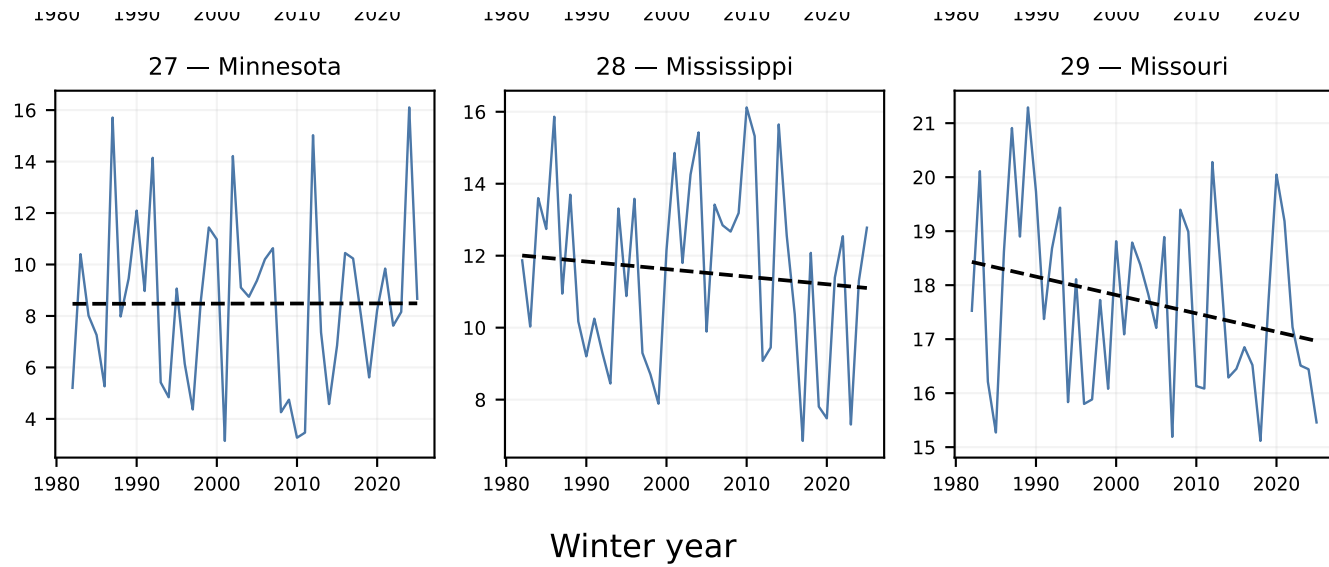
PRISM: Freeze-thaw days per winter by state



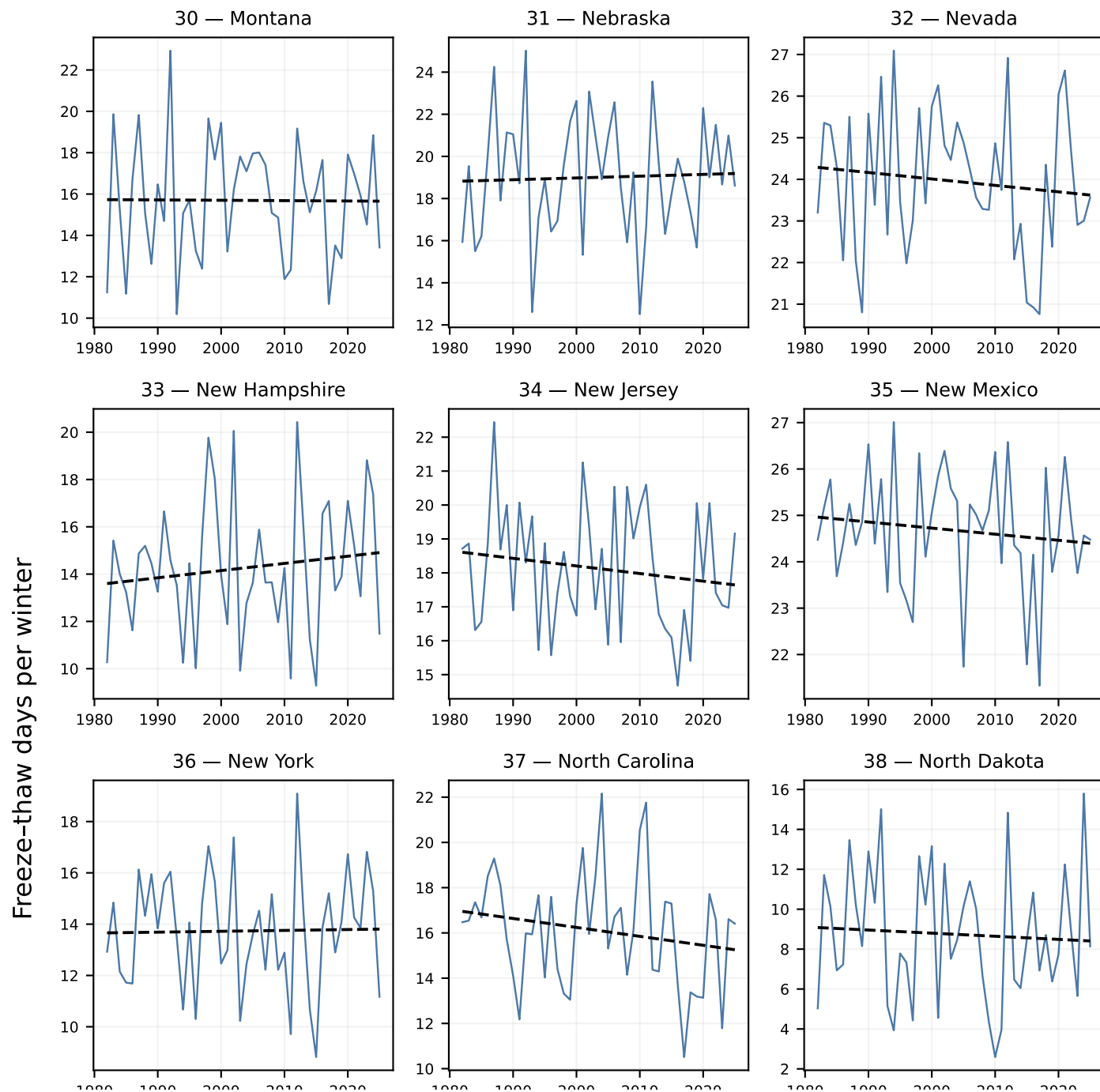


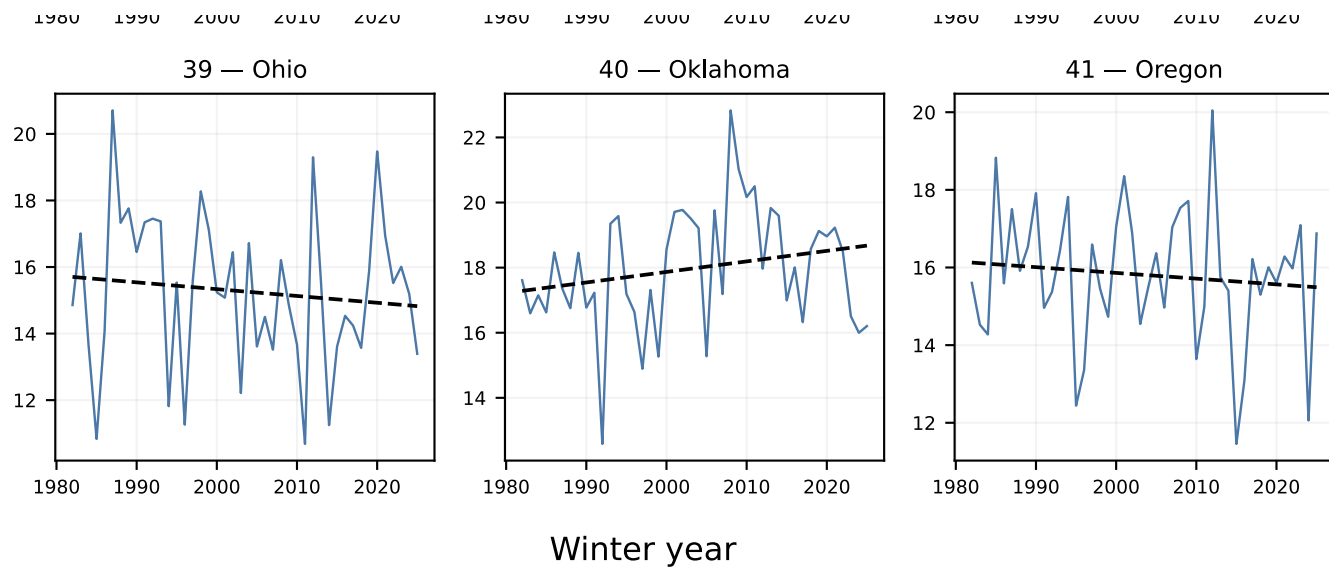
PRISM: Freeze-thaw days per winter by state



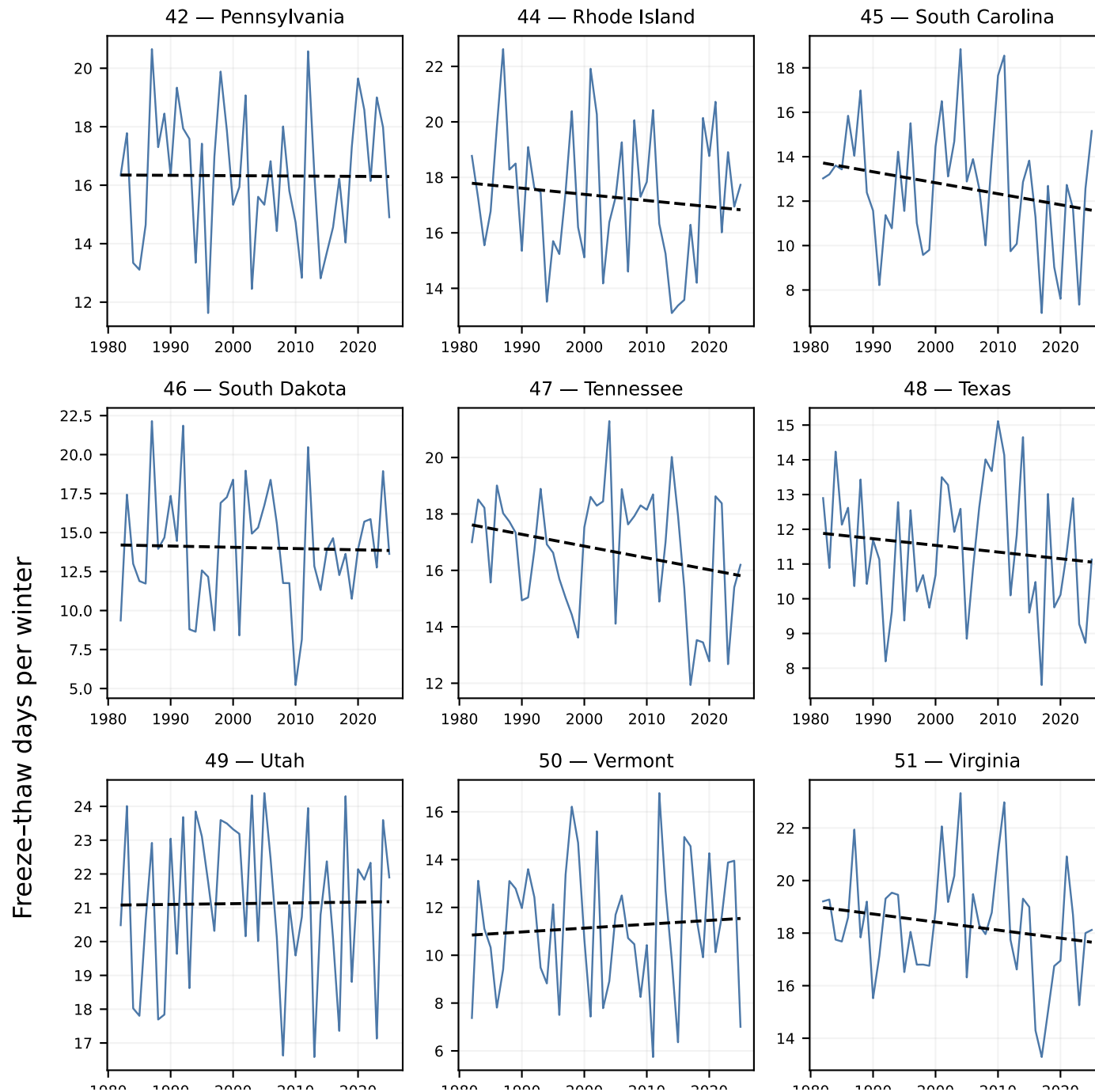


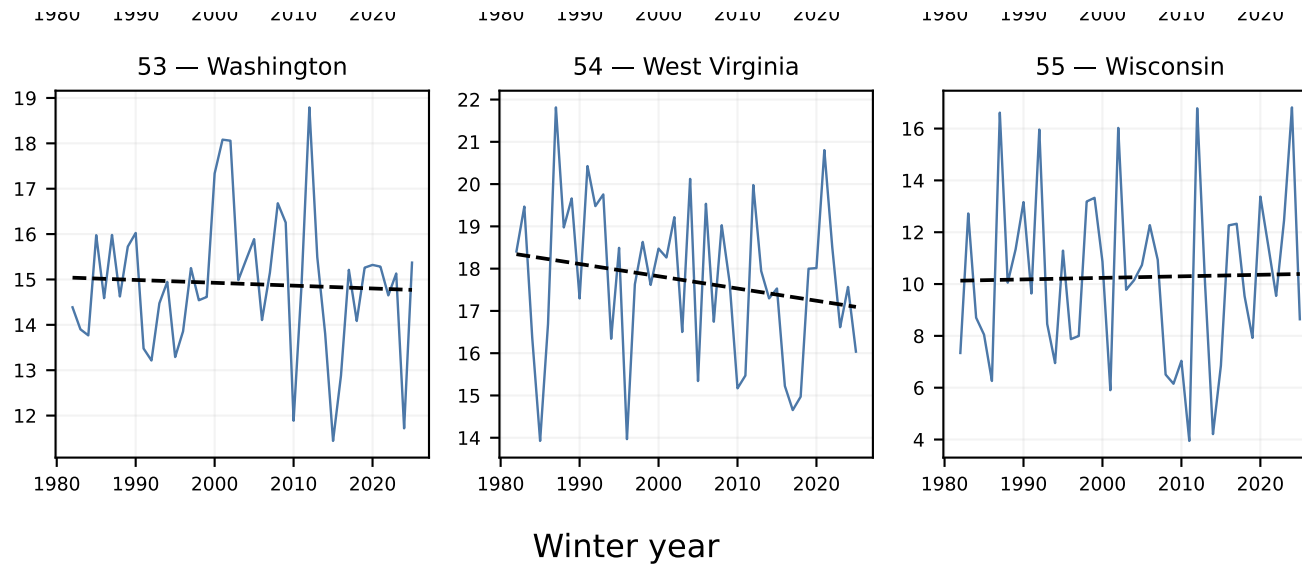
PRISM: Freeze-thaw days per winter by state



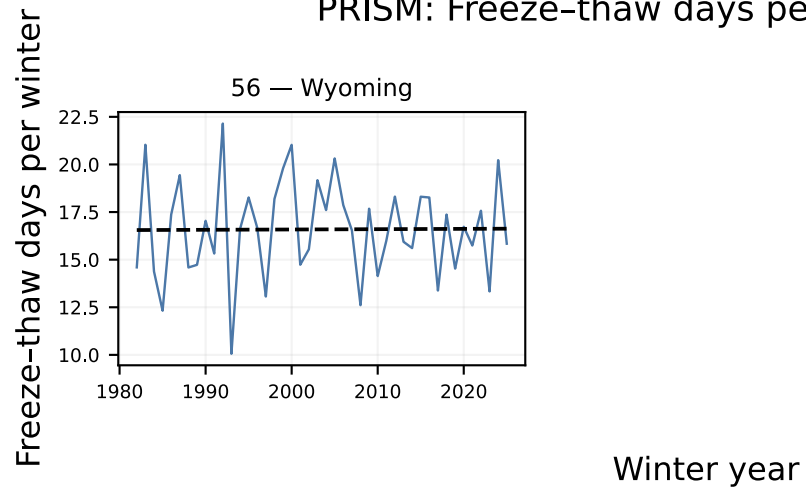


PRISM: Freeze-thaw days per winter by state





PRISM: Freeze-thaw days per winter by state



Saved [/mnt/data_d/Dropbox/Research/AnimalCollisionsWeather/figures/weather/tier1/prism_state_trends_freeze_thaw_days.pdf](#)

9. County spaghetti plots by state

For each state, plot one thin, transparent line per county and a prominent annual state-mean line. Optionally add a distinct dashed linear trend. Use these figures to inspect within-county variation, between-county dispersion, common shocks, outliers,

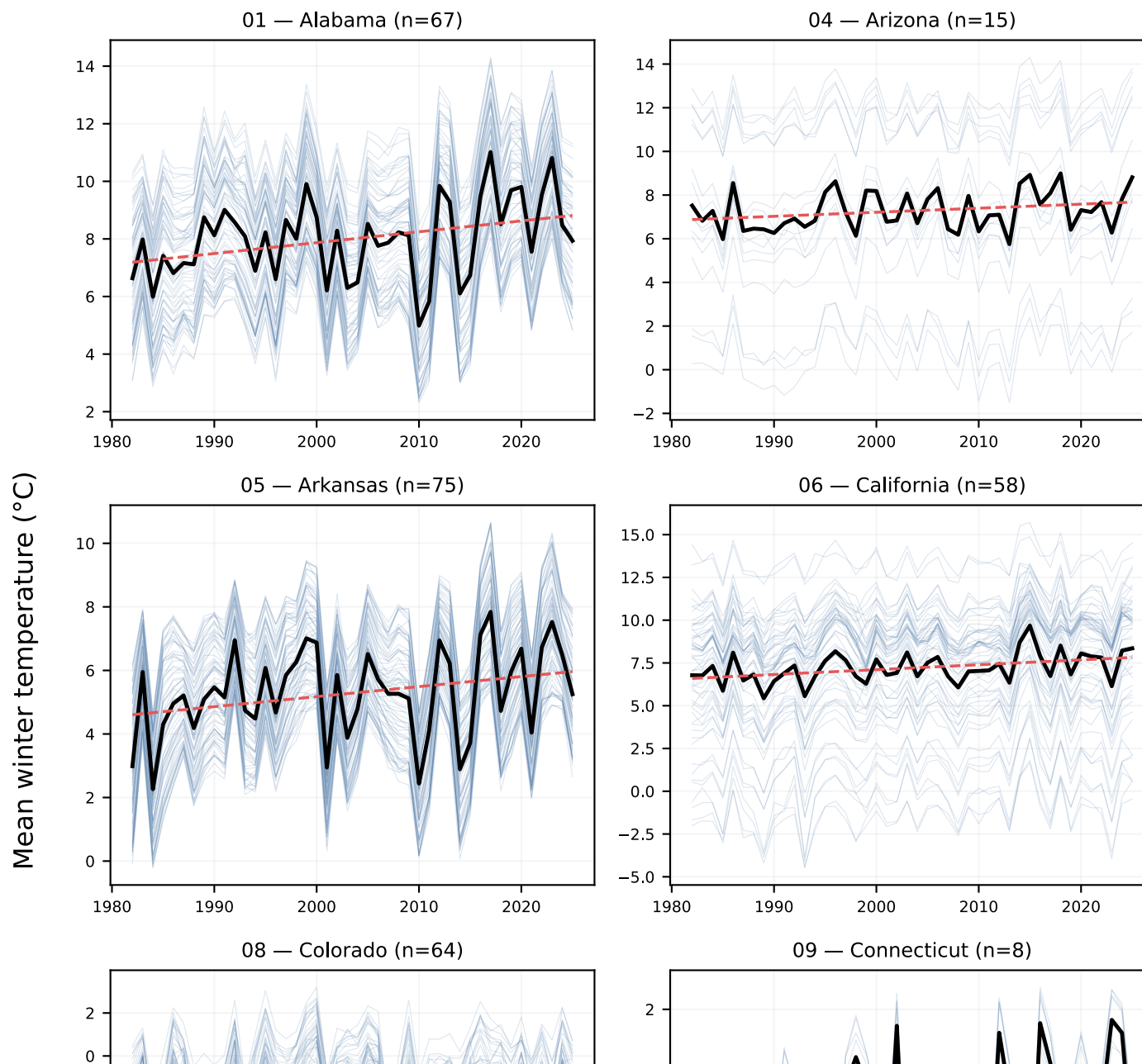
discontinuities, and changes in dispersion over time.

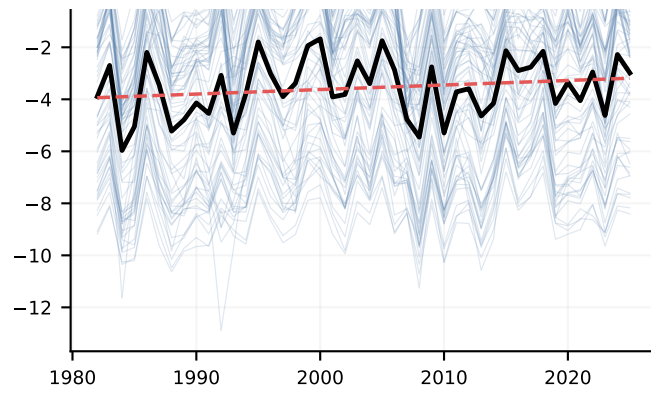
```
In [45]: def save_spaghetti_multipage(frame, variable, states_per_page=6, n_cols=2):
    states = sorted(frame["state_fips"].dropna().unique())
    path = FIGURES_DIR / f"{config.name.lower()}_county_spaghetti_{variable}.pdf"
    with PdfPages(path) as pdf:
        for start in range(0, len(states), states_per_page):
            page_states = states[start:start + states_per_page]
            n_rows = int(np.ceil(len(page_states) / n_cols))
            fig, axes = plt.subplots(n_rows, n_cols, figsize=(7.5, 3.3 * n_rows), squeeze=False)
            for ax, state in zip(axes.flat, page_states):
                state_frame = frame[frame["state_fips"].eq(state)]
                for _, county in state_frame.groupby("geoid"):
                    county = county.sort_values("winter_year")
                    ax.plot(county["winter_year"], county[variable],
                            color="#4C78A8", alpha=0.18, lw=0.45)
                annual_mean = state_frame.groupby("winter_year", as_index=False)[variable].mean()
                ax.plot(annual_mean["winter_year"], annual_mean[variable],
                        color="black", lw=1.8, label="Annual state mean")
                trend = _linear_trend(annual_mean["winter_year"], annual_mean[variable])
                if trend is not None:
                    ax.plot(trend[0], trend[1], color="#E45756", lw=1.3, ls="--",
                            label="State linear trend")
                ax.set_title(f"{state_label(state)} (n={state_frame['geoid'].nunique()})", fontsize=9)
                ax.tick_params(labelsize=7)
                ax.grid(alpha=0.12)
            for ax in list(axes.flat)[len(page_states):]:
                ax.axis("off")
            handles, labels = axes.flat[0].get_legend_handles_labels()
            if handles:
                fig.legend(handles, labels, loc="upper center", ncol=2, frameon=False)
            fig.suptitle(
                f"{config.name}: county trajectories - {variable_label(variable)}",
                fontsize=13, y=0.995,
            )
            fig.supxlabel("Winter year")
            fig.supylabel(variable_label(variable))
            fig.tight_layout(rect=[0, 0, 1, 0.97])
            pdf.savefig(fig, bbox_inches="tight")
            display_figure_inline(fig)
            plt.close(fig)
```

```
    print(f"Saved {path}")
    return path

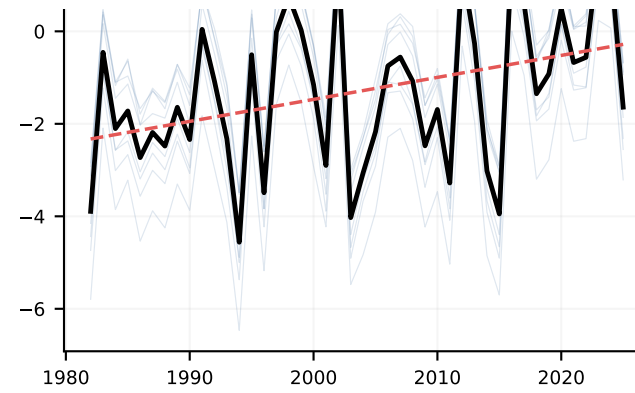
spaghetti_paths = [
    save_spaghetti_multipage(county_winter, variable)
    for variable in SPAGHETTI_VARS if variable in county_winter
]
```


PRISM: county trajectories — Mean winter temperature (°C)



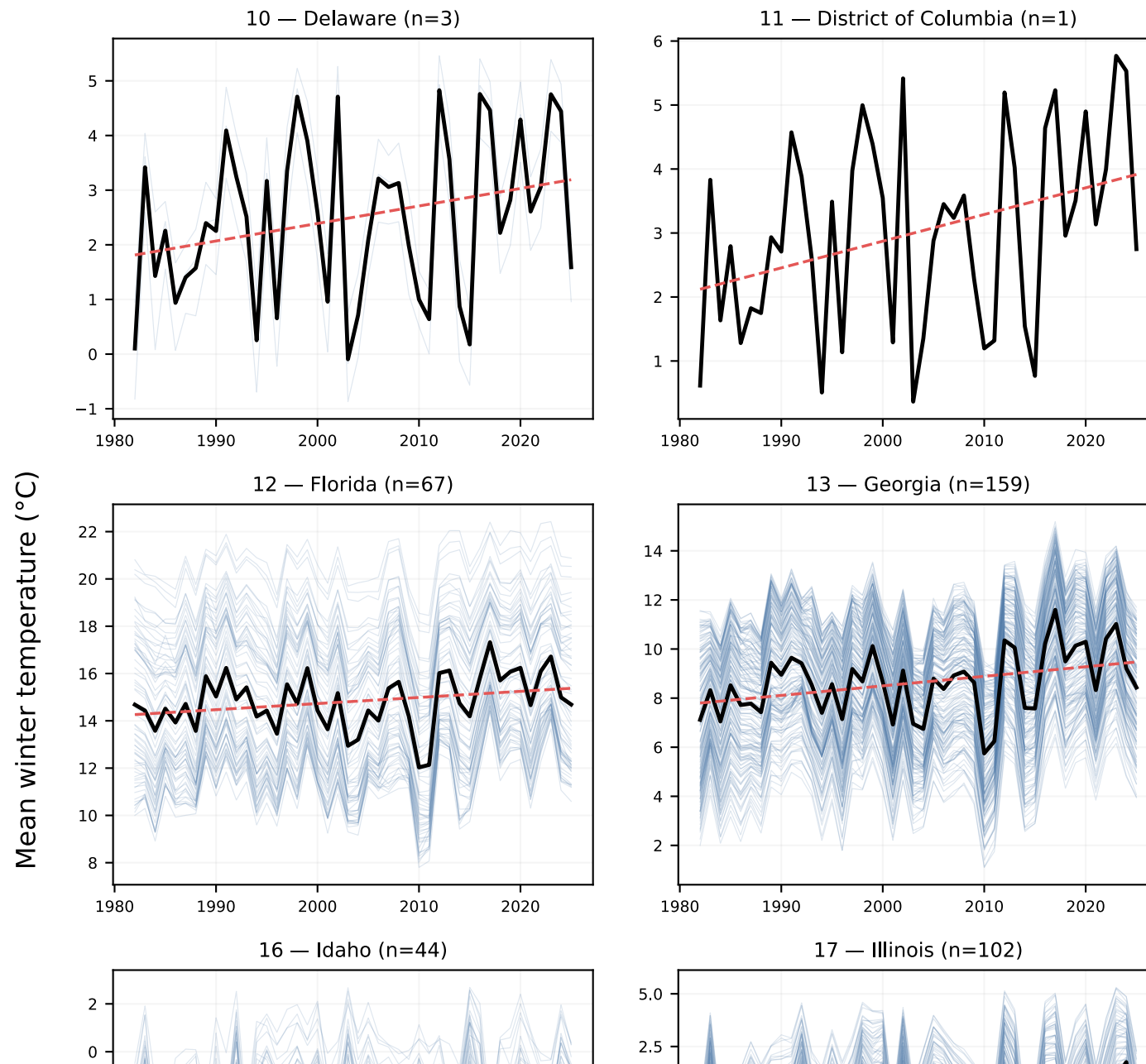


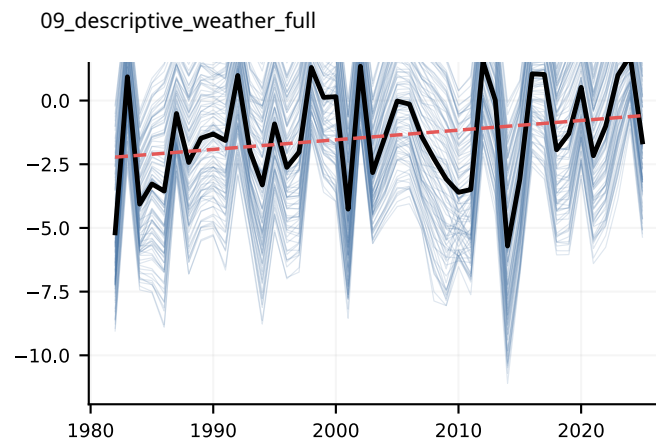
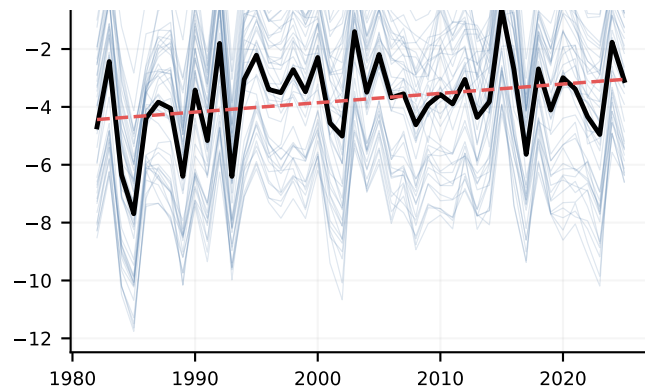
09_descriptive_weather_full



Winter year

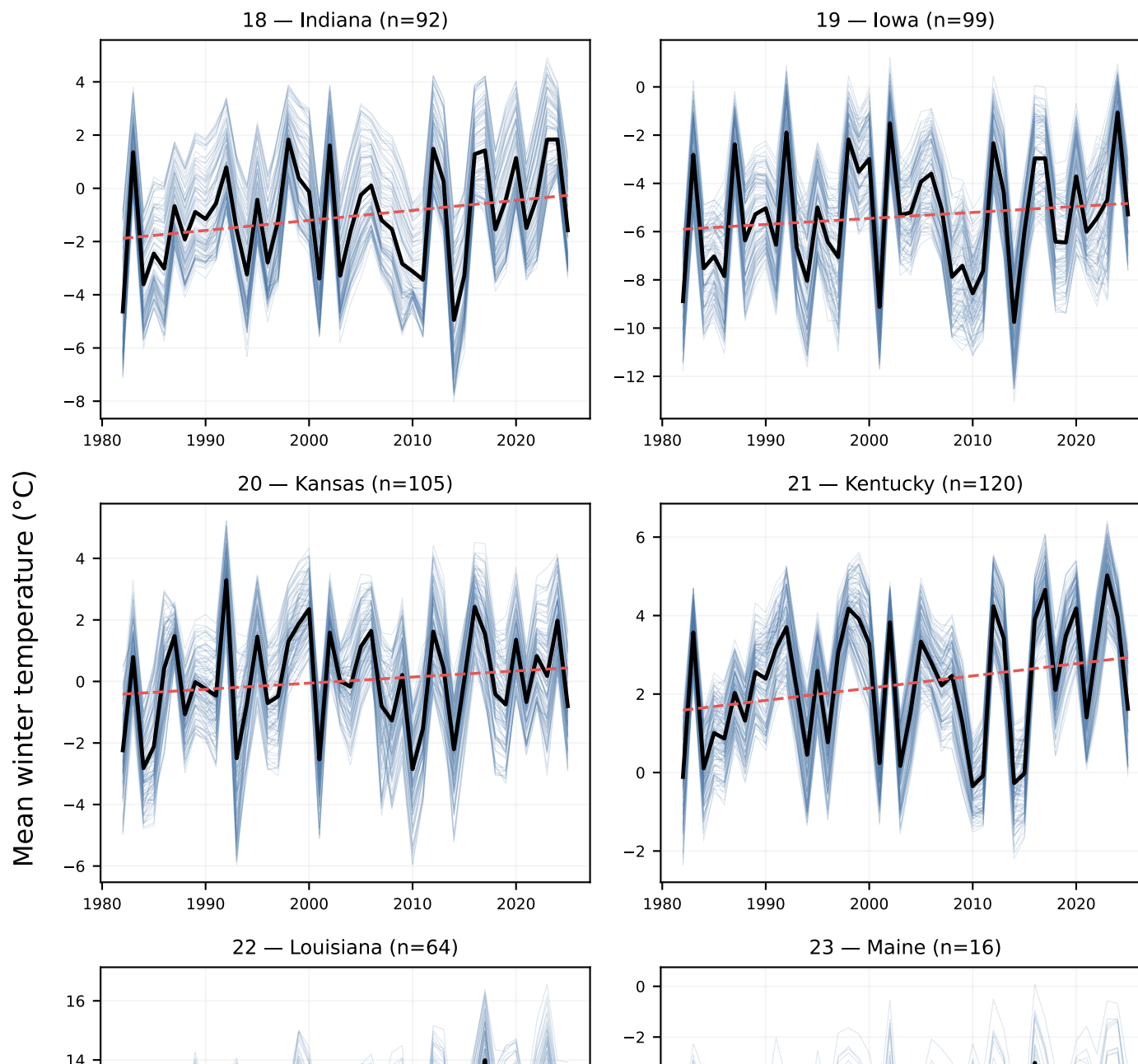
PRISM: county trajectories — Mean winter temperature (°C)

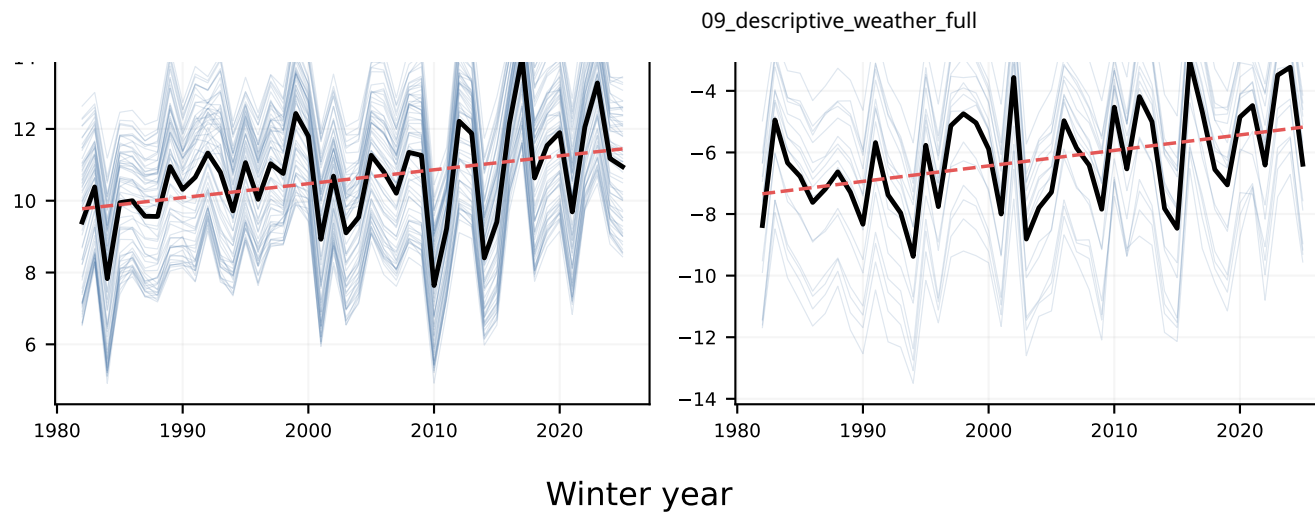




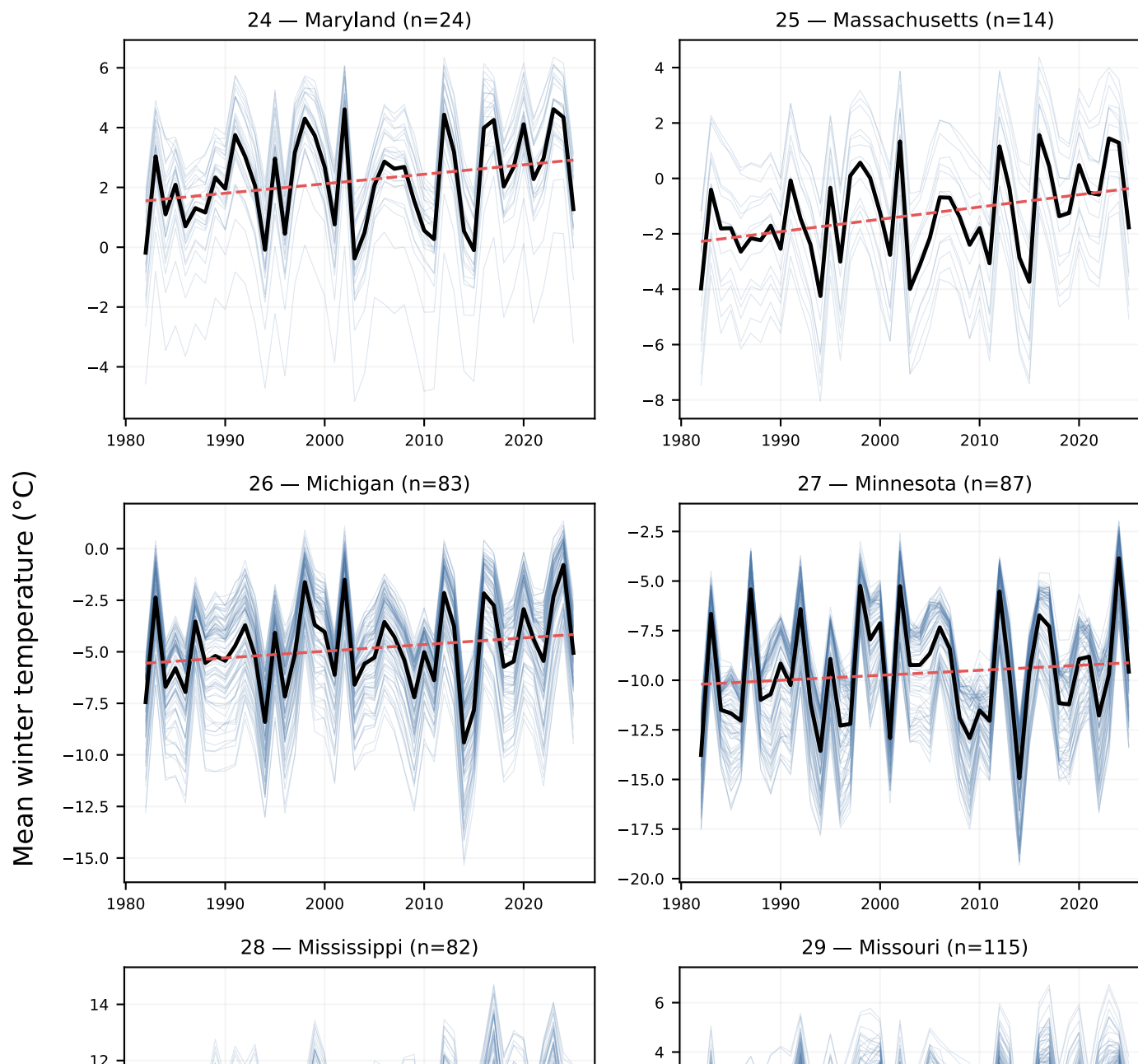
Winter year

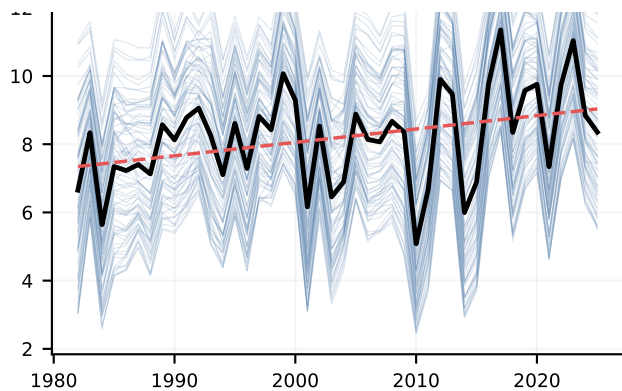
PRISM: county trajectories — Mean winter temperature (°C)



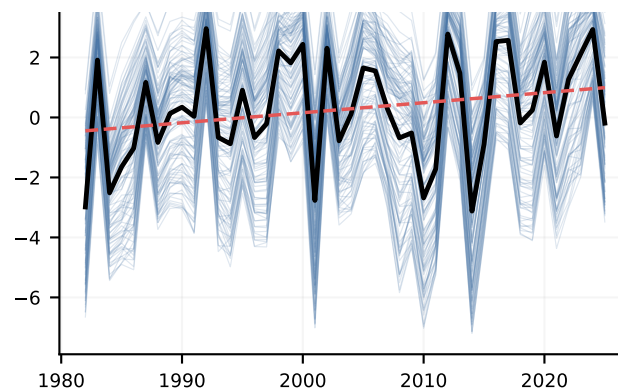


PRISM: county trajectories — Mean winter temperature (°C)



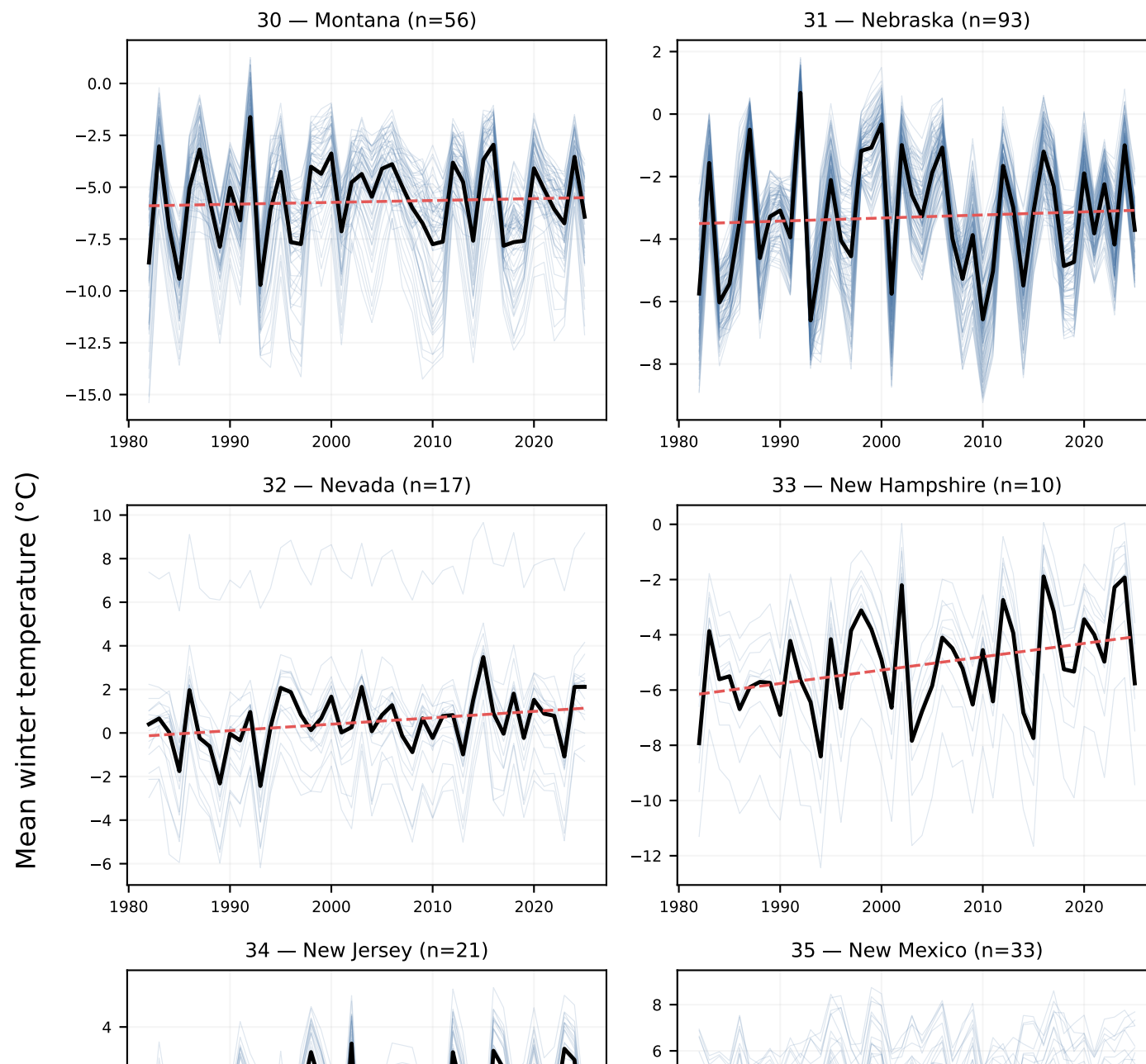


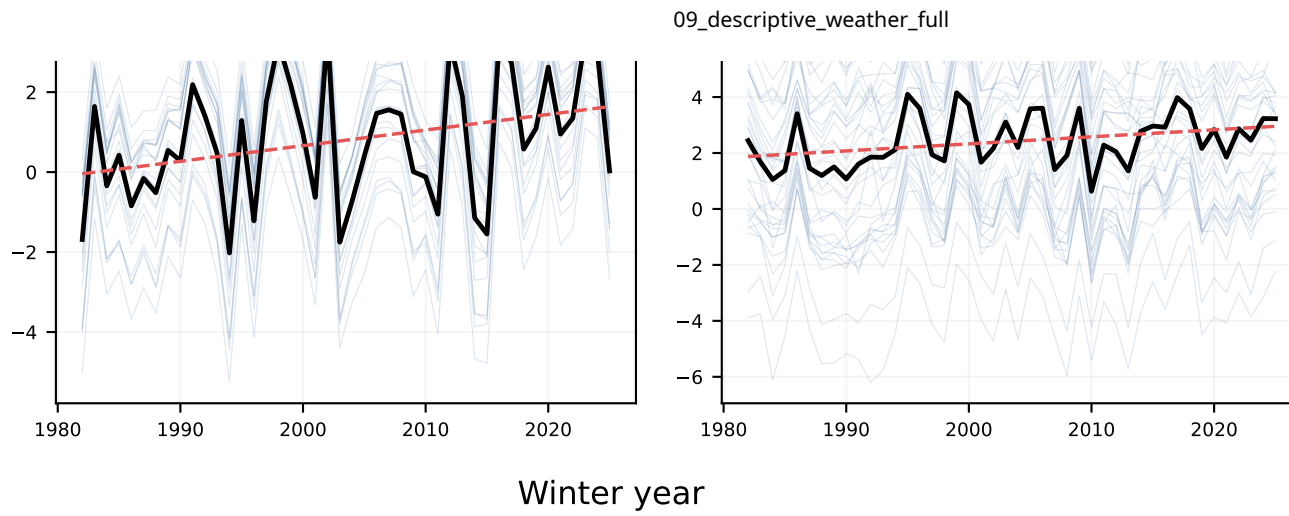
09_descriptive_weather_full



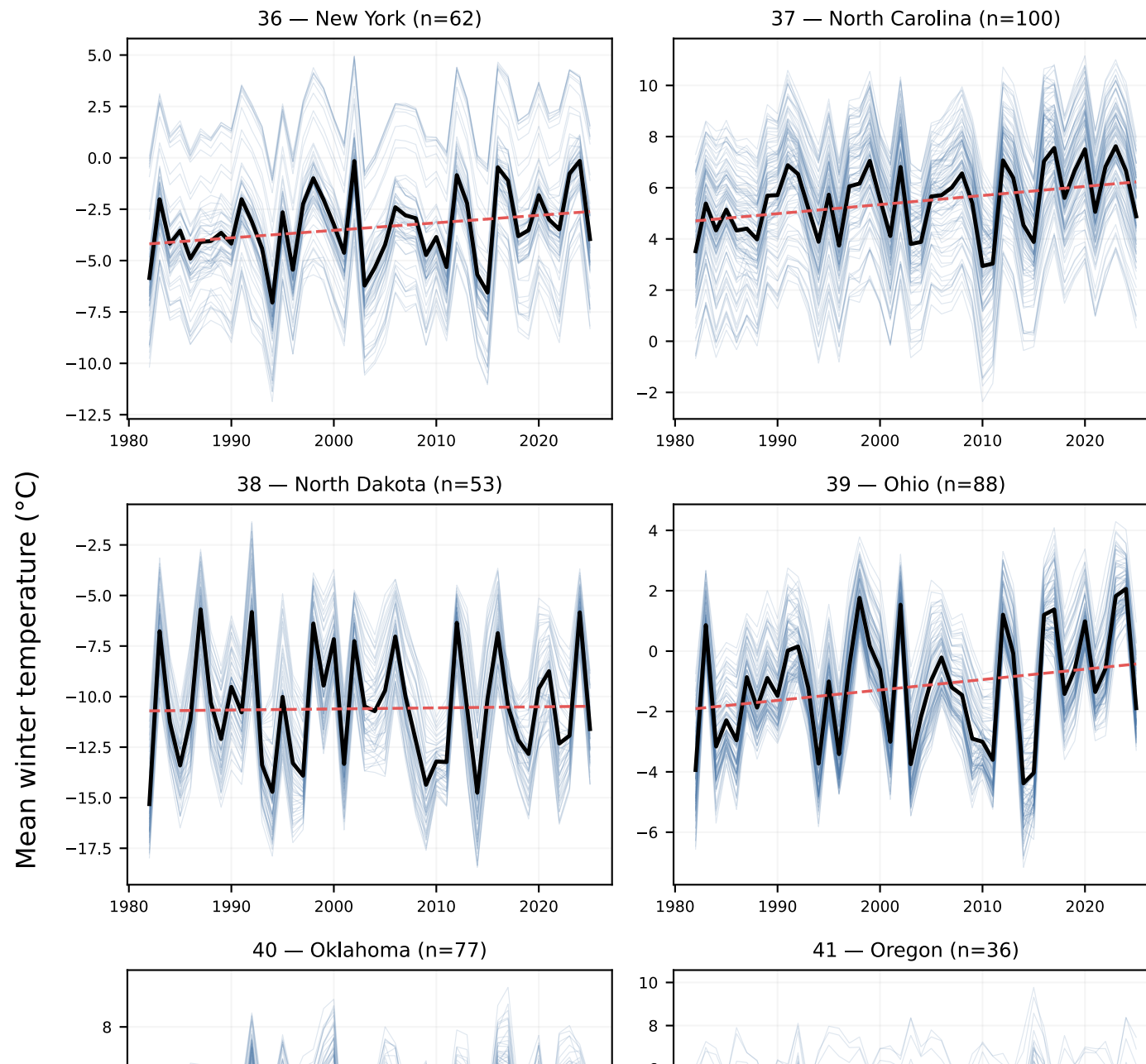
Winter year

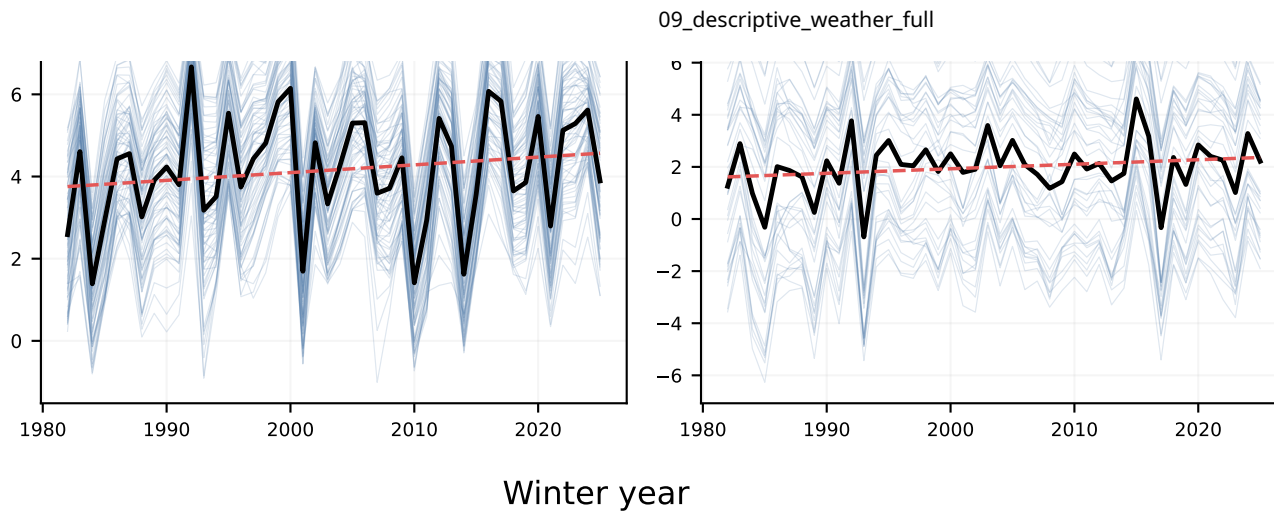
PRISM: county trajectories — Mean winter temperature (°C)



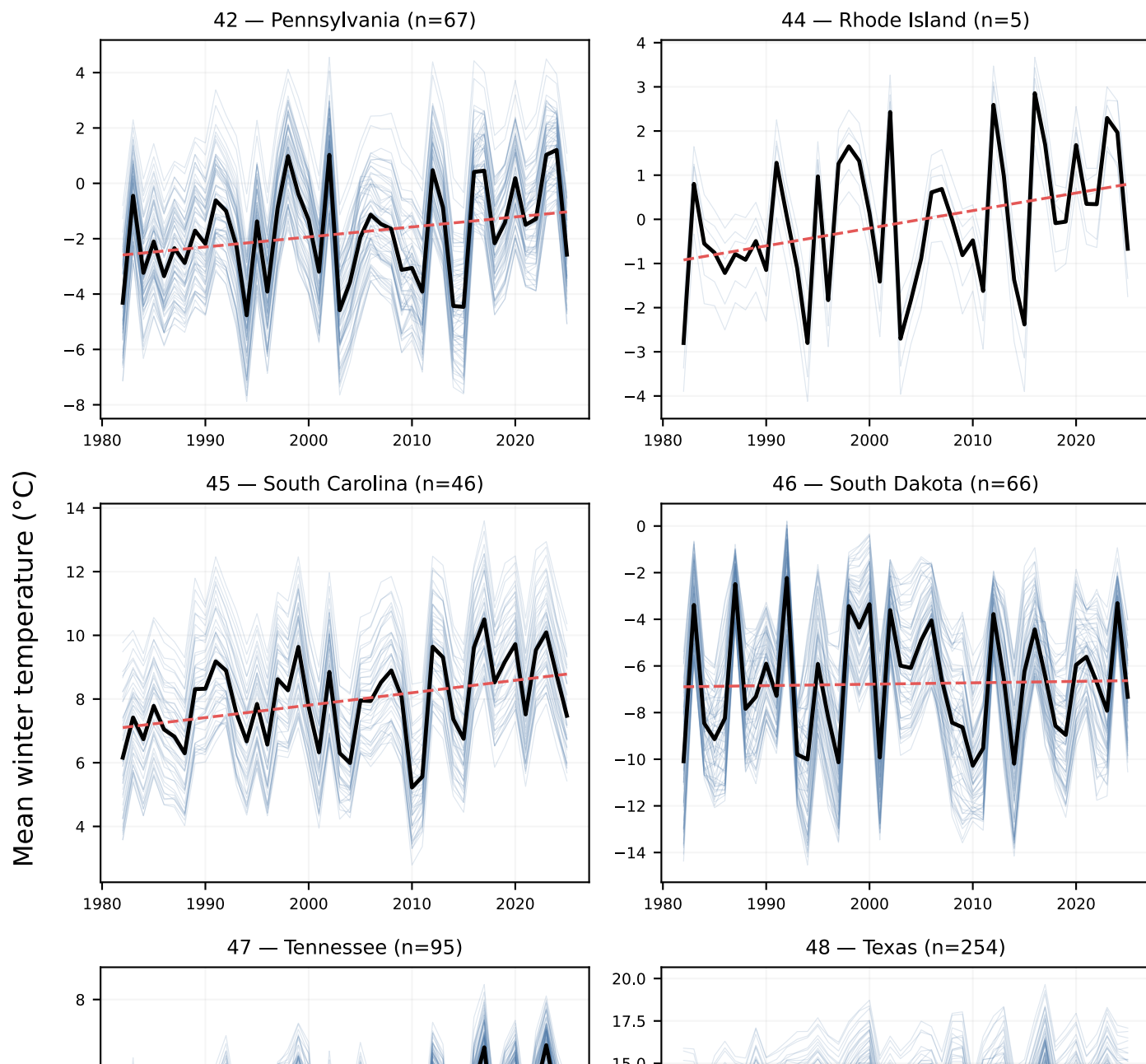


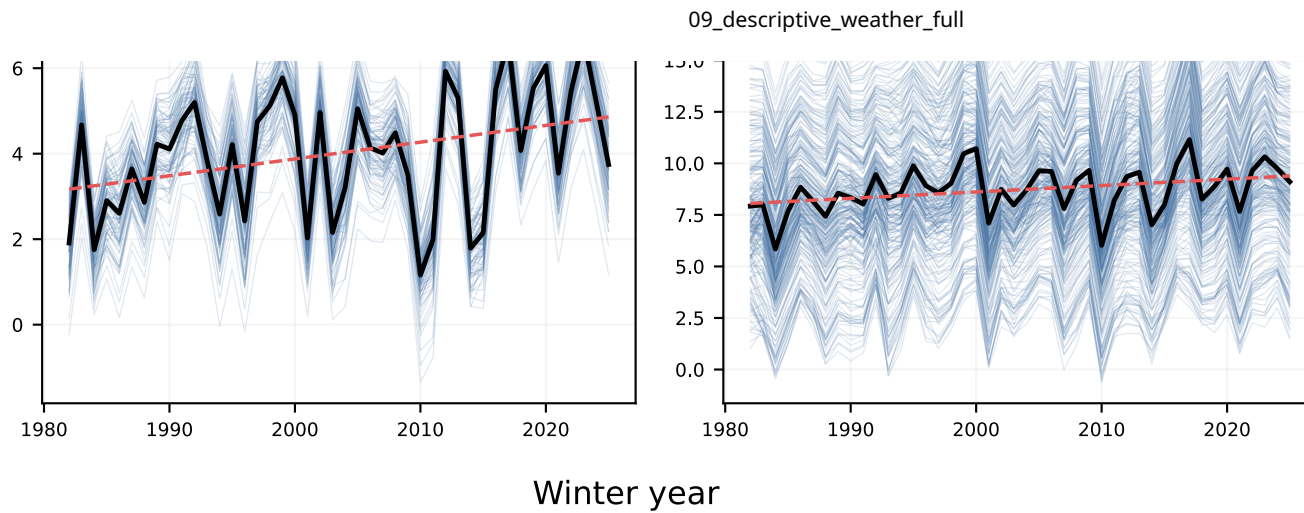
PRISM: county trajectories — Mean winter temperature (°C)



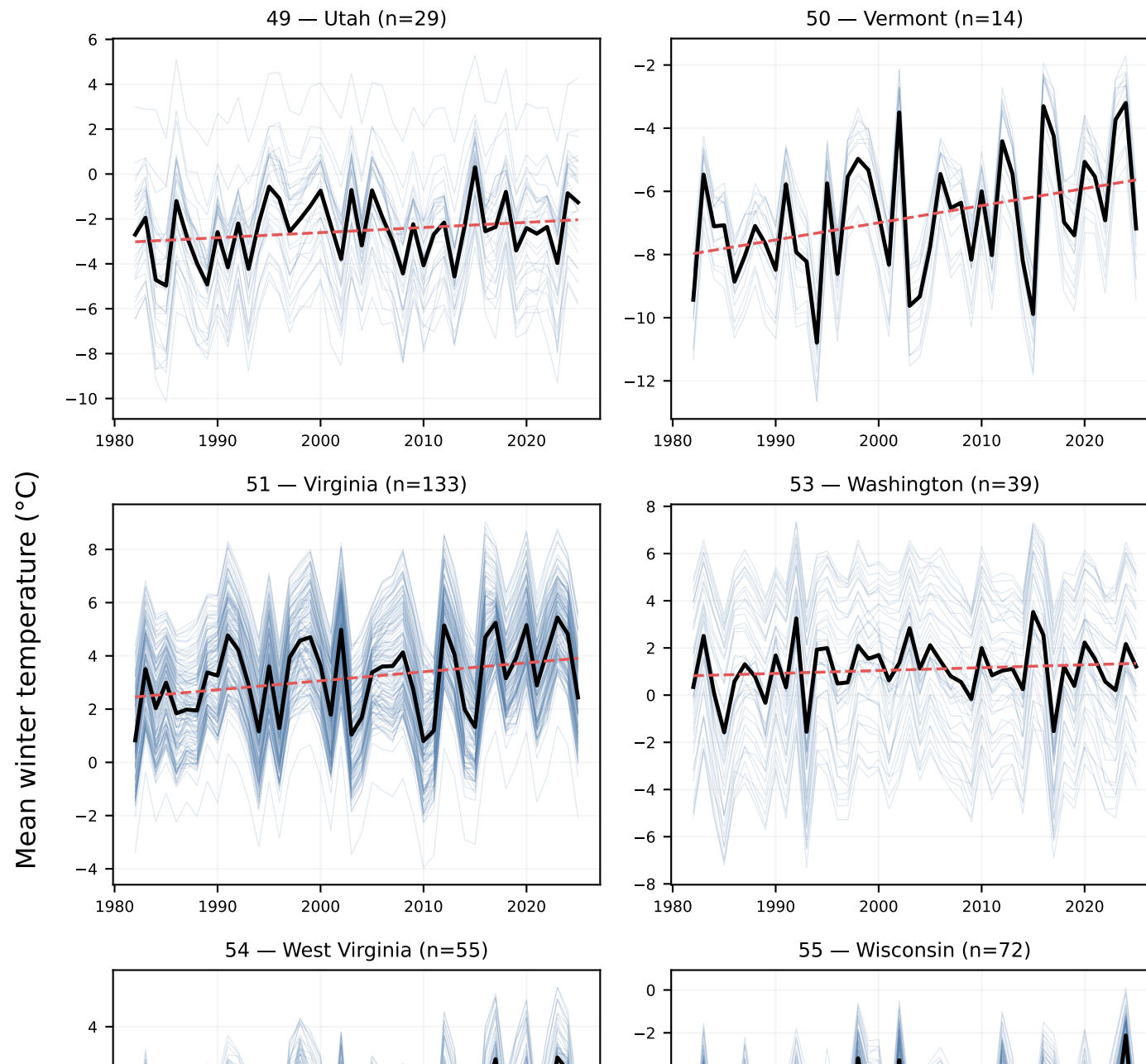


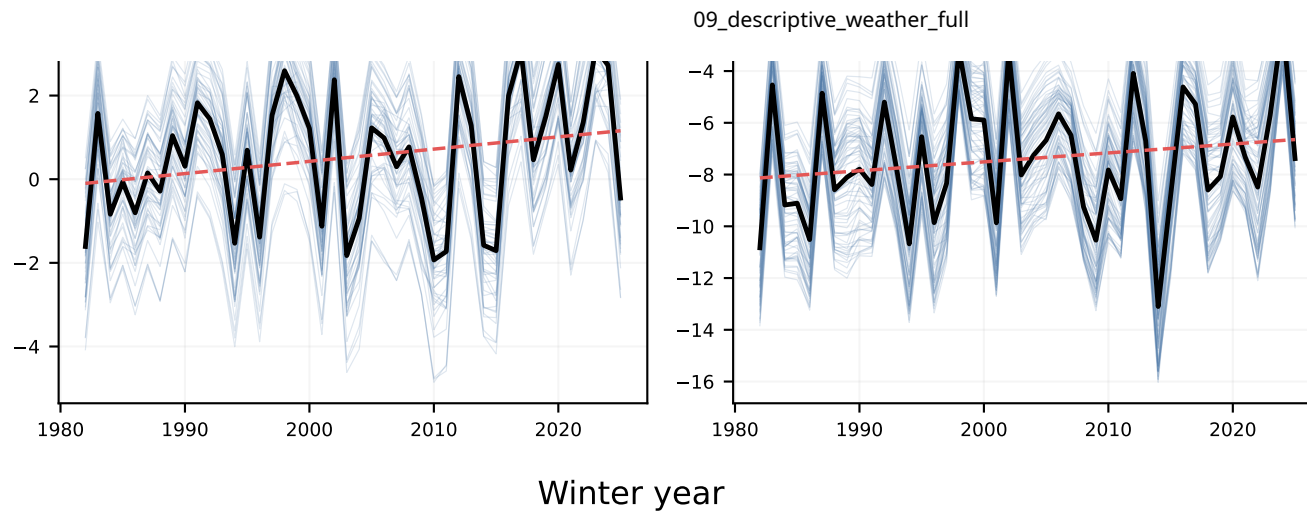
PRISM: county trajectories — Mean winter temperature (°C)



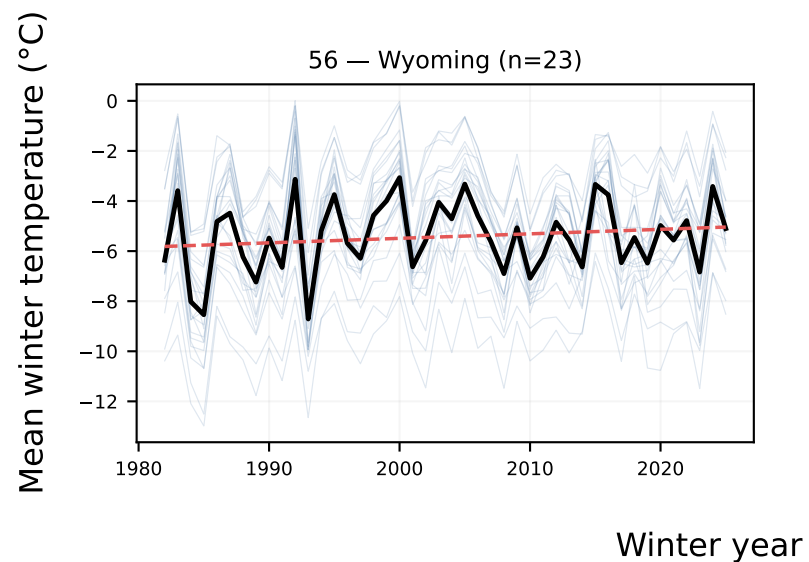


PRISM: county trajectories — Mean winter temperature (°C)





PRISM: county trajectories — Mean winter temperature (°C)



Saved /mnt/data_d/Dropbox/Research/AnimalCollisionsWeather/figures/weather/tier1/prism_county_spaghetti_mean_temp_c.pdf

10. Decade-distribution plots by state

Compare county-level distributions of mean winter temperature across decades within each state. Use these exploratory plots to determine whether the distribution shifts in location, spread, or shape—not merely whether its mean changes.

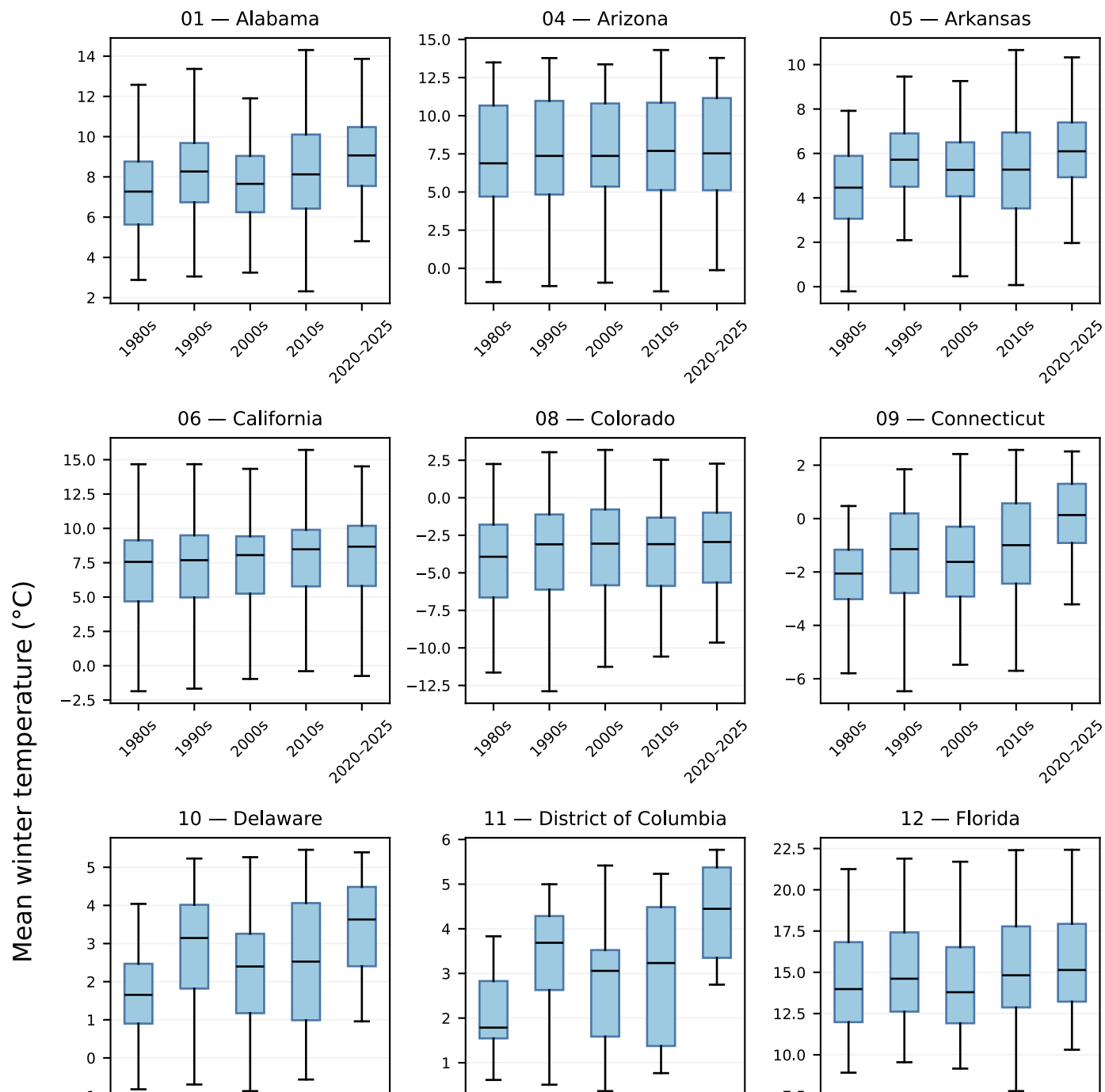
```
In [46]: def decade_label(year):
    if 2020 <= int(year) <= 2025:
        return "2020-2025"
    start = (int(year) // 10) * 10
    return f"{start}s"

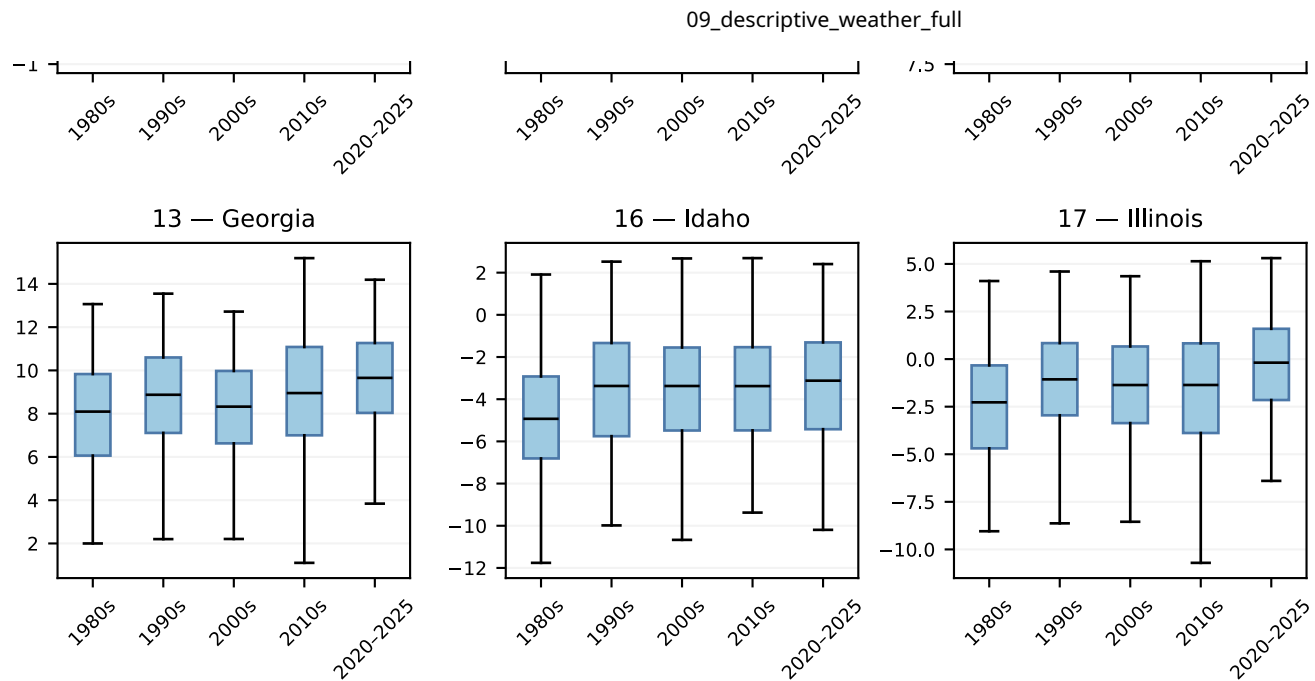
county_winter["decade"] = county_winter["winter_year"].map(decade_label)

def save_decade_distributions(frame, variable="mean_temp_c", states_per_page=12, n_cols=3):
    states = sorted(frame["state_fips"].dropna().unique())
    decades = sorted(frame["decade"].unique())
    path = FIGURES_DIR / f"{config.name.lower()}_state_decade_distributions_{variable}.pdf"
    with PdfPages(path) as pdf:
        for start in range(0, len(states), states_per_page):
            page_states = states[start:start + states_per_page]
            n_rows = int(np.ceil(len(page_states) / n_cols))
            fig, axes = plt.subplots(n_rows, n_cols, figsize=(7.5, 2.75 * n_rows), squeeze=False)
            for ax, state in zip(axes.flat, page_states):
                sub = frame[frame["state_fips"].eq(state)]
                values = [sub.loc[sub["decade"].eq(decade), variable].dropna() for decade in decades]
                ax.boxplot(values, tick_labels=decades, showfliers=False, patch_artist=True,
                           boxprops={"facecolor": "#9ECAE1", "edgecolor": "#4C78A8"},
                           medianprops={"color": "black"})
                ax.set_title(state_label(state), fontsize=9)
                ax.tick_params(axis="x", rotation=45, labels=7)
                ax.tick_params(axis="y", labels=7)
                ax.grid(axis="y", alpha=0.15)
            for ax in list(axes.flat)[len(page_states)::]:
                ax.axis("off")
            fig.suptitle(f"{config.name}: decade distributions of {variable_label(variable)}", fontsize=13)
            fig.supylabel(variable_label(variable))
            fig.tight_layout()
            pdf.savefig(fig, bbox_inches="tight")
            display_figure_inline(fig)
            plt.close(fig)
    print(f"Saved {path}")
    return path
```

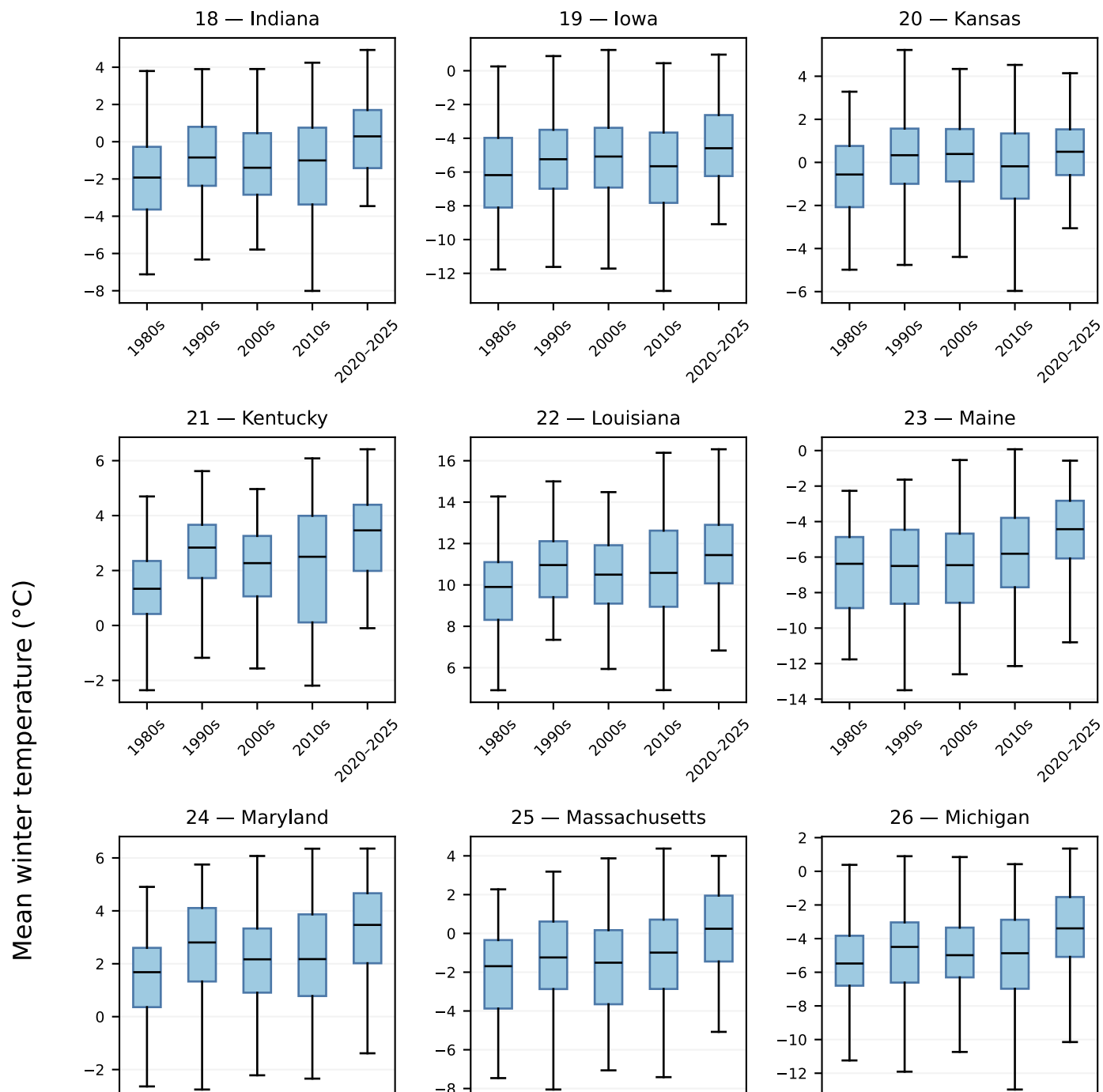
```
decade_distribution_path = save_decade_distributions(county_winter)
```

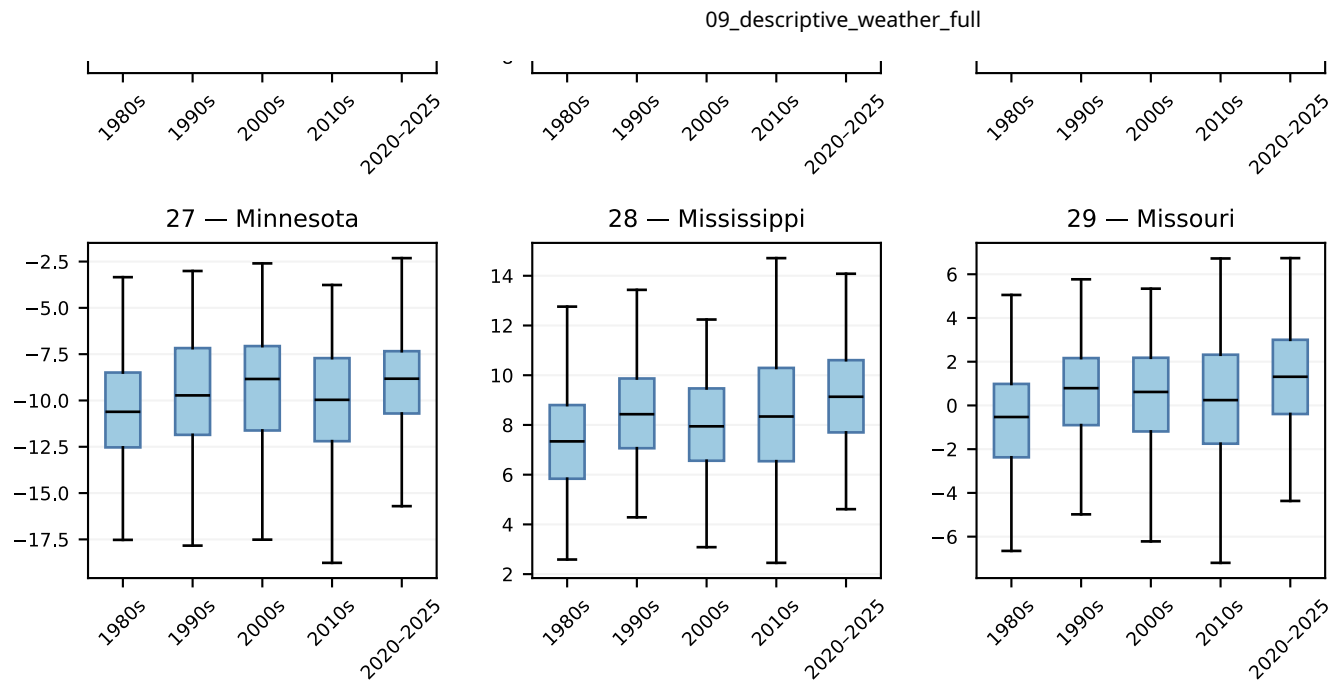
PRISM: decade distributions of Mean winter temperature (°C)



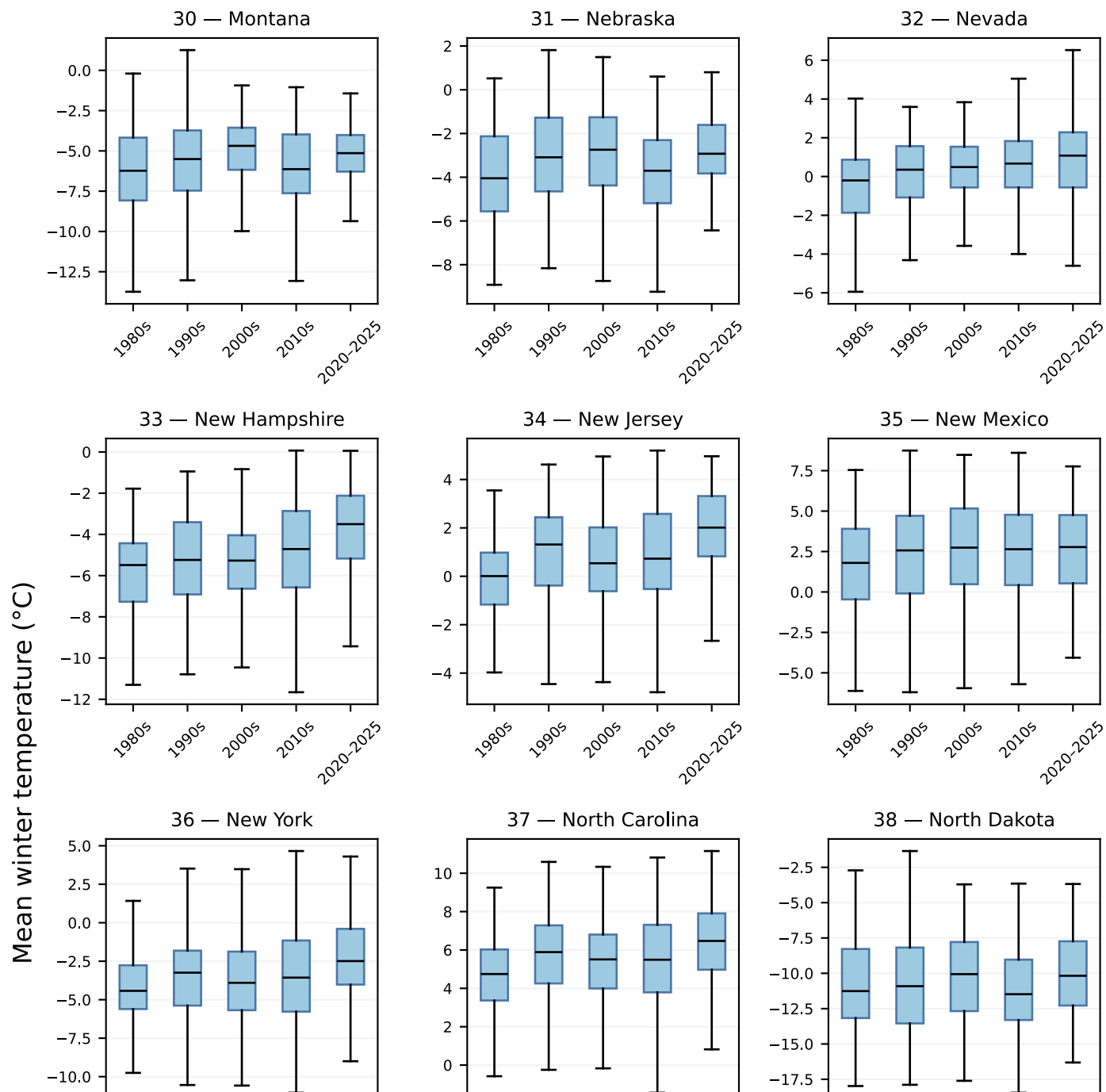


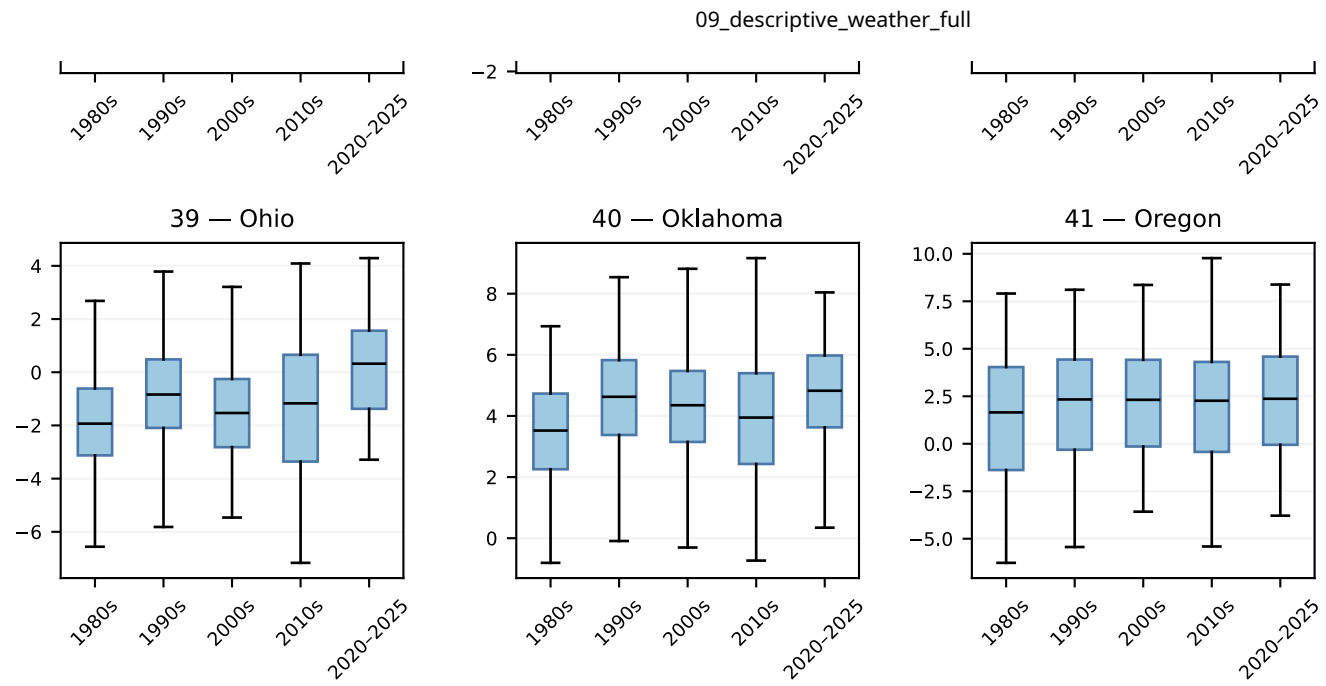
PRISM: decade distributions of Mean winter temperature (°C)



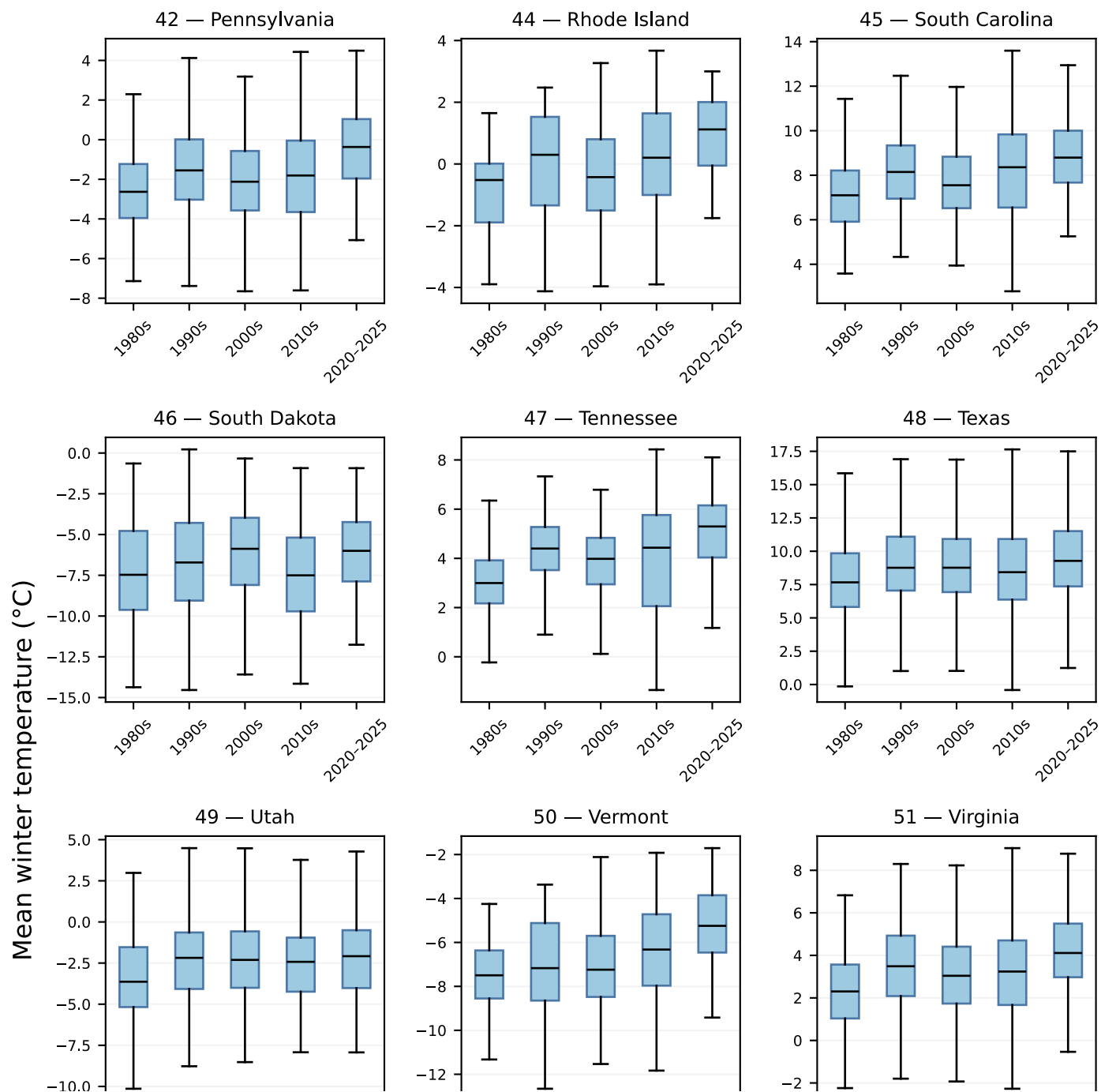


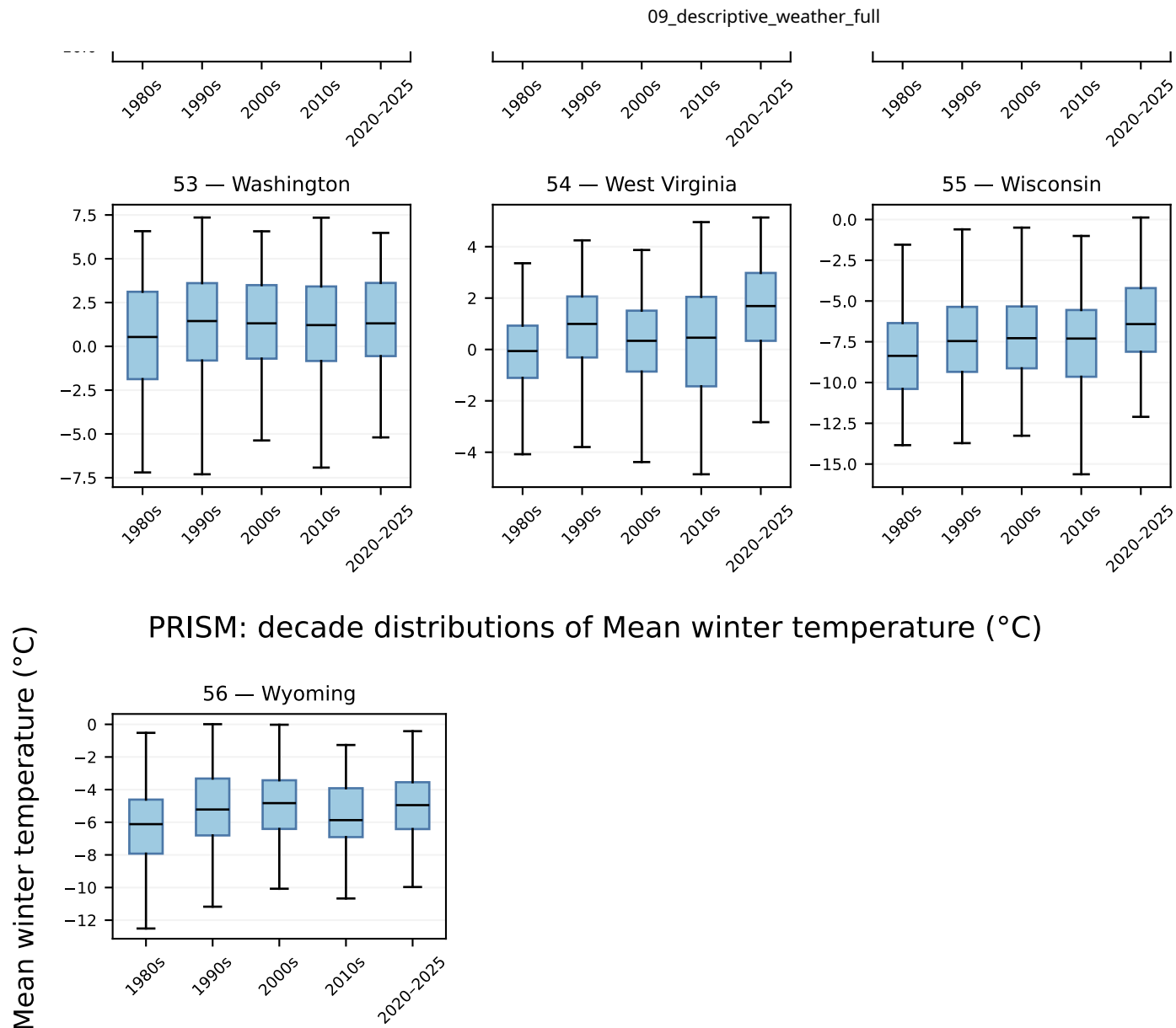
PRISM: decade distributions of Mean winter temperature (°C)





PRISM: decade distributions of Mean winter temperature (°C)





Saved /mnt/data_d/Dropbox/Research/AnimalCollisionsWeather/figures/weather/tier1/prism_state_decade_distributions_mean_temp_c.pdf

11. Interannual-variability summary

Calculate the standard deviation of county-level mean winter temperature across winters and summarize its distribution nationally and by state. This measure captures general year-to-year variability; it does not specifically identify anomalously warm winters.

```
In [47]: if county_winter.empty:
    warnings.warn(
        "The county-winter panel is empty; interannual-variability outputs will have no rows. "
        "Review the completeness diagnostics from Section 2 before interpreting this exhibit."
    )

    county_variability = (
        county_winter.groupby(["geoid", "state_fips", "county_name"], as_index=False)
        .agg(
            winter_temp_sd=("mean_temp_c", "std"),
            winter_temp_mean=("mean_temp_c", "mean"),
            n_winters=("mean_temp_c", "count"),
        )
    )
    county_variability["winter_temp_cv"] = (
        county_variability["winter_temp_sd"] / county_variability["winter_temp_mean"].abs()
    )
    county_variability.to_csv(
        TABLES_DIR / f"{config.name.lower()}_county_interannual_variability.csv", index=False
    )

    variability_national = summarize_long(
        county_variability.rename(columns={"winter_temp_sd": "interannual_sd_c"})
    )
    variability_by_state = (
        county_variability.groupby("state_fips")["winter_temp_sd"]
        .agg(n_counties="count", mean="mean", sd="std", min="min", median="median", max="max")
        .reset_index()
    )
    variability_by_state["state"] = variability_by_state["state_fips"].map(state_label)
    variability_by_state = variability_by_state[
        ["state_fips", "state", "n_counties", "mean", "sd", "min", "median", "max"]
    ]
    variability_by_state.to_csv(
        TABLES_DIR / f"{config.name.lower()}_interannual_variability_by_state.csv", index=False
    )
    save_table_pdf(
```

```
variability_by_state,  
TABLES_DIR / f"{config.name.lower()}_interannual_variability_by_state.pdf",  
f"{config.name}: interannual SD of mean winter temperature by state",  
rows_per_page=28,  
)  
display(variability_by_state)
```

PRISM: interannual SD of mean winter temperature by state

state_fips	state	n_counties	mean	sd	min	median	max
01	01 — Alabama	67	1.384	0.041	1.307	1.372	1.472
04	04 — Arizona	15	0.941	0.092	0.802	0.963	1.061
05	05 — Arkansas	75	1.407	0.088	1.264	1.406	1.566
06	06 — California	58	0.947	0.130	0.724	0.923	1.349
08	08 — Colorado	64	1.278	0.244	0.971	1.200	2.309
09	09 — Connecticut	8	1.675	0.054	1.601	1.689	1.740
10	10 — Delaware	3	1.478	0.047	1.428	1.488	1.520
11	11 — District of Columbia	1	1.525		1.525	1.525	1.525
12	12 — Florida	67	1.200	0.109	0.955	1.226	1.366
13	13 — Georgia	159	1.340	0.053	1.238	1.323	1.458
16	16 — Idaho	44	1.510	0.225	1.173	1.479	1.983
17	17 — Illinois	102	1.956	0.185	1.636	1.970	2.314
18	18 — Indiana	92	1.895	0.131	1.654	1.937	2.079
19	19 — Iowa	99	2.267	0.125	2.036	2.269	2.459
20	20 — Kansas	105	1.627	0.150	1.290	1.632	1.928
21	21 — Kentucky	120	1.564	0.052	1.427	1.564	1.709
22	22 — Louisiana	64	1.347	0.035	1.264	1.350	1.415
23	23 — Maine	16	1.643	0.095	1.458	1.626	1.796
24	24 — Maryland	24	1.493	0.053	1.384	1.480	1.632
25	25 — Massachusetts	14	1.590	0.150	1.353	1.579	1.793
26	26 — Michigan	83	1.968	0.118	1.762	1.958	2.320
27	27 — Minnesota	87	2.679	0.154	2.391	2.668	2.960
28	28 — Mississippi	82	1.404	0.039	1.281	1.400	1.487
29	29 — Missouri	115	1.768	0.159	1.513	1.729	2.088
30	30 — Montana	56	2.031	0.485	1.247	2.107	2.750
31	31 — Nebraska	93	1.904	0.166	1.450	1.935	2.199
32	32 — Nevada	17	1.267	0.154	1.010	1.243	1.570

33	33 — New Hampshire	10	1.697	0.062	1.612	1.688	1.780
----	--------------------	----	-------	-------	-------	-------	-------

PRISM: interannual SD of mean winter temperature by state

state_fips	state	n_counties	mean	sd	min	median	max
34	34 — New Jersey	21	1.565	0.081	1.462	1.566	1.732
35	35 — New Mexico	33	1.031	0.076	0.873	1.036	1.269
36	36 — New York	62	1.773	0.112	1.470	1.797	1.968
37	37 — North Carolina	100	1.325	0.042	1.232	1.335	1.402
38	38 — North Dakota	53	2.795	0.090	2.568	2.816	2.927
39	39 — Ohio	88	1.828	0.125	1.545	1.831	2.042
40	40 — Oklahoma	77	1.337	0.086	1.166	1.326	1.536
41	41 — Oregon	36	1.119	0.192	0.914	1.014	1.493
42	42 — Pennsylvania	67	1.707	0.093	1.513	1.699	1.909
44	44 — Rhode Island	5	1.510	0.060	1.444	1.516	1.595
45	45 — South Carolina	46	1.319	0.028	1.255	1.314	1.383
46	46 — South Dakota	66	2.452	0.261	1.741	2.512	2.837
47	47 — Tennessee	95	1.468	0.071	1.329	1.477	1.608
48	48 — Texas	254	1.201	0.103	0.916	1.217	1.391
49	49 — Utah	29	1.395	0.197	1.046	1.385	2.028
50	50 — Vermont	14	1.881	0.063	1.776	1.881	1.988
51	51 — Virginia	133	1.405	0.060	1.256	1.393	1.554
53	53 — Washington	39	1.200	0.287	0.866	1.197	1.754
54	54 — West Virginia	55	1.524	0.074	1.390	1.516	1.689
55	55 — Wisconsin	72	2.339	0.141	2.038	2.325	2.611
56	56 — Wyoming	23	1.558	0.204	1.241	1.527	1.940

Saved /mnt/data_d/Dropbox/Research/AnimalCollisionsWeather/tables/weather/tier1/prism_interannual_variability_by_state.pdf

	state_fips	state	n_counties	mean	sd	min	median	max
0	01	01 — Alabama	67	1.384397	0.041075	1.306876	1.372446	1.472043
1	04	04 — Arizona	15	0.941373	0.091912	0.802422	0.963487	1.061431
2	05	05 — Arkansas	75	1.407298	0.087703	1.264297	1.405864	1.566263
3	06	06 — California	58	0.947348	0.130118	0.724386	0.922969	1.349097
4	08	08 — Colorado	64	1.278184	0.243988	0.970638	1.200044	2.308930
5	09	09 — Connecticut	8	1.675022	0.054193	1.600826	1.689453	1.739601
6	10	10 — Delaware	3	1.478492	0.046576	1.427995	1.487711	1.519769
7	11	11 — District of Columbia	1	1.525411	NaN	1.525411	1.525411	1.525411
8	12	12 — Florida	67	1.200406	0.109458	0.954927	1.225948	1.366046
9	13	13 — Georgia	159	1.340115	0.052839	1.237647	1.323232	1.458166
10	16	16 — Idaho	44	1.510318	0.224721	1.172687	1.478660	1.982668
11	17	17 — Illinois	102	1.956491	0.184514	1.635704	1.969966	2.314189
12	18	18 — Indiana	92	1.894619	0.131195	1.653739	1.936664	2.079294
13	19	19 — Iowa	99	2.266870	0.124935	2.035643	2.269162	2.459033
14	20	20 — Kansas	105	1.626676	0.149975	1.289866	1.631958	1.928235
15	21	21 — Kentucky	120	1.564227	0.052066	1.427284	1.564449	1.708555
16	22	22 — Louisiana	64	1.346585	0.035330	1.263881	1.350296	1.415404
17	23	23 — Maine	16	1.642977	0.095083	1.457618	1.625601	1.795552
18	24	24 — Maryland	24	1.493395	0.053446	1.383529	1.480201	1.631539
19	25	25 — Massachusetts	14	1.590143	0.150409	1.353289	1.579447	1.793100
20	26	26 — Michigan	83	1.967846	0.117890	1.762348	1.957917	2.319773
21	27	27 — Minnesota	87	2.679368	0.154000	2.391368	2.667892	2.959973
22	28	28 — Mississippi	82	1.403782	0.038691	1.281236	1.400422	1.487153
23	29	29 — Missouri	115	1.768324	0.158925	1.512990	1.729077	2.088228
24	30	30 — Montana	56	2.030887	0.484956	1.246624	2.106854	2.750474

	state_fips	state	n_counties	mean	sd	min	median	max
25	31	31 — Nebraska	93	1.904438	0.165617	1.450011	1.935484	2.199173
26	32	32 — Nevada	17	1.266886	0.153813	1.009735	1.242683	1.570122
27	33	33 — New Hampshire	10	1.697144	0.061914	1.611801	1.688420	1.779899
28	34	34 — New Jersey	21	1.565310	0.081366	1.461825	1.566275	1.731917
29	35	35 — New Mexico	33	1.031427	0.075961	0.872658	1.035500	1.269238
30	36	36 — New York	62	1.772969	0.111932	1.469529	1.796518	1.968418
31	37	37 — North Carolina	100	1.324638	0.041911	1.231676	1.335090	1.402050
32	38	38 — North Dakota	53	2.795287	0.089589	2.568486	2.816092	2.927325
33	39	39 — Ohio	88	1.828358	0.124803	1.545212	1.831019	2.041672
34	40	40 — Oklahoma	77	1.337303	0.086197	1.166104	1.326323	1.535918
35	41	41 — Oregon	36	1.118909	0.191740	0.913877	1.014452	1.492514
36	42	42 — Pennsylvania	67	1.706502	0.093334	1.512810	1.699000	1.909123
37	44	44 — Rhode Island	5	1.509930	0.059672	1.444176	1.516239	1.594712
38	45	45 — South Carolina	46	1.319240	0.027514	1.255426	1.314149	1.382642
39	46	46 — South Dakota	66	2.451573	0.261037	1.741319	2.511923	2.837497
40	47	47 — Tennessee	95	1.468025	0.070790	1.328697	1.476538	1.607951
41	48	48 — Texas	254	1.201282	0.102561	0.915635	1.217088	1.390840
42	49	49 — Utah	29	1.394785	0.197258	1.046297	1.385321	2.028480
43	50	50 — Vermont	14	1.881338	0.063249	1.775682	1.880621	1.987892
44	51	51 — Virginia	133	1.405362	0.059595	1.255545	1.392967	1.553571
45	53	53 — Washington	39	1.200212	0.286766	0.866183	1.197259	1.754161
46	54	54 — West Virginia	55	1.523941	0.073922	1.389704	1.516221	1.688877
47	55	55 — Wisconsin	72	2.338514	0.140791	2.038360	2.324784	2.611060
48	56	56 — Wyoming	23	1.558193	0.203843	1.240718	1.527102	1.939915

12. Interannual-variability choropleth

Map the standard deviation of mean winter temperature across winters by county. The purpose is to identify regions with high general interannual variability and distinguish that concept from long-run warming and anomalous-warm-winter frequency.

```
In [48]: COUNTY_GEOMETRY_CANDIDATES = [
    REPO_ROOT / "dataRAW" / "shapefiles" / "cb_2023_us_county_20m.shp",
    REPO_ROOT / "dataRAW" / "cb_2023_us_county_20m.shp",
]
CENSUS_COUNTY_URL = "https://www2.census.gov/geo/tiger/GENZ2023/shp/cb_2023_us_county_20m.zip"

def load_county_geometry():
    try:
        import geopandas as gpd
    except ImportError as exc:
        raise ImportError(
            "Tier 1 choropleths require GeoPandas. Install the project dependencies "
            "in this notebook kernel with: %pip install -r ../requirements.txt "
            "Then restart the kernel and rerun the notebook."
        ) from exc
    local = next((path for path in COUNTY_GEOMETRY_CANDIDATES if path.exists()), None)
    source = local if local is not None else CENSUS_COUNTY_URL
    geometry = gpd.read_file(source)
    geoid_col = next((c for c in ("GE0ID", "GE0ID20", "geoid") if c in geometry), None)
    if geoid_col is None:
        raise KeyError("County geometry does not contain a recognized GE0ID column.")
    geometry["geoid"] = geometry[geoid_col].astype(str).str.zfill(5)
    return geometry[["geoid", "geometry"]]

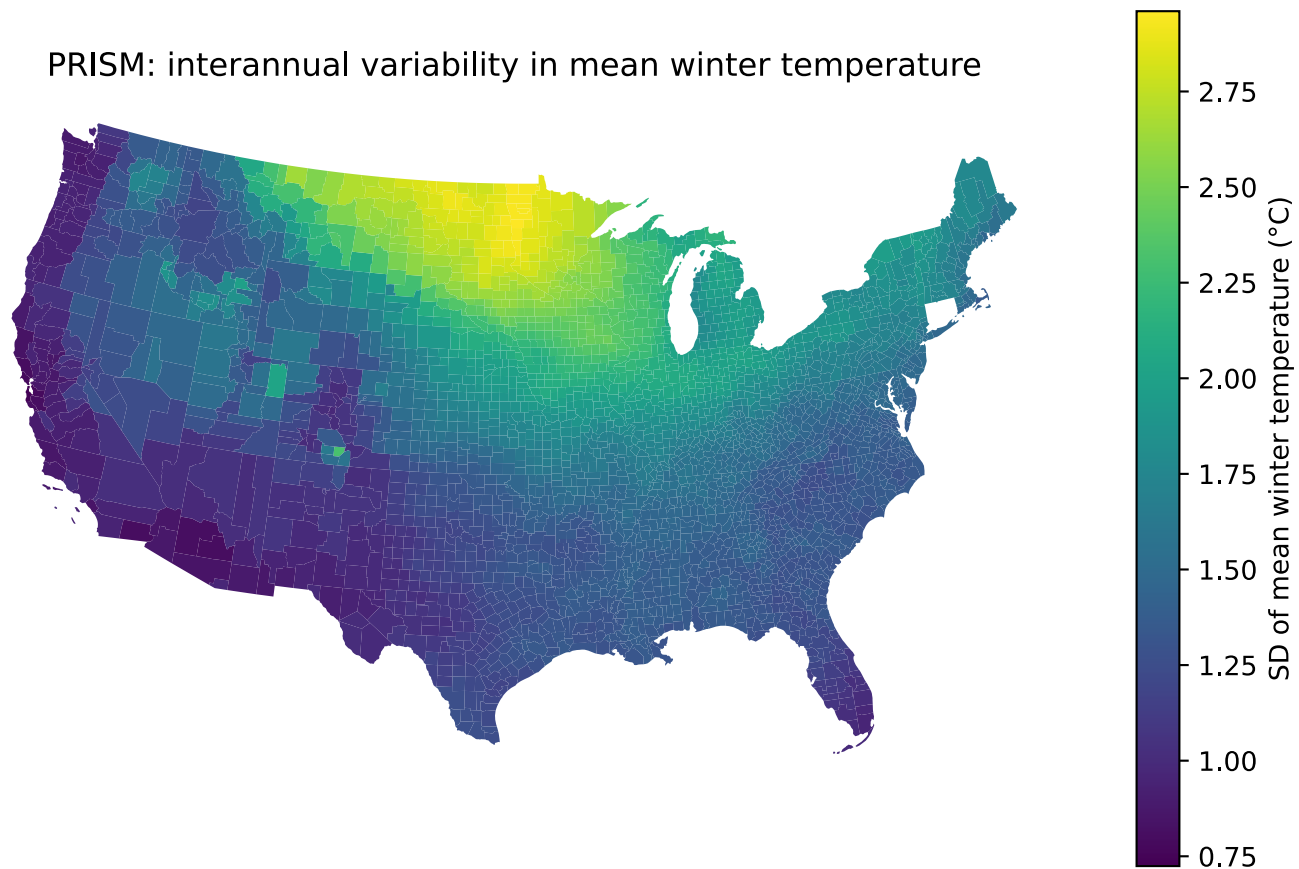
county_geometry = load_county_geometry()
variability_map = county_geometry.merge(
    county_variability[["geoid", "winter_temp_sd"]], on="geoid", how="inner", validate="one_to_one"
)
variability_map = variability_map.to_crs("EPSG:5070")
fig, ax = plt.subplots(figsize=(7.5, 4.8))
variability_map.plot(
    column="winter_temp_sd", cmap="viridis", linewidth=0,
    legend=True, legend_kwds={"label": "SD of mean winter temperature (°C)"},
    missing_kwds={"color": "lightgrey"}, ax=ax,
```

```

)
ax.set_title(f"{config.name}: interannual variability in mean winter temperature")
ax.axis("off")
fig.tight_layout()
interannual_variability_map_path = save_figure_pdf(
    fig, f"{config.name.lower()}_interannual_variability_choropleth.pdf"
)

```

PRISM: interannual variability in mean winter temperature



Saved /mnt/data_d/Dropbox/Research/AnimalCollisionsWeather/figures/weather/tier1/prism_interannual_variability_choropleth.pdf

13. Tier 1 QA findings and flagged observations

Summarize the main QA findings in plain language. List implausible values, unusual counties or winters, merge or coverage issues, and decisions requiring follow-up. Distinguish confirmed data problems from genuine but unusual weather observations.

```
In [49]: qa_findings = [
    {
        "check": "Duplicate county-year-month keys",
        "result": load_diagnostics["duplicate_monthly_keys"] + load_diagnostics["duplicate_derived_keys"],
        "status": "PASS" if not (load_diagnostics["duplicate_monthly_keys"] + load_diagnostics["duplicate_derived_keys"]) else "REVIEW",
    },
    {
        "check": "Rows appearing in only one merge source",
        "result": load_diagnostics["monthly_only_rows"] + load_diagnostics["derived_only_rows"],
        "status": "PASS" if not (load_diagnostics["monthly_only_rows"] + load_diagnostics["derived_only_rows"]) else "REVIEW",
    },
    {
        "check": "Incomplete county-months",
        "result": load_diagnostics["incomplete_months"],
        "status": "PASS" if not load_diagnostics["incomplete_months"] else "REVIEW",
    },
    {
        "check": "Excluded incomplete county-winters",
        "result": winter_diagnostics["excluded_incomplete_county_winters"],
        "status": "INFO",
    },
    {
        "check": f"County-variable mean flags ( $|z| \geq \{OUTLIER\_Z\_THRESHOLD:g\}$ )",
        "result": len(flagged_counties),
        "status": "REVIEW" if len(flagged_counties) else "PASS",
    },
]
qa_findings = pd.DataFrame(qa_findings)
qa_findings.to_csv(TABLES_DIR / f"{config.name.lower()}_tier1_qa_findings.csv", index=False)
save_table_pdf(
    qa_findings,
    TABLES_DIR / f"{config.name.lower()}_tier1_qa_findings.pdf",
    f"{config.name}: Tier 1 QA findings",
)
display(qa_findings)

print("Tier 1 outputs complete.")
```

```
print(f"Tables: {TABLES_DIR}")
print(f"Figures: {FIGURES_DIR}")
```

PRISM: Tier 1 QA findings

check	result	status
Duplicate county-year-month keys	0	PASS
Rows appearing in only one merge source	0	PASS
Incomplete county-months	0	PASS
Excluded incomplete county-winters	6,216	INFO
County-variable mean flags ($ z \geq 5$)	20	REVIEW

Saved /mnt/data_d/Dropbox/Research/AnimalCollisionsWeather/tables/weather/tier1/prism_tier1_qa_findings.pdf

	check	result	status
0	Duplicate county-year-month keys	0	PASS
1	Rows appearing in only one merge source	0	PASS
2	Incomplete county-months	0	PASS
3	Excluded incomplete county-winters	6216	INFO
4	County-variable mean flags ($ z \geq 5$)	20	REVIEW

Tier 1 outputs complete.

Tables: /mnt/data_d/Dropbox/Research/AnimalCollisionsWeather/tables/weather/tier1

Figures: /mnt/data_d/Dropbox/Research/AnimalCollisionsWeather/figures/weather/tier1

Tier 2 — Polished explanatory exhibits

Tier 2 communicates the two weather forces central to the project's causal story: long-run winter warming and short-run anomalously warm winter shocks. These exhibits should use publication-quality labels, units, annotations, legends, colorblind-accessible palettes, and vector output where possible.

14. National long-run warming exhibit

Present a polished national figure showing annual mean winter temperature and its long-run trend over the complete-winter sample. Clearly identify the geographic weighting convention and retain annual observations so anomalous winters remain visible.

```
In [50]: # Tier 2 outputs are kept separate from Tier 1 QA artifacts.
TIER2_TABLES_DIR = REPO_ROOT / "tables" / "weather" / "tier2"
TIER2_FIGURES_DIR = REPO_ROOT / "figures" / "weather" / "tier2"
TIER2_TABLES_DIR.mkdir(parents=True, exist_ok=True)
TIER2_FIGURES_DIR.mkdir(parents=True, exist_ok=True)
tier2_outputs = []

TIER2_COLORS = {"annual": "#0072B2", "trend": "#D55E00", "warm": "#B2182B"}

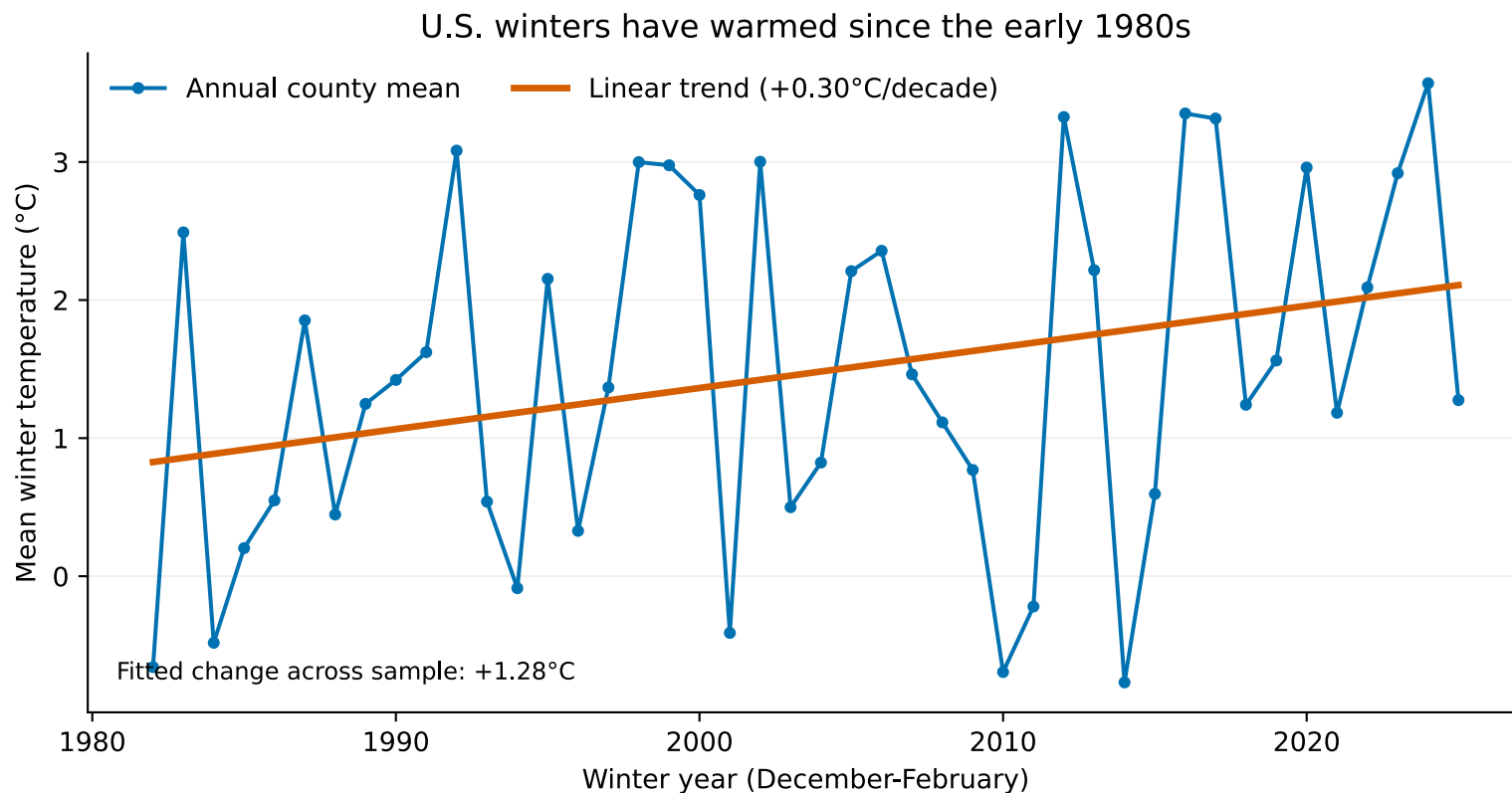
def save_tier2_figure(fig, filename):
    path = TIER2_FIGURES_DIR / filename
    fig.savefig(path, format="pdf", bbox_inches="tight", metadata={"Creator": "09_descriptive_weather_full"})
    display_figure_inline(fig)
    plt.close(fig)
    tier2_outputs.append(path)
    print(f"Saved {path}")
    return path

def save_tier2_placeholder(filename, title, message):
    """Create a vector diagnostic exhibit when the local sample is too short."""
    fig, ax = plt.subplots(figsize=(8.2, 4.8))
    ax.axis("off"); ax.set_title(title, loc="left", fontsize=14, pad=18)
    ax.text(0.02, 0.78, message, transform=ax.transAxes, ha="left", va="top", fontsize=11, wrap=True)
    ax.text(0.02, 0.08, "This placeholder is intentional; rerun unchanged on Kodama with the complete 1981")
    fig.tight_layout(); return save_tier2_figure(fig, filename)
```

```

national_winter = county_winter.groupby("winter_year", as_index=False).agg(
    mean_temp_c=("mean_temp_c", "mean"), n_counties=("geoid", "nunique")
)
FULL_LONG_RUN_SAMPLE = len(national_winter) >= 30
if not FULL_LONG_RUN_SAMPLE:
    national_warming_path = save_tier2_placeholder(f"{config.name.lower()}_national_long_run_warming.pdf",
else:
    trend = _linear_trend(national_winter["winter_year"], national_winter["mean_temp_c"])
    fig, ax = plt.subplots(figsize=(8.2, 4.8))
    ax.plot(national_winter["winter_year"], national_winter["mean_temp_c"], color=TIER2_COLORS["annual"],
    change = trend[2] * (national_winter["winter_year"].max() - national_winter["winter_year"].min())
    ax.plot(trend[0], trend[1], color=TIER2_COLORS["trend"], lw=2.5, label=f"Linear trend ({trend[2] * 10:-
    ax.text(0.02, 0.05, f"Fitted change across sample: {change:+.2f}°C", transform=ax.transAxes, fontsize=
    ax.set(title="U.S. winters have warmed since the early 1980s", xlabel="Winter year (December-February)"
    ax.text(0, -0.22, f"Source: {config.name}. Unweighted mean across counties; complete DJF winters only."
    ax.grid(axis="y", alpha=0.2); ax.spines[["top", "right"]].set_visible(False); ax.legend(frameon=False,
    fig.tight_layout(); national_warming_path = save_tier2_figure(fig, f"{config.name.lower()}_national_lo

```



Source: PRISM. Unweighted mean across counties; complete DJF winters only.

Saved /mnt/data_d/Dropbox/Research/AnimalCollisionsWeather/figures/weather/tier2/prism_national_long_run_warming.pdf

15. State long-run warming exhibit

Present polished state-level small multiples showing geographic heterogeneity in winter-temperature trends. Consider displaying trend slopes or early-to-recent changes if annual series make the panels too dense.

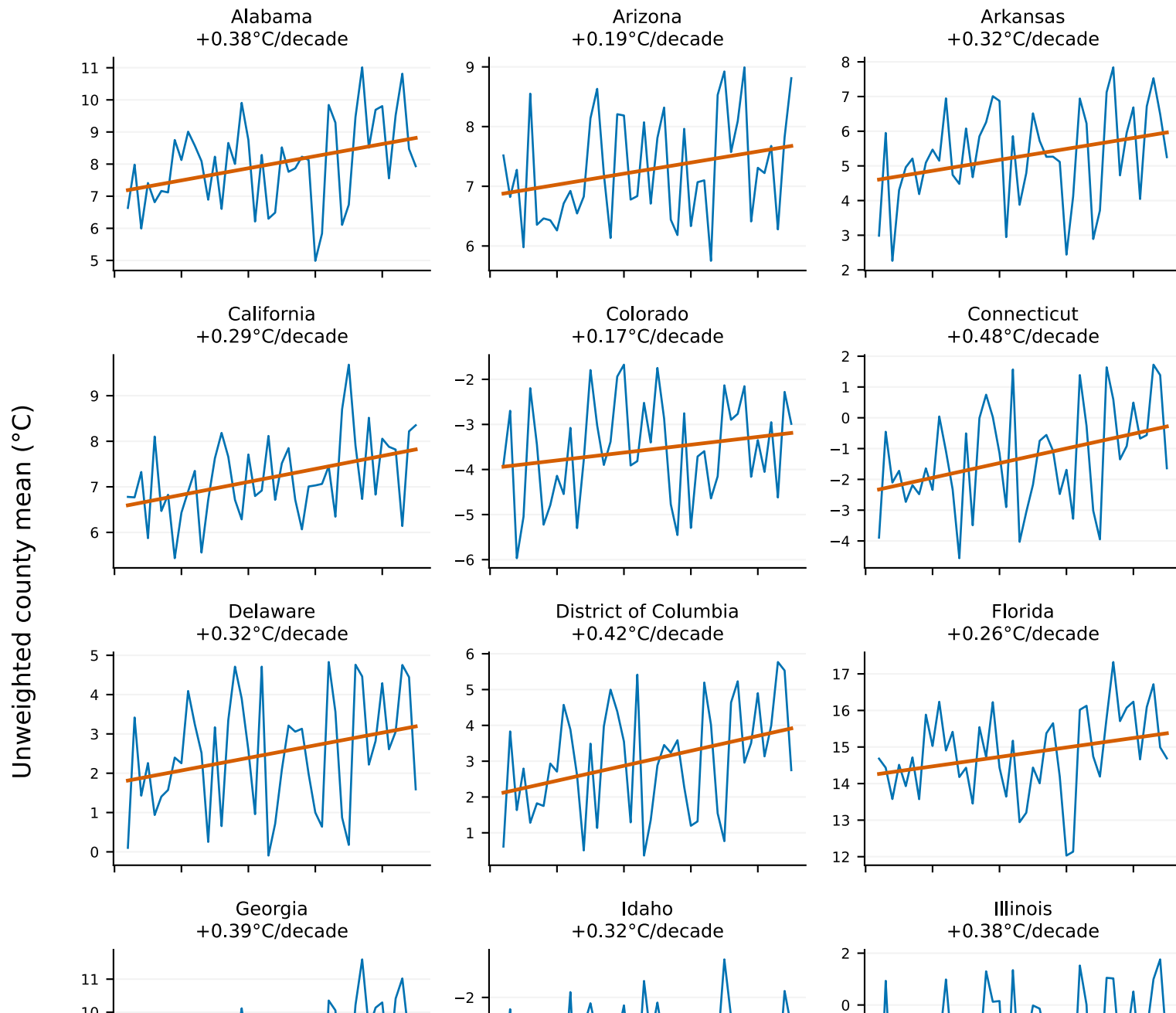
```
In [51]: state_annual = county_winter.groupby(["state_fips", "winter_year"], as_index=False)["mean_temp_c"].mean()
states = sorted(state_annual["state_fips"].dropna().unique())
state_warming_path = TIER2_FIGURES_DIR / f"{config.name.lower()}_state_long_run_warming.pdf"
if not FULL_LONG_RUN_SAMPLE:
    state_warming_path = save_tier2_placeholder(state_warming_path.name, "State long-run winter warming",
```

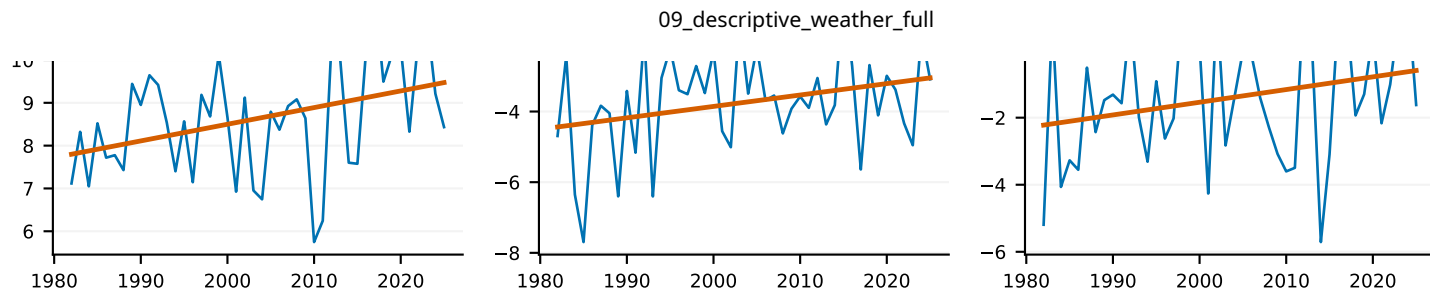
```

else:
    with PdfPages(state_warming_path, metadata={"Creator": "09_descriptive_weather_full.ipynb"}) as pdf:
        for start in range(0, len(states), 12):
            page_states = states[start:start + 12]
            fig, axes = plt.subplots(4, 3, figsize=(8.2, 9.5), sharex=True, squeeze=False)
            for ax, state in zip(axes.flat, page_states):
                annual = state_annual[state_annual["state_fips"].eq(state)]
                state_trend = _linear_trend(annual["winter_year"], annual["mean_temp_c"])
                ax.plot(annual["winter_year"], annual["mean_temp_c"], color=TIER2_COLORS["annual"], lw=1)
                slope_text = "trend unavailable"
                if state_trend is not None:
                    ax.plot(state_trend[0], state_trend[1], color=TIER2_COLORS["trend"], lw=1.8)
                    slope_text = f"{state_trend[2] * 10:+.2f}°C/decade"
                ax.set_title(f"{STATE_FIPS_TO_NAME.get(str(state).zfill(2), state)}\n{slope_text}", fontsize=9)
                ax.grid(axis="y", alpha=0.15); ax.tick_params(labelsize=7); ax.spines[["top", "right"]].set_visible(False)
            for ax in list(axes.flat)[len(page_states):]: ax.axis("off")
            fig.suptitle("Winter-temperature trends vary across states", fontsize=14)
            fig.supxlabel("Winter year"); fig.supylabel("Unweighted county mean (°C)")
            fig.text(0.01, 0.005, f"Source: {config.name}; complete DJF winters. Orange lines show state-specific trends")
            fig.tight_layout(rect=[0.02, 0.025, 1, 0.97]); pdf.savefig(fig, bbox_inches="tight"); display_figures()
        if state_warming_path not in tier2_outputs: tier2_outputs.append(state_warming_path)
    print(f"Saved {state_warming_path}")

```


Winter-temperature trends vary across states

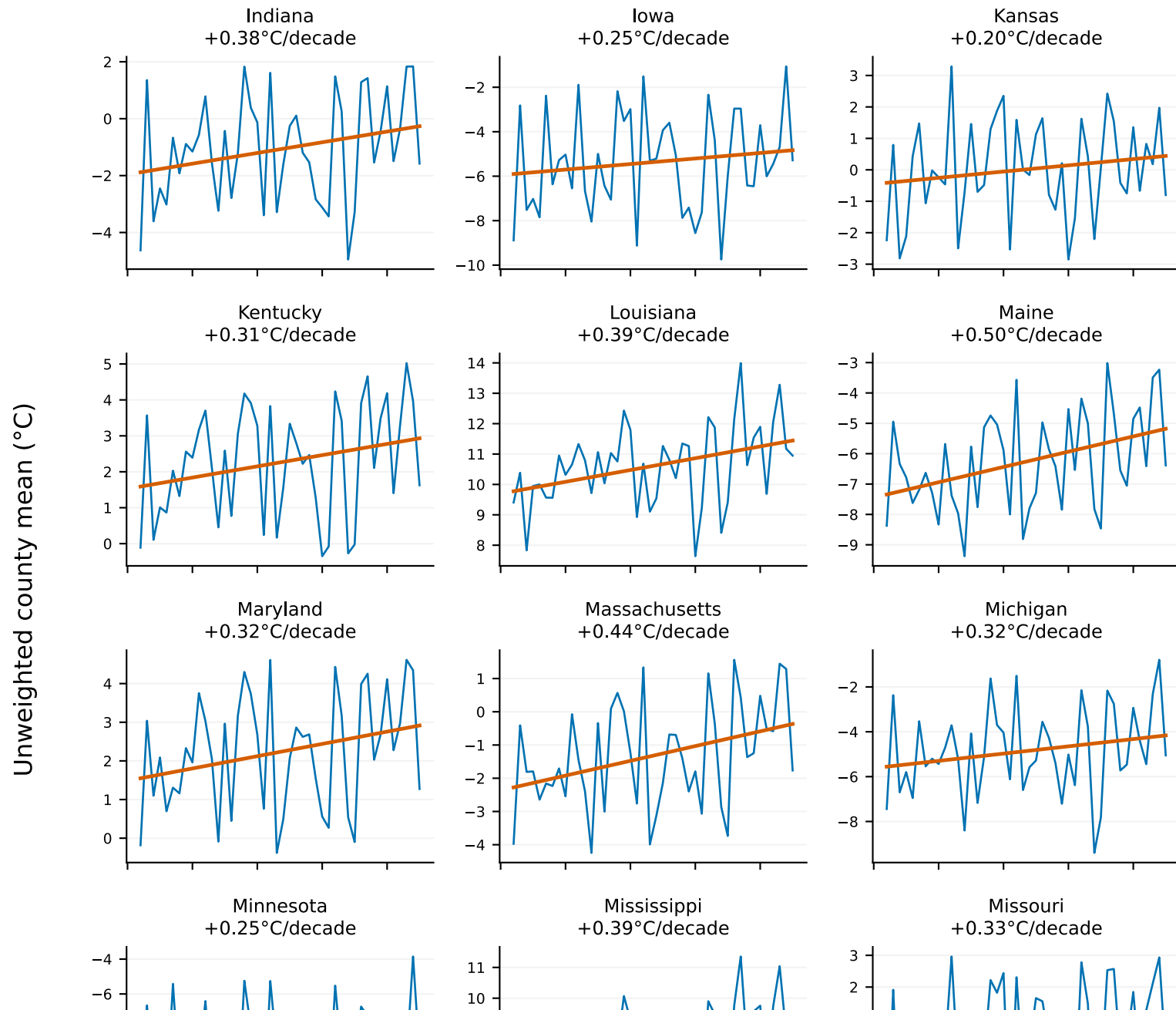


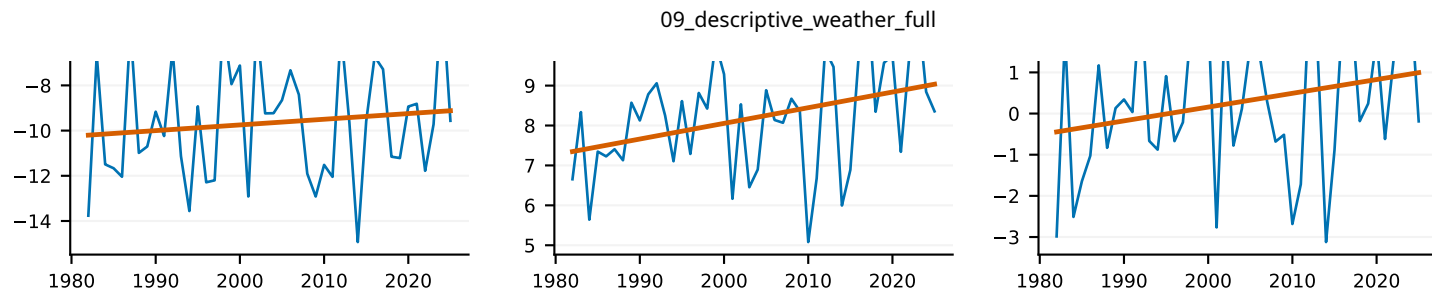


Winter year

Source: PRISM; complete DJF winters. Orange lines show state-specific linear trends.

Winter-temperature trends vary across states

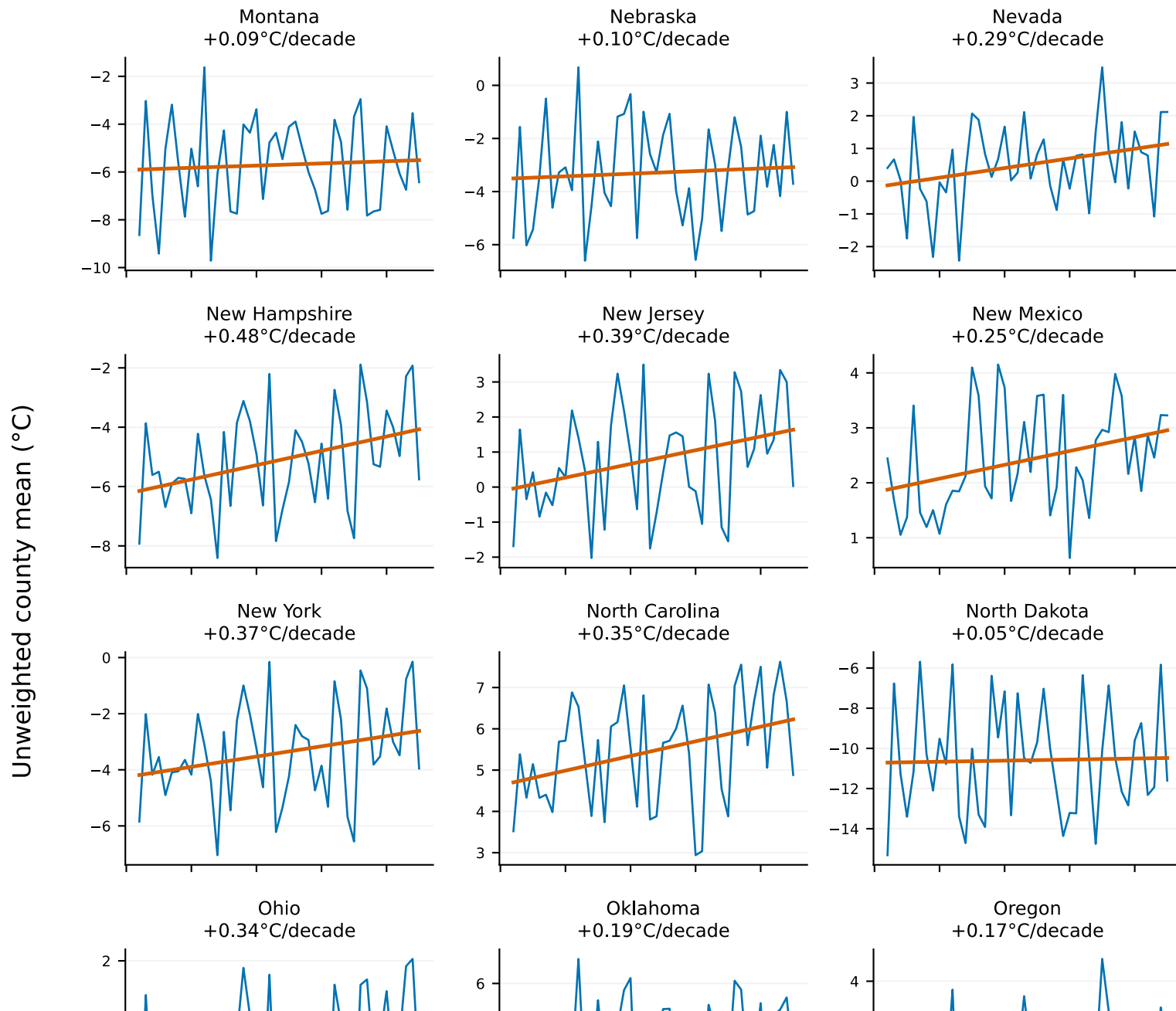


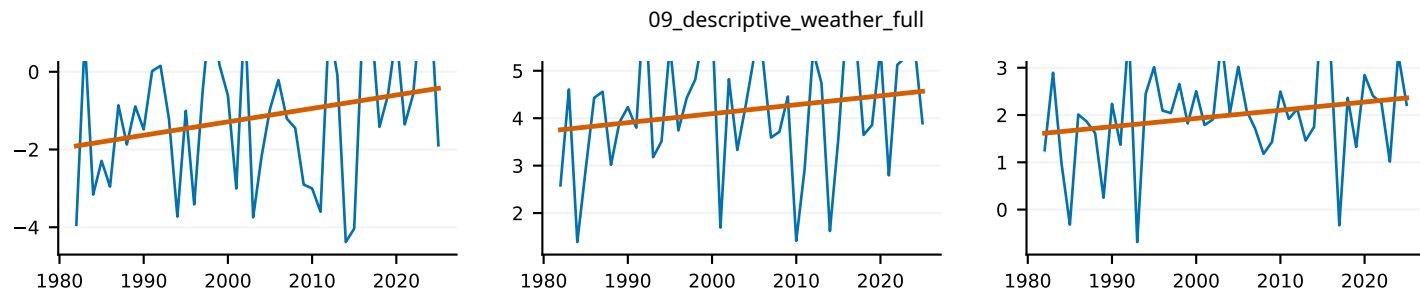


Winter year

Source: PRISM; complete DJF winters. Orange lines show state-specific linear trends.

Winter-temperature trends vary across states

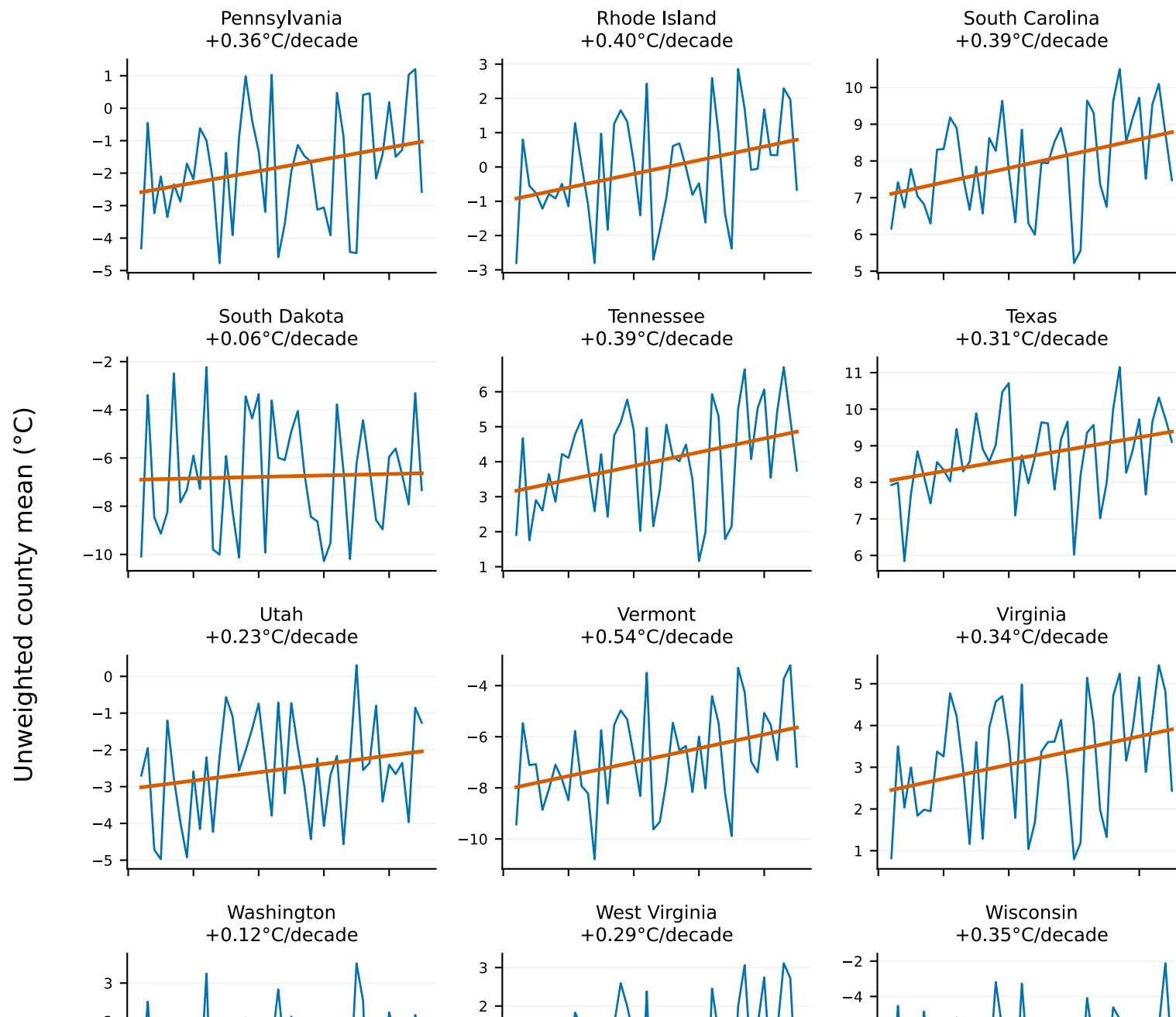


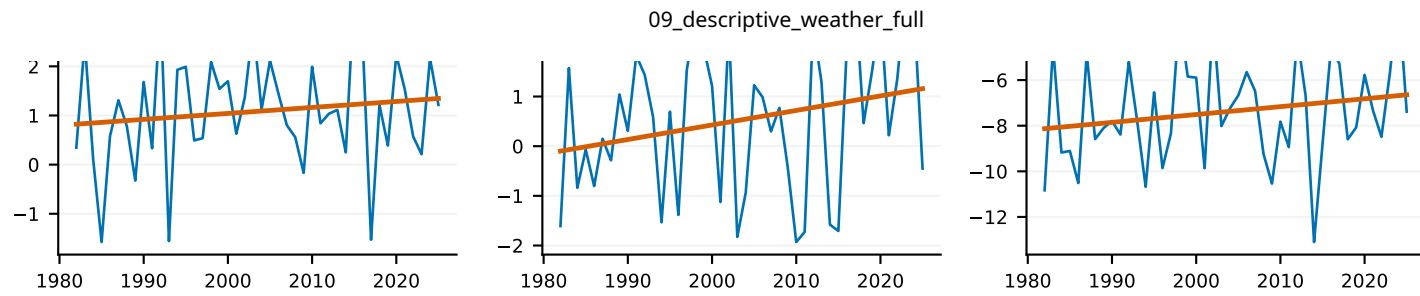


Winter year

Source: PRISM; complete DJF winters. Orange lines show state-specific linear trends.

Winter-temperature trends vary across states

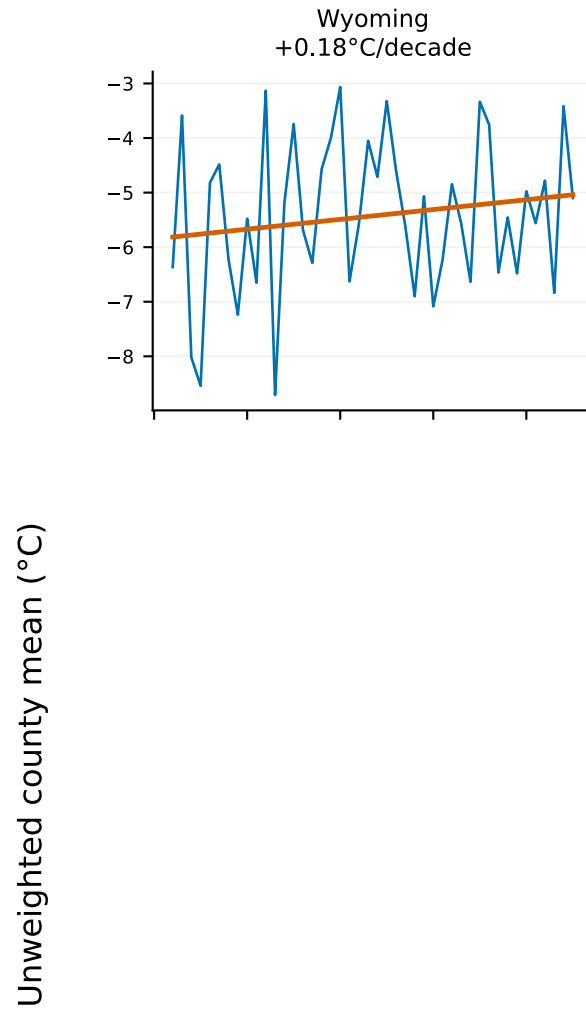




Winter year

Source: PRISM; complete DJF winters. Orange lines show state-specific linear trends.

Winter-temperature trends vary across states



Winter year

Source: PRISM; complete DJF winters. Orange lines show state-specific linear trends.

Saved /mnt/data_d/Dropbox/Research/AnimalCollisionsWeather/figures/weather/tier2/prism_state_long_run_warming.pdf

16. Early-versus-recent winter-temperature choropleths

Map county mean winter temperature for a clearly defined early period and recent period. Use identical classification rules and color limits so that the two maps are directly comparable. Document the selected periods.

```
In [52]: # Ten-winter windows balance temporal contrast with resistance to single-winter noise.
EARLY_PERIOD = (1982, 1991)
RECENT_PERIOD = (2016, 2025)

def period_mean(frame, variable, period, output_name):
    lo, hi = period
    return (frame[frame["winter_year"].between(lo, hi)].groupby("geoid", as_index=False)
            .agg(**{output_name: (variable, "mean")}, n_winters=(variable, "count")))

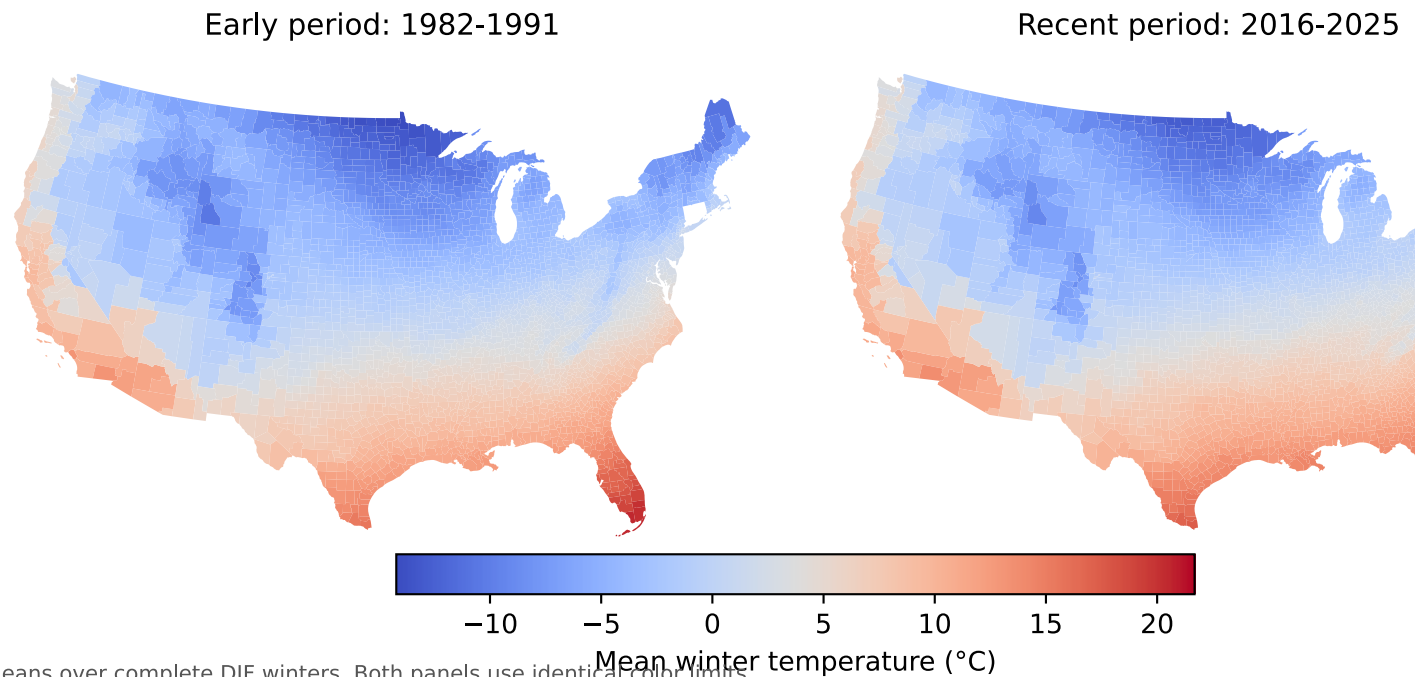
early_temp = period_mean(county_winter, "mean_temp_c", EARLY_PERIOD, "early_temp_c")
recent_temp = period_mean(county_winter, "mean_temp_c", RECENT_PERIOD, "recent_temp_c")
temperature_periods = early_temp.merge(recent_temp, on="geoid", suffixes=("_early", "_recent"), validate="one_to_one")
temperature_periods.to_csv(TIER2_TABLES_DIR / f"{config.name.lower()}_county_temperature_early_recent.csv")
temperature_map = county_geometry.merge(temperature_periods, on="geoid", how="inner", validate="one_to_one")
if temperature_map.empty:
    early_recent_map_path = save_tier2_placeholder(f"{config.name.lower()}_winter_temperature_early_vs_recent")
else:
    common_min = float(temperature_map[["early_temp_c", "recent_temp_c"]].min().min())
    common_max = float(temperature_map[["early_temp_c", "recent_temp_c"]].max().max())
    fig, axes = plt.subplots(1, 2, figsize=(11, 4.5))
```

```

for ax, column, period, title in zip(axes, ["early_temp_c", "recent_temp_c"], [EARLY_PERIOD, RECENT_PERIOD], [title]):
    temperature_map.plot(column=column, cmap="coolwarm", vmin=common_min, vmax=common_max, linewidth=0)
    ax.set_title(f"{title}: {period[0]}-{period[1]}", fontsize=11); ax.axis("off")
sm = plt.cm.ScalarMappable(cmap="coolwarm", norm=plt.Normalize(common_min, common_max)); sm.set_array([common_min, common_max])
fig.colorbar(sm, ax=axes, orientation="horizontal", fraction=0.06, pad=0.03, label="Mean winter temperature (°C)")
fig.suptitle("Mean winter temperature: early and recent decades", fontsize=14)
fig.text(0.01, 0.01, f"Source: {config.name}; county means over complete DJF winters. Both panels use identical color limits.",
        fontdict=dict(fontsize=10, color="red"))
fig.subplots_adjust(bottom=0.15, top=0.88, wspace=0.02)
early_recent_map_path = save_tier2_figure(fig, f"{config.name.lower()}_winter_temperature_early_vs_recent_decades.pdf")

```

Mean winter temperature: early and recent decades



Source: PRISM; county means over complete DJF winters. Both panels use identical color limits.

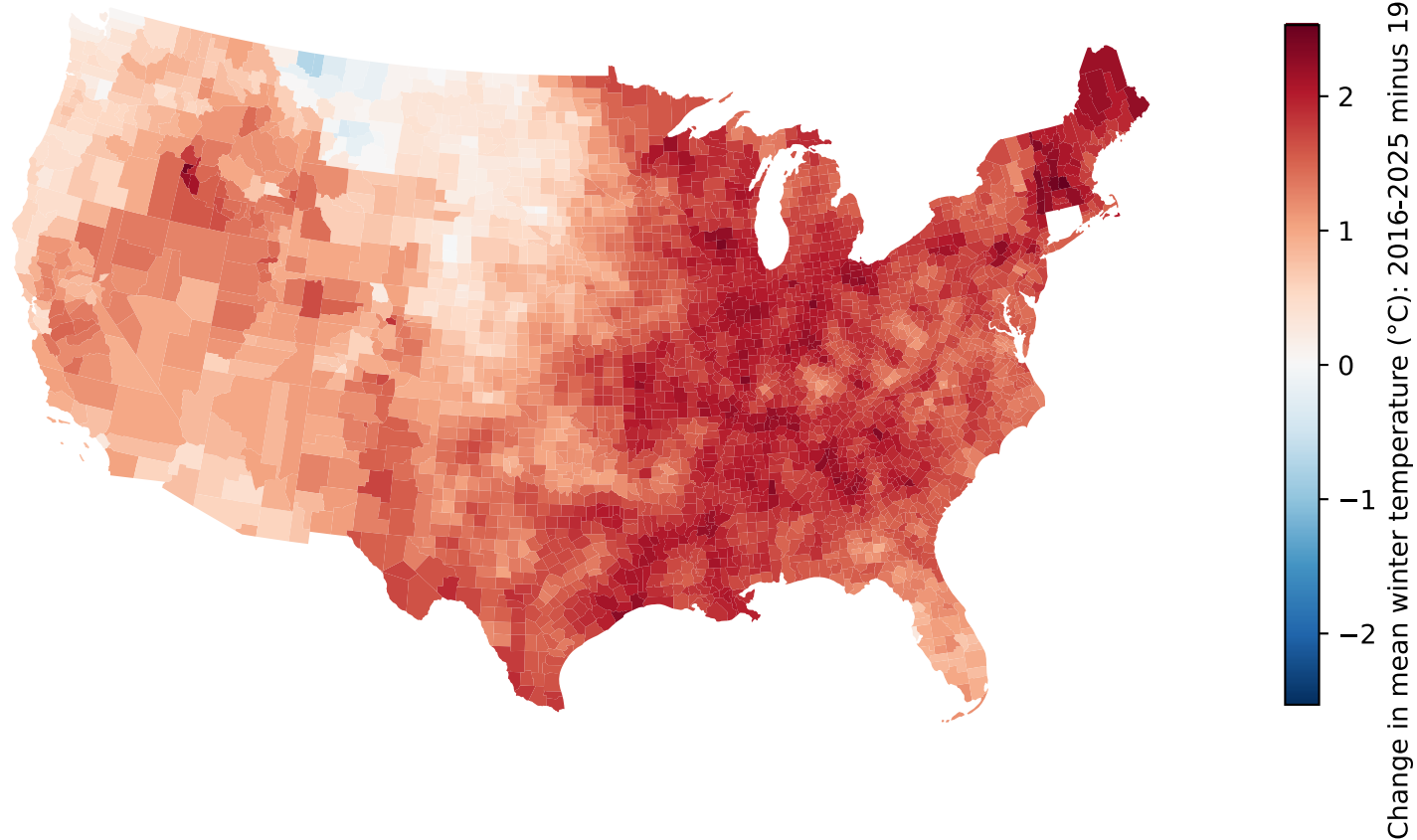
Saved /mnt/data_d/Dropbox/Research/AnimalCollisionsWeather/figures/weather/tier2/prism_winter_temperature_early_vs_recent.pdf

17. Change-in-winter-temperature choropleth

Map the county-level change in mean winter temperature between the early and recent periods. This exhibit should directly identify which places experienced the greatest long-run winter warming.

```
In [53]: temperature_periods["change_temp_c"] = temperature_periods["recent_temp_c"] - temperature_periods["early_temp_c"]
change_map = county_geometry.merge(temperature_periods[["geoid", "change_temp_c"]], on="geoid", how="inner")
if change_map.empty:
    warming_change_map_path = save_tier2_placeholder(f"{config.name.lower()}_winter_temperature_change.pdf")
else:
    change_limit = float(np.nanmax(np.abs(change_map["change_temp_c"])))
    fig, ax = plt.subplots(figsize=(8.5, 5.2))
    change_map.plot(column="change_temp_c", cmap="RdBu_r", vmin=-change_limit, vmax=change_limit, linewidth=1)
    ax.set_title("Where U.S. winters have warmed the most", fontsize=14); ax.axis("off")
    fig.text(0.01, 0.01, f"Source: {config.name}; county means over complete DJF winters. Diverging scale :")
    fig.tight_layout(rect=[0, 0.03, 1, 1])
    warming_change_map_path = save_tier2_figure(fig, f"{config.name.lower()}_winter_temperature_change.pdf")
```

Where U.S. winters have warmed the most



Source: PRISM; county means over complete DJF winters. Diverging scale is centered at zero.

Saved /mnt/data_d/Dropbox/Research/AnimalCollisionsWeather/figures/weather/tier2/prism_winter_temperature_change.pdf

18. Extreme-cold-day choropleth

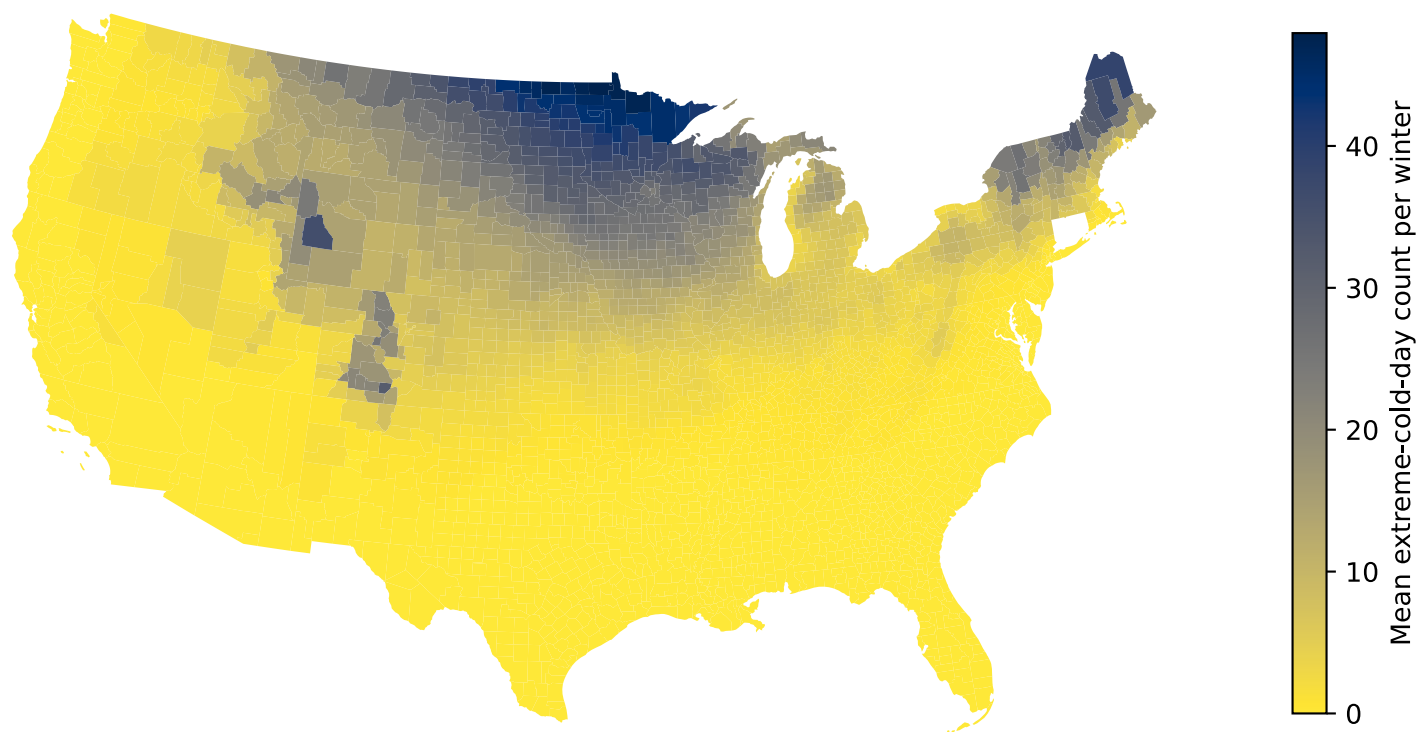
Map the mean annual number of winter days with minimum temperature below 0°F, or the change in that count between the early and recent periods. Label this measure **extreme-cold-day count**, not Winter Severity Index. A complete Wisconsin-style WSI would require additional components such as deep-snow days.

```

In [54]: if "days_extremely_cold" not in county_winter:
    extreme_cold = pd.DataFrame(columns=["geoid", "mean_extreme_cold_days", "n_winters"])
    extreme_cold_map_path = save_tier2_placeholder(f"{config.name.lower()}_extreme_cold_day_count.pdf", "E
else:
    extreme_cold = (county_winter.groupby("geoid", as_index=False).agg(mean_extreme_cold_days=("days_extrem
    extreme_cold_map = county_geometry.merge(extreme_cold, on="geoid", how="inner", validate="one_to_one")
    fig, ax = plt.subplots(figsize=(8.5, 5.2))
    extreme_cold_map.plot(column="mean_extreme_cold_days", cmap="cividis_r", linewidth=0, legend=True, leg
    ax.set_title("Extreme-cold winter days are concentrated in the northern U.S.", fontsize=13); ax.axis("
    fig.text(0.01, 0.01, f"Source: {config.name}; days with daily minimum temperature below 0°F, averaged (
    fig.tight_layout(rect=[0, 0.03, 1, 1])
    extreme_cold_map_path = save_tier2_figure(fig, f"{config.name.lower()}_extreme_cold_day_count.pdf")
    extreme_cold.to_csv(TIER2_TABLES_DIR / f"{config.name.lower()}_county_extreme_cold_days.csv", index=False)

```

Extreme-cold winter days are concentrated in the northern U.S.



Source: PRISM; days with daily minimum temperature below 0°F, averaged over complete DJF winters. This is not a full Winter Severity Index.

Saved /mnt/data_d/Dropbox/Research/AnimalCollisionsWeather/figures/weather/tier2/prism_extreme_cold_day_count.pdf

19. Define anomalously warm winters

Specify and justify the provisional anomaly definition before producing anomaly exhibits. Document the winter metric, county-specific reference distribution, treatment of the long-run trend, threshold, minimum coverage, and whether results are expressed as counts or shares. Candidate definitions require Eyal's confirmation before being treated as final.

```
In [55]: ANOMALY_SD_THRESHOLD = 1.5
ANOMALY_MIN_WINTERS = 30
ANOMALY_DEFINITION_STATUS = "PROVISIONAL - requires Eyal's confirmation"

def classify_warm_anomalies(frame, threshold=ANOMALY_SD_THRESHOLD, min_winters=ANOMALY_MIN_WINTERS):
    """Flag warm shocks relative to each county's detrended full-sample distribution."""
    pieces = []
    for geoid, county in frame.groupby("geoid"):
        county = county.dropna(subset=["winter_year", "mean_temp_c"]).sort_values("winter_year").copy()
        if len(county) < min_winters or county["winter_year"].nunique() < 2:
            continue
        slope, intercept = np.polyfit(county["winter_year"], county["mean_temp_c"], 1)
        county["fitted_temp_c"] = intercept + slope * county["winter_year"]
        county["detrended_anomaly_c"] = county["mean_temp_c"] - county["fitted_temp_c"]
        residual_sd = county["detrended_anomaly_c"].std(ddof=1)
        county["anomaly_z"] = county["detrended_anomaly_c"] / residual_sd if residual_sd > 0 else np.nan
        county["anomalously_warm"] = county["anomaly_z"].gt(threshold)
        county["county_trend_c_per_decade"] = slope * 10
        pieces.append(county)
    return pd.concat(pieces, ignore_index=True) if pieces else pd.DataFrame()

warm_anomalies = classify_warm_anomalies(county_winter)
warm_anomalies.to_csv(TIER2_TABLES_DIR / f"{config.name.lower()}_county_winter_warm_anomalies_provisional.csv")
anomaly_definition = pd.DataFrame([{"status": ANOMALY_DEFINITION_STATUS, "metric": "county-winter mean temperature"}])
anomaly_definition.to_csv(TIER2_TABLES_DIR / f"{config.name.lower()}_anomaly_definition_provisional.csv", index=False)
display(anomaly_definition)
```

	status	metric	reference_distribution	trend_treatment	threshold	minimum_complete_winters	frequency_measure
0	PROVISIONAL - requires Eyal's confirmation	county- winter mean temperature	county-specific full sample	linear county trend removed	> 1.5 residual SD	30	share of eligible winters

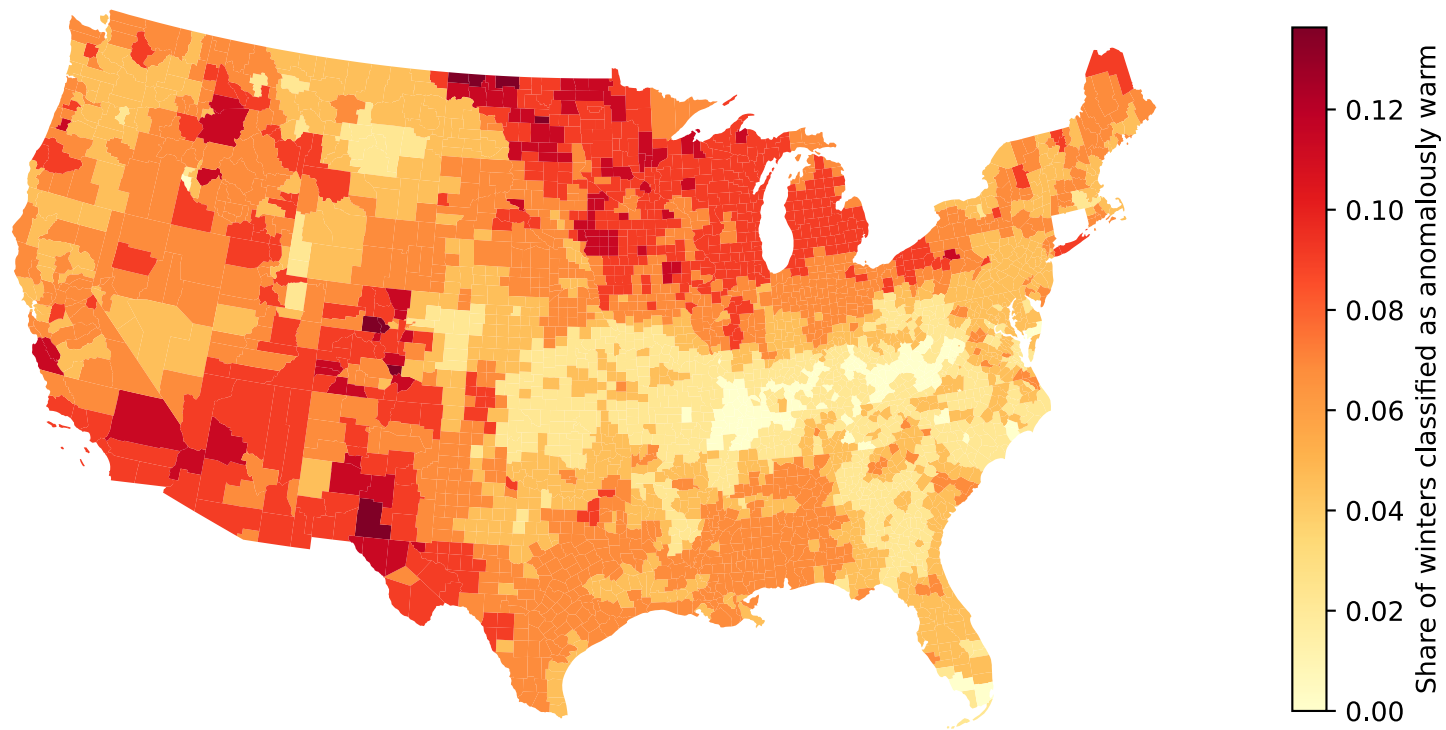
20. Anomalous-warm-winter frequency choropleth

Map the number or share of winters classified as anomalously warm within each county. This exhibit captures the geographic concentration of short-run warm shocks rather than general variability or long-run warming. Clearly label the anomaly definition

as provisional or confirmed.

```
In [56]: anomaly_frequency = (warm_anomalies.groupby(["geoid", "state_fips", "county_name"], as_index=False).agg(
    if not anomaly_frequency.empty: anomaly_frequency["anomalous_warm_share"] = anomaly_frequency["anomalous_w
    anomaly_frequency.to_csv(TIER2_TABLES_DIR / f"{config.name.lower()}_county_anomalous_warm_winter_frequency_
    anomaly_map = county_geometry.merge(anomaly_frequency, on="geoid", how="inner", validate="one_to_one").to_
    if anomaly_map.empty:
        anomaly_frequency_map_path = save_tier2_placeholder(f"{config.name.lower()}_anomalous_warm_winter_frequ
    else:
        fig, ax = plt.subplots(figsize=(8.5, 5.2))
        anomaly_map.plot(column="anomalous_warm_share", cmap="YlOrRd", vmin=0, linewidth=0, legend=True, legend
        ax.set_title("Frequency of anomalously warm winters", fontsize=14); ax.axis("off")
        fig.text(0.01, 0.01, f"{ANOMALY_DEFINITION_STATUS}. County-specific detrended residual > {ANOMALY_SD_TI
        fig.tight_layout(rect=[0, 0.03, 1, 1])
        anomaly_frequency_map_path = save_tier2_figure(fig, f"{config.name.lower()}_anomalous_warm_winter_frequ
```

Frequency of anomalously warm winters



PROVISIONAL - requires Eyal's confirmation. County-specific detrended residual > 1.5 SD; at least 30 complete winters.

Saved /mnt/data_d/Dropbox/Research/AnimalCollisionsWeather/figures/weather/tier2/prism_anomalously_winter_frequency_provisional.pdf

21. Anomalous-winter time-series exhibit

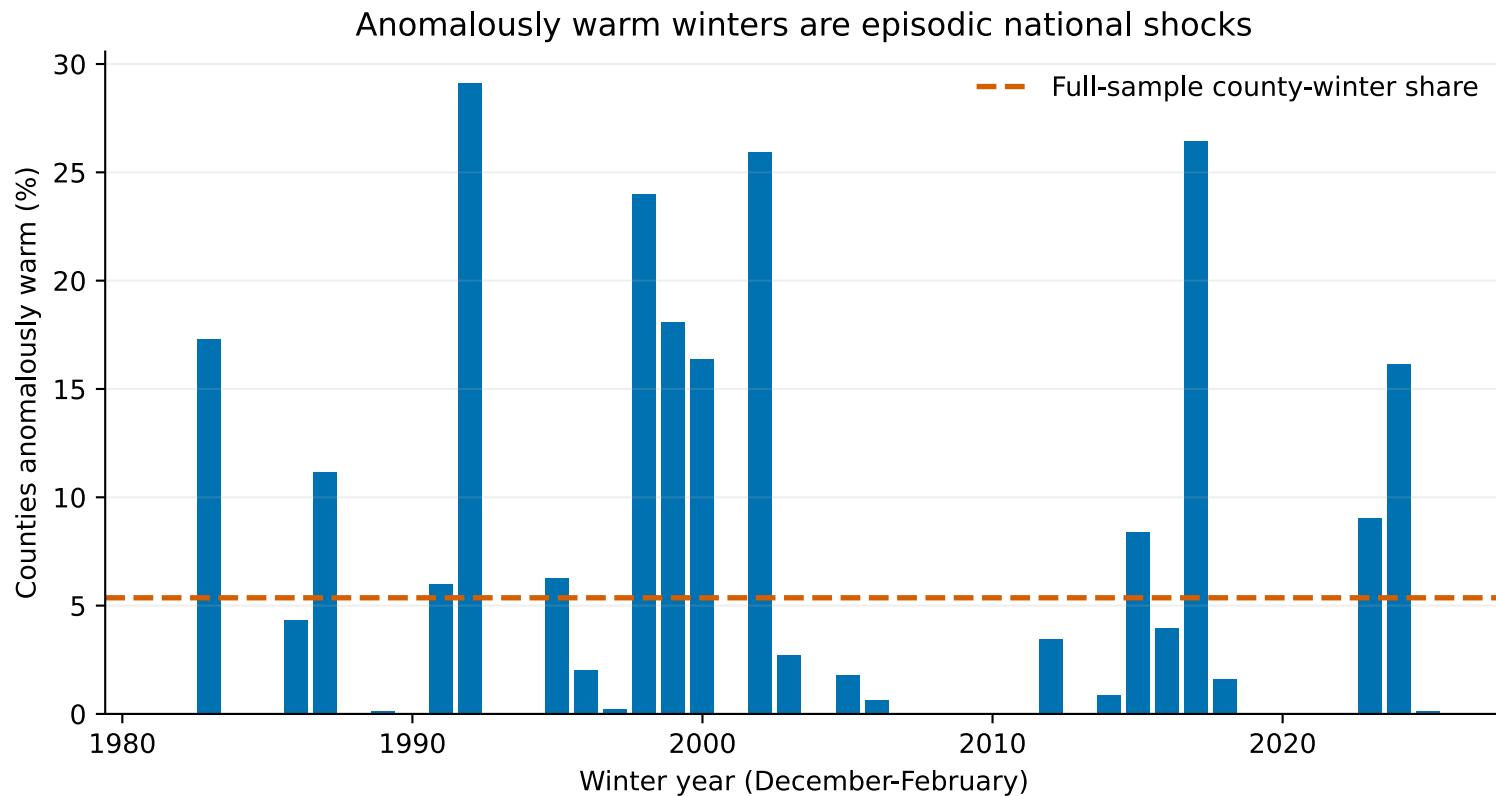
Show the national or state-level frequency of anomalously warm county-winters over time. Use this figure to distinguish unusual short-run shocks from the gradual long-run warming trend.

```
In [57]: anomaly_time = (warm_anomalies.groupby("winter_year", as_index=False).agg(anomalous_counties=("anomalously_
if not anomaly_time.empty: anomaly_time["anomalous_county_share"] = anomaly_time["anomalous_counties"] / an
anomaly_time.to_csv(TIER2_TABLES_DIR / f"{config.name.lower()}_annual_anomalously_winter_frequency_prov:
```

```

if anomaly_time.empty:
    anomaly_time_path = save_tier2_placeholder(f"{config.name.lower()}_anomalous_warm_winter_time_series_p
else:
    fig, ax = plt.subplots(figsize=(8.2, 4.8))
    ax.bar(anomaly_time["winter_year"], anomaly_time["anomalous_county_share"] * 100, color=TIER2_COLORS["
    ax.axhline(100 * warm_anomalies["anomalously_warm"].mean(), color=TIER2_COLORS["trend"], lw=2, ls="--"
    ax.set(title="Anomalously warm winters are episodic national shocks", xlabel="Winter year (December-Fel
    ax.grid(axis="y", alpha=0.2); ax.spines[["top", "right"]].set_visible(False); ax.legend(frameon=False)
    ax.text(0, -0.22, f"{ANOMALY_DEFINITION_STATUS}. Detrending separates short-run shocks from gradual co
    fig.tight_layout()
    anomaly_time_path = save_tier2_figure(fig, f"{config.name.lower()}_anomalous_warm_winter_time_series_p

```



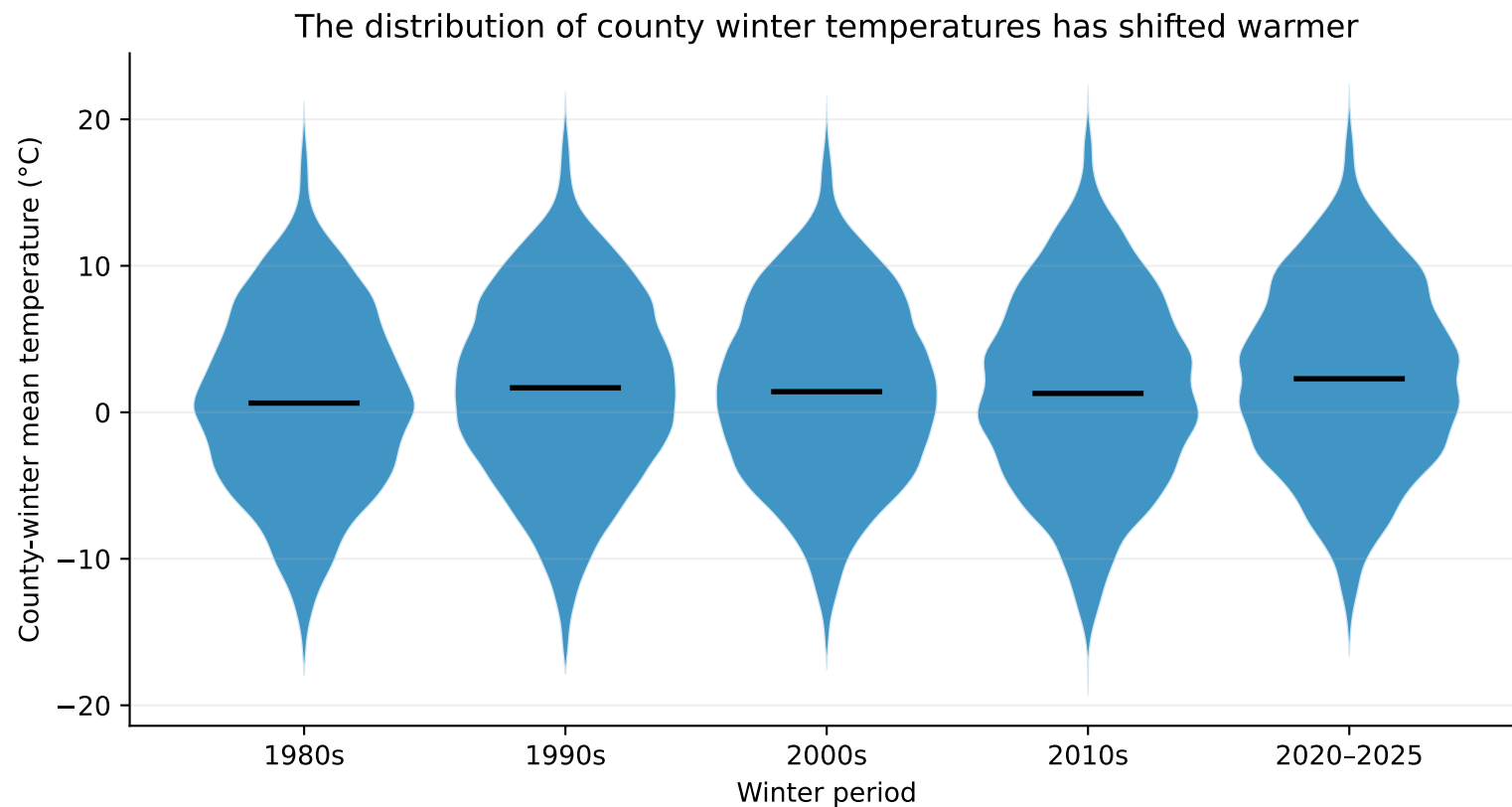
PROVISIONAL - requires Eyal's confirmation. Detrending separates short-run shocks from gradual county warming.

Saved /mnt/data_d/Dropbox/Research/AnimalCollisionsWeather/figures/weather/tier2/prism_anomalous_warm_winter_time_series_provisional.pdf

22. Polished decade-distribution exhibit

Select the most informative national, regional, or state examples from the Tier 1 decade-distribution plots. Use them to demonstrate whether climate change shifts the location, spread, or shape of the winter-temperature distribution.

```
In [58]: decade_order = [d for d in ["1980s", "1990s", "2000s", "2010s", "2020-2025"] if d in set(county_winter["decade"])]
decade_values = [county_winter.loc[county_winter["decade"].eq(d), "mean_temp_c"].dropna().values for d in decade_order]
if len(decade_order) < 2:
    decade_polished_path = save_tier2_placeholder(f"{config.name.lower()}_national_winter_temperature_distribution.png")
else:
    fig, ax = plt.subplots(figsize=(8.2, 4.8))
    violins = ax.violinplot(decade_values, positions=np.arange(len(decade_order)), showmeans=False, showmedians=True)
    for body in violins["bodies"]: body.set_facecolor(TIER2_COLORS["annual"]); body.set_edgecolor("white")
    violins["cmedians"].set_color("black"); violins["cmedians"].set_linewidth(2)
    ax.set_xticks(np.arange(len(decade_order)), decade_order); ax.set(title="The distribution of county winter temperatures by decade")
    ax.grid(axis="y", alpha=0.2); ax.spines[["top", "right"]].set_visible(False)
    ax.text(0, -0.20, f"Source: {config.name}; all complete county-winters. Black bars mark medians; violin plots show distributions.")
    fig.tight_layout(); decade_polished_path = save_tier2_figure(fig, f"{config.name.lower()}_national_winter_temperature_distribution.png")
```



Source: PRISM; all complete county-winters. Black bars mark medians; violin width shows density.

Saved /mnt/data_d/Dropbox/Research/AnimalCollisionsWeather/figures/weather/tier2/prism_national_winter_temperature_distributions_by_decade.pdf

23. Optional targeted PRISM–ERA5 comparison

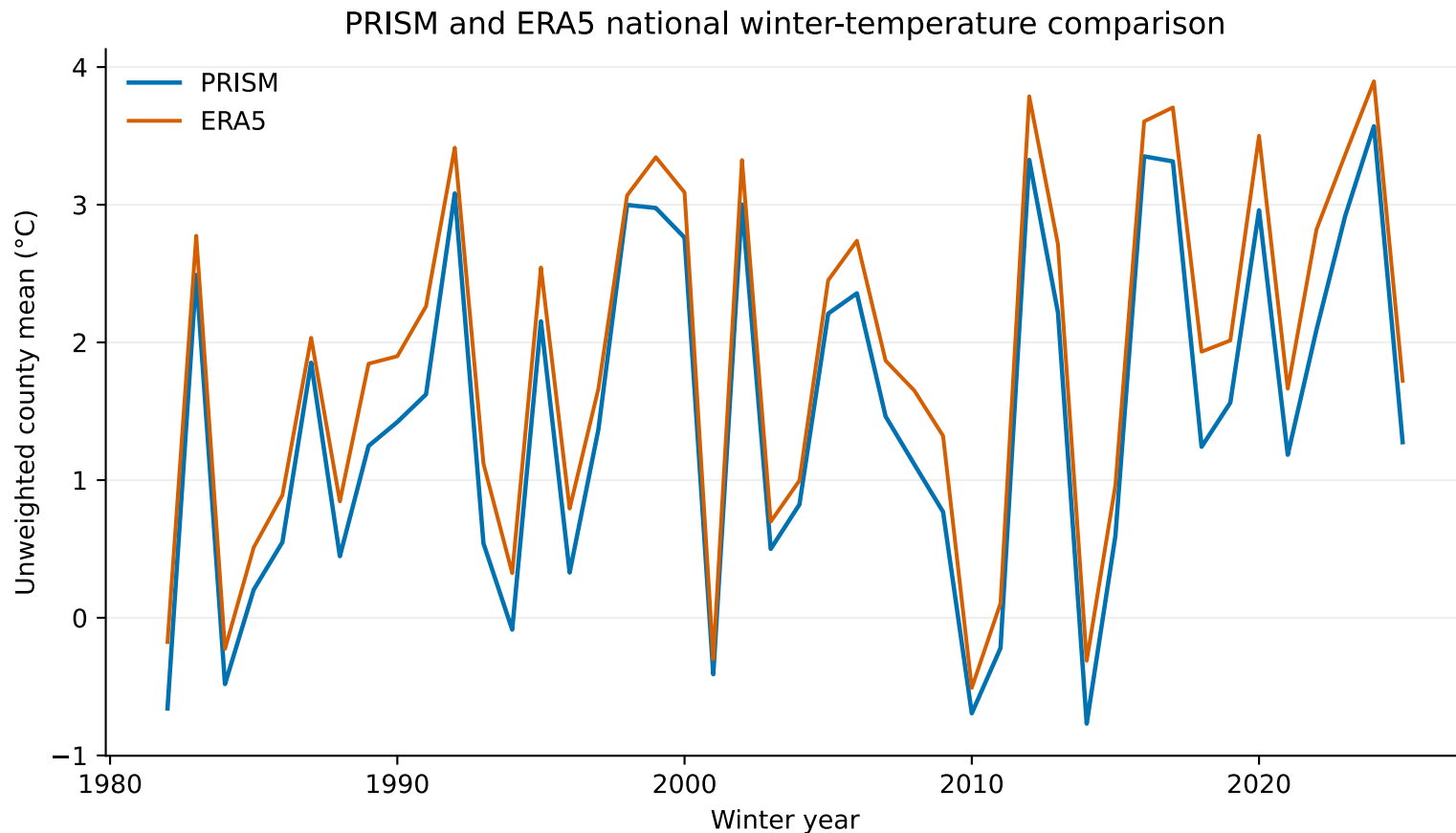
If needed, compare selected PRISM and ERA5 measures to assess robustness and agreement. Prioritize targeted comparisons over duplicating the complete PRISM exhibit suite.

```
In [59]: # ERA5 is a targeted robustness check only. Skip cleanly when the local checkout lacks full ERA5 inputs.
era5_comparison_path = None
if ERA5_CONFIG.monthly_path.exists() and ERA5_CONFIG.derived_path.exists():
    era5_monthly, _ = load_weather_panel(ERA5_CONFIG)
    era5_winter, _, _ = construct_county_winter_panel(era5_monthly)
```

```

prism_national = county_winter.groupby("winter_year", as_index=False)["mean_temp_c"].mean().rename(column:
era5_national = era5_winter.groupby("winter_year", as_index=False)["mean_temp_c"].mean().rename(column:
comparison = prism_national.merge(era5_national, on="winter_year", validate="one_to_one")
comparison.to_csv(TIER2_TABLES_DIR / "prism_era5_national_winter_temperature_comparison.csv", index=False)
fig, ax = plt.subplots(figsize=(8.2, 4.8))
ax.plot(comparison["winter_year"], comparison["PRISM"], label="PRISM", color=TIER2_COLORS["annual"], lw=1)
ax.plot(comparison["winter_year"], comparison["ERA5"], label="ERA5", color=TIER2_COLORS["trend"], lw=1)
ax.set(title="PRISM and ERA5 national winter-temperature comparison", xlabel="Winter year", ylabel="Unw
fig.tight_layout(); era5_comparison_path = save_tier2_figure(fig, "prism_era5_national_winter_temperat
else:
print("Skipped optional PRISM-ERA5 comparison: full ERA5 monthly and derived files are not both present")

```



Saved /mnt/data_d/Dropbox/Research/AnimalCollisionsWeather/figures/weather/tier2/prism_era5_national_winter_temperature_comparison.pdf

24. Exhibit index, output manifest, and decisions for Eyal

End the report with an ordered exhibit index and a manifest of saved tables and figures. Summarize unresolved methodological choices—especially anomaly definition, weighting, comparison periods, and any questionable observations—for efficient PI review.

```
In [60]: exhibit_index = pd.DataFrame([
    {"order": 1, "section": 14, "exhibit": "National long-run warming", "path": national_warming_path},
    {"order": 2, "section": 15, "exhibit": "State long-run warming", "path": state_warming_path},
    {"order": 3, "section": 16, "exhibit": "Early versus recent temperature", "path": early_recent_map_path},
    {"order": 4, "section": 17, "exhibit": "Temperature change", "path": warming_change_map_path},
    {"order": 5, "section": 18, "exhibit": "Extreme-cold-day count", "path": extreme_cold_map_path},
    {"order": 6, "section": 20, "exhibit": "Warm-anomaly frequency (provisional)", "path": anomaly_frequency},
    {"order": 7, "section": 21, "exhibit": "Warm-anomaly time series (provisional)", "path": anomaly_time_series},
    {"order": 8, "section": 22, "exhibit": "Temperature distributions by decade", "path": decade_polished_
])
if era5_comparison_path is not None:
    exhibit_index.loc[len(exhibit_index)] = [9, 23, "Targeted PRISM-ERA5 comparison", era5_comparison_path]
exhibit_index["path"] = exhibit_index["path"].map(str)
exhibit_index.to_csv(TIER2_TABLES_DIR / f"{config.name.lower()}_tier2_exhibit_index.csv", index=False)

manifest_rows = [{"artifact_type": "figure", "path": str(path)} for path in tier2_outputs]
manifest_rows += [{"artifact_type": "table", "path": str(path)} for path in sorted(TIER2_TABLES_DIR.glob("
output_manifest = pd.DataFrame(manifest_rows).drop_duplicates().sort_values(["artifact_type", "path"])
output_manifest.to_csv(TIER2_TABLES_DIR / f"{config.name.lower()}_tier2_output_manifest.csv", index=False)

decisions_for_eyal = pd.DataFrame([
    {"decision": "Anomaly definition", "current_choice": f"Detrended county residual > {ANOMALY_SD_THRESHOLD}"},
    {"decision": "Geographic weighting", "current_choice": "Unweighted mean across counties", "status": "C"},
    {"decision": "Comparison periods", "current_choice": f"{EARLY_PERIOD[0]}-{EARLY_PERIOD[1]} versus {RECI"},
    {"decision": "Extreme-cold exhibit", "current_choice": "Mean days with TMIN < 0°F; not labeled WSI", "s"},
    {"decision": "ERA5 robustness exhibit", "current_choice": "Targeted national comparison only when full
])
decisions_for_eyal.to_csv(TIER2_TABLES_DIR / f"{config.name.lower()}_tier2_decisions_for_eyal.csv", index=False)
display(exhibit_index); display(output_manifest); display(decisions_for_eyal)
print(f"Tier 2 complete: {len(tier2_outputs)} vector-PDF exhibit files in {TIER2_FIGURES_DIR}")
```

order	section	exhibit	path
0	1	14	National long-run warming /mnt/data_d/Dropbox/Research/AnimalCollisionsW...
1	2	15	State long-run warming /mnt/data_d/Dropbox/Research/AnimalCollisionsW...
2	3	16	Early versus recent temperature /mnt/data_d/Dropbox/Research/AnimalCollisionsW...
3	4	17	Temperature change /mnt/data_d/Dropbox/Research/AnimalCollisionsW...
4	5	18	Extreme-cold-day count /mnt/data_d/Dropbox/Research/AnimalCollisionsW...
5	6	20	Warm-anomaly frequency (provisional) /mnt/data_d/Dropbox/Research/AnimalCollisionsW...
6	7	21	Warm-anomaly time series (provisional) /mnt/data_d/Dropbox/Research/AnimalCollisionsW...
7	8	22	Temperature distributions by decade /mnt/data_d/Dropbox/Research/AnimalCollisionsW...
8	9	23	Targeted PRISM-ERA5 comparison /mnt/data_d/Dropbox/Research/AnimalCollisionsW...

	artifact_type	path
5	figure	/mnt/data_d/Dropbox/Research/AnimalCollisionsW...
6	figure	/mnt/data_d/Dropbox/Research/AnimalCollisionsW...
8	figure	/mnt/data_d/Dropbox/Research/AnimalCollisionsW...
4	figure	/mnt/data_d/Dropbox/Research/AnimalCollisionsW...
0	figure	/mnt/data_d/Dropbox/Research/AnimalCollisionsW...
7	figure	/mnt/data_d/Dropbox/Research/AnimalCollisionsW...
1	figure	/mnt/data_d/Dropbox/Research/AnimalCollisionsW...
3	figure	/mnt/data_d/Dropbox/Research/AnimalCollisionsW...
2	figure	/mnt/data_d/Dropbox/Research/AnimalCollisionsW...
9	table	/mnt/data_d/Dropbox/Research/AnimalCollisionsW...
10	table	/mnt/data_d/Dropbox/Research/AnimalCollisionsW...
11	table	/mnt/data_d/Dropbox/Research/AnimalCollisionsW...
12	table	/mnt/data_d/Dropbox/Research/AnimalCollisionsW...
13	table	/mnt/data_d/Dropbox/Research/AnimalCollisionsW...
14	table	/mnt/data_d/Dropbox/Research/AnimalCollisionsW...
15	table	/mnt/data_d/Dropbox/Research/AnimalCollisionsW...
16	table	/mnt/data_d/Dropbox/Research/AnimalCollisionsW...

	decision	current_choice	status
0	Anomaly definition	Detrended county residual > 1.5 SD; minimum 30...	CONFIRM
1	Geographic weighting	Unweighted mean across counties	CONFIRM
2	Comparison periods	1982-1991 versus 2016-2025	CONFIRM
3	Extreme-cold exhibit	Mean days with TMIN < 0°F; not labeled WSI	DOCUMENTED
4	ERA5 robustness exhibit	Targeted national comparison only when full in...	OPTIONAL

Tier 2 complete: 9 vector-PDF exhibit files in /mnt/data_d/Dropbox/Research/AnimalCollisionsWeather/figures/weather/tier2

